
Flex 2 SDK User Manual v2.0

Hangzhou Xynova Technology Co., Ltd.

Aug 13, 2026

OVERVIEW

1	Overview	1
2	Disclaimer	2
3	Getting Started	4
4	Documentation	5
4.1	Examples	5
4.1.1	Introduction	5
4.1.2	What You Can Learn	5
4.1.3	Examples by Module	5
4.1.3.1	Communication	5
4.1.3.1.1	Initialize Communication Manager	5
4.1.3.1.2	Register CAN Drivers	9
4.1.3.1.3	Upgrade Firmware	13
4.1.3.2	Hand Device	14
4.1.3.2.1	Run Dexterous Hand Device	14
4.1.3.2.2	Product Information	16
4.1.3.2.3	Joint Control Example	18
4.1.3.2.4	Asynchronous Joint Control	20
4.1.3.2.5	Tactile Sensing	21
4.1.3.2.6	Tactile Taxel Sensing	23
4.1.3.2.7	Range Sensing	28
4.1.3.2.8	Error State	31
4.1.3.3	Logging	34
4.1.3.3.1	Create Logger Instance	34
4.1.3.4	C API	37
4.1.3.4.1	Dexterous Hand Control (C API)	37
4.1.4	Full Examples	40
4.1.4.1	Dexterous Hand — Servo Control	40
4.1.4.1.1	Overview	40
4.1.4.1.2	Features	40
4.1.4.1.3	Prerequisites	40
4.1.4.1.4	Execution Flow	40
4.1.4.1.5	Source Code	41
4.1.5	Usage Notes	45
4.2	API Reference	45
4.2.1	Overview	45
4.2.2	API Categories	45
4.2.2.1	C API	45

4.2.2.1.1	C API Reference	45
4.2.2.1.2	Communication Module (C API)	46
4.2.2.1.3	CAN Transport Module (C API)	52
4.2.2.1.4	Hand Control Module (C API)	58
4.2.2.1.5	Dexterous Hand (C API)	59
4.2.2.1.6	Infrastructure (C API)	79
4.2.2.1.7	Logging (C API)	90
4.2.2.1.8	Error Codes	99
4.2.2.2	C++ API	102
4.2.2.2.1	C++ API Reference	102
4.2.2.2.2	Communication (C++ API)	103
4.2.2.2.3	CAN Communication (C++ API)	109
4.2.2.2.4	Hand Control (C++ API)	111
4.2.2.2.5	Dexterous Hand (C++ API)	112
4.2.2.2.6	Infrastructure (C++ API)	122
4.2.2.2.7	Logging (C++ API)	123
4.2.2.2.8	Value Types (C++ API)	130
4.2.3	Notes	134
4.2.4	Usage	134
4.3	Glossary	135
5	Additional Resources	137
	Index	138

OVERVIEW

Xynova SDK provides the public C and C++ interfaces for controlling and monitoring Xynova dexterous hand devices. It combines a stable C ABI with modern C++ wrappers and supports communication, device control, logging, and CAN backend integration.

The SDK surface is organized around the following areas:

- **Communication:** manager, master/device lifecycle, and CAN transport setup.
- **Dexterous hand control:** support for the shared dexterous hand API and the Dexterous Hand device wrapper.
- **Infrastructure:** logging, error handling, and common value helpers.
- **Drivers:** SocketCAN and vendor-specific CAN backends.

The SDK targets Linux (x86_64 / aarch64) and controls Xynova Dexterous Hand devices over CAN (SocketCAN or a vendor driver such as ChuangXinCAN). Realtime scheduling (SCHED_FIFO) requires root privileges or CAP_SYS_NICE.

DISCLAIMER

Thank you for purchasing the Dexterous Hand Product (“the Product”) provided by Hangzhou Xynova Technology Co., Ltd. (“the Company”).

To safeguard your rights and interests, please read this Disclaimer carefully and ensure that you fully understand all of its terms and conditions, in particular those provisions set forth in bold, underlined, or otherwise conspicuous type. **BY USING THIS PRODUCT, YOU ACKNOWLEDGE THAT YOU HAVE READ, UNDERSTOOD, AND AGREE TO BE BOUND BY ALL TERMS AND CONDITIONS OF THIS DISCLAIMER. IF YOU DO NOT AGREE TO ANY PROVISION HEREOF, YOU MUST NOT USE THIS PRODUCT.**

If any provision of this Disclaimer is held to be invalid, illegal, or unenforceable by a court of competent jurisdiction, such invalidity, illegality, or unenforceability shall not affect the validity or enforceability of the remaining provisions, which shall remain in full force and effect.

ARTICLE I USE SPECIFICATIONS AND ASSUMPTION OF LIABILITY

You shall strictly adhere to and comply with the accompanying *User Manual* and all applicable laws, statutes, and regulations, and shall install and use this Product only in a safe environment and for its intended purpose.

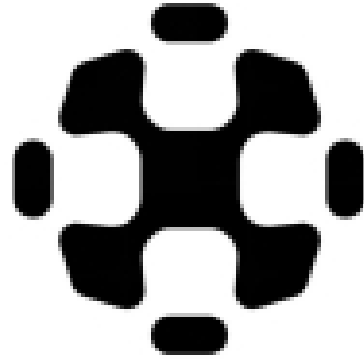
You shall bear sole responsibility for, and hereby assume all liability arising from, any loss, damage, or liability directly caused by any of the following acts or omissions (including without limitation):

1. failure to install, operate, maintain, or store the Product in strict accordance with the instructions and specifications set forth in the *User Manual*;
2. any unauthorized modification, alteration, disassembly, or repair of the Product;
3. use of the Product for any purpose or in any environment other than that for which it was designed and intended;
4. use of the Product in conjunction with any accessories, parts, components, or consumables that have not been manufactured, designated, or approved by the Company; or
5. any other act or omission constituting gross negligence or willful misconduct.

TO THE FULLEST EXTENT PERMITTED BY APPLICABLE LAW, THE COMPANY SHALL NOT BE LIABLE FOR ANY LOSS OR DAMAGE ARISING OUT OF OR IN CONNECTION WITH THE USER CONDUCT DESCRIBED ABOVE. NOTWITHSTANDING THE FOREGOING, NOTHING IN THIS DISCLAIMER SHALL EXCLUDE OR LIMIT THE COMPANY’ S LIABILITY FOR: (A) DEATH OR PERSONAL INJURY, OR PROPERTY DAMAGE, CAUSED BY DEFECTS IN THE COMPANY’ S PRODUCTS AS DEFINED UNDER THE PRODUCT QUALITY LAW OF THE PEOPLE’ S REPUBLIC OF CHINA AND OTHER APPLICABLE LAWS AND REGULATIONS; OR (B) ANY LIABILITY THAT CANNOT BE EXCLUDED OR LIMITED UNDER APPLICABLE MANDATORY LAW.

ARTICLE II INTELLECTUAL PROPERTY RIGHTS

Xynova



“Xynova” ; “曦诺未来” are the registered trademarks of the Company. All other product names, service marks, trade names, brand logos, and other indicate appearing in this Disclaimer or on the Product packaging are the property of their respective owners.

This Product and all related technical solutions are protected by patents, design patents, and other intellectual property rights under the laws of the People’ s Republic of China. All contents of this Disclaimer and the accompanying *User Manual*, including without limitation all text, charts, diagrams, and images, are the copyrighted works of the Company. **NO PERSON OR ENTITY SHALL REPRODUCE, REPRINT, TRANSLATE, ADAPT, DISTRIBUTE, OR OTHERWISE USE OR EXPLOIT ANY OF THE FOREGOING MATERIALS FOR COMMERCIAL PURPOSES WITHOUT THE COMPANY’ S PRIOR WRITTEN CONSENT.**

Hangzhou Xynova Technology Co., Ltd.

GETTING STARTED

1. Start from the public SDK interfaces for communication, device control, and logging.
2. Include the relevant headers in your application:

```
#include "xynova/cpp/communication/Manager.hpp"  
#include "xynova/cpp/hand/dexterous_hand/Device.hpp"  
#include "xynova/cpp/infra/log/LogManager.hpp"
```

Note

The C++ API requires a compiler with C++17 support.

3. Initialize the communication manager and configure a master/device pair.
4. Start the device and use the supported synchronous or asynchronous control APIs.
5. Configure logging early so runtime diagnostics are available.

For complete runnable workflows, see the *Examples* section —start with *Run Dexterous Hand Device* for manager initialization and device startup, or the end-to-end *Dexterous Hand —Servo Control* example.

DOCUMENTATION

The documentation is organized into SDK usage examples and API references:

4.1 Examples

4.1.1 Introduction

This section collects practical examples that mirror the implementation in the repository under the communication, hand-control, and infrastructure modules. Each example is intentionally focused so it can be adapted into a larger application.

4.1.2 What You Can Learn

The examples cover the following topics:

- Initialize the communication manager and configure a CAN master
- Register custom CAN drivers
- Upgrade device firmware
- Connect and run dexterous hand devices
- Read product information (model, serial number, versions, hand type)
- Use synchronous and asynchronous control interfaces
- Read region-level and taxel-level tactile feedback
- Read proximity (DToF) range measurements
- Monitor device error states (global warning/fault and joint errors)
- Configure and use the logging system
- Use the C API for a complete control flow
- Walk through a complete servo-control workflow

4.1.3 Examples by Module

4.1.3.1 Communication

4.1.3.1.1 Initialize Communication Manager

This example demonstrates how to initialize the **Xynova Communication Manager** using the C++ API and create a hand device connected through CAN.

Workflow

The initialization consists of the following steps:

1. Configure a CAN master
2. Configure slave and device
3. Register them to the manager
4. Initialize and start the master
5. Acquire the device handle

Configuration Notes

Master

- `name` – logical identifier of the *master*
- `cpu_index` – CPU binding. Use `-1` to disable binding (root only)
- `period_us` – control loop period (μ s)
- `comm.type` – currently only CAN is supported
- **`comm.sub_type` – driver backend**
 - `SocketCAN` : Linux SocketCAN
 - `ChuangXinCAN` : vendor driver
 - user-defined drivers can also be registered
- **`iface` – physical CAN interface identifier**
 - For `SocketCAN`, specify the Linux SocketCAN interface name, such as `can0` or `can1`.
 - For `ChuangXinCAN`, specify the vendor CAN device/channel identifier, such as `0.0` or `0.1`.
- `bitrate / dbitrate` – nominal / data bitrate (CAN-FD)
- `sample_point / dsample_point` – bit timing sample point
- `manager_can` – required for SocketCAN auto power on/off (root only)

Slave

- Represents the physical hardware (currently dexterous hand *slave*)
- `type` – slave implementation name, e.g. `DexterousHandSlave`
- `id` – slave identifier, `0xFFFFFFFF` (all Fs)
- Two timing profiles are available:

General profile

Used for normal configuration APIs (getters/setters)

- `user_max_timeout_us` – max timeout on the user side
- `user_poll_interval_us` – polling interval while waiting
- `execute_timeout_us` – backend execution timeout
- `execute_poll_interval_us` – backend retry interval
- `execute_max_retry` – maximum retry count

Fast profile

Used for high-frequency control APIs

- Same fields as above but optimized for low latency

Device

- Logical high-level *device* wrapper that binds: - one master - one or more slaves

Example Code

```
using namespace xynova;
using namespace xynova::communication;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

// -----
// Configure master (CAN)
// -----

MasterConfig master{};
master.name = "test_master";
master.cpu_index = -1;
master.period_us = 1000;

master.comm.type = CommType::Can;
master.comm.sub_type = "ChuangXinCAN";

master.comm.can.iface = "0.0";
master.comm.can.bitrate = 1'000'000;
master.comm.can.dbitrate = 5'000'000;
master.comm.can.sample_point = 0.75;
master.comm.can.dsampling_point = 0.75;

// -----
// Configure slave
// -----

SlaveConfig slave{};
slave.type = "DexterousHandSlave";
slave.id = static_cast<uint32_t>(-1); // ID = 0xFFFFFFFF (all Fs)

// General timing
slave.general_time_setting.user_max_timeout_us = 50 * 1000;
slave.general_time_setting.user_poll_interval_us = 1 * 1000;
```

(continues on next page)

(continued from previous page)

```

slave.general_time_setting.execute_timeout_us      = 5 * 1000;
slave.general_time_setting.execute_poll_interval_us = 10 * 1000;
slave.general_time_setting.execute_max_retry       = 3;

// Fast timing (control path)
slave.fast_time_setting.user_max_timeout_us       = 5 * 1000;
slave.fast_time_setting.user_poll_interval_us      = 100;
slave.fast_time_setting.execute_timeout_us         = 3 * 1000;
slave.fast_time_setting.execute_poll_interval_us   = 3 * 1000;
slave.fast_time_setting.execute_max_retry          = 0;

// -----
// Configure device
// -----

DeviceConfig device_cfg{};
device_cfg.type      = "DexterousHandDevice";
device_cfg.name      = "test_hand";
device_cfg.master_name = master.name;
device_cfg.slaves.push_back(slave);

// -----
// Configure manager
// -----

ManagerConfig cfg;
cfg.masters.push_back(master);
cfg.devices.push_back(device_cfg);

// -----
// Initialize and run
// -----

XynovaErrorCode ec;

auto& mgr = Manager::instance();

CHECK_EC(mgr.init(cfg));
CHECK_EC(mgr.run(master.name));

// -----
// Acquire device handle
// -----

auto hand = mgr.getDevice<xynova::hand::dexterous_hand::Device>(
    master.name,
    device_cfg.type,
    device_cfg.name);

if (!hand || !hand->valid())
{
    std::cerr << "[ERROR] device invalid" << std::endl;

```

(continues on next page)

(continued from previous page)

```

return -1;
}

```

Notes

- The manager owns all masters and devices inside the process.
- Always verify `device->valid()` before using the device.
- Manager follows the singleton pattern.

4.1.3.1.2 Register CAN Drivers

Overview

The Xynova SDK supports **pluggable CAN backends** via a driver abstraction layer.

Each CAN backend implements the low-level communication logic (e.g. SocketCAN, USB-CAN, PCIe-CAN, vendor SDK, or simulation).

Drivers are registered at runtime using:

```
XynovaCommunicationCan_registerDriver()
```

This enables the communication manager to dynamically select a CAN driver by its string type.

Typical use cases:

- Custom hardware adapters
- Vendor-specific CAN libraries
- Simulation backends
- Unit testing / mock drivers

Architecture

Driver integration follows this flow:

1. Implement `XynovaCommunicationCanVTable`
2. Provide all required callbacks
3. Register driver with a unique type name
4. Use the type name in `MasterConfig.comm.sub_type`

SDK Manager

```
lookup("socketcan")
```

```
VTable → create/open/send/recv
```

When to Implement a Custom Driver

Create a custom driver if you need:

- non-SocketCAN hardware
- proprietary vendor SDKs
- virtual CAN / simulator
- deterministic testing without real bus
- embedded or RTOS ports

Example: Minimal SocketCAN Driver

The following example shows how to implement and register a basic SocketCAN backend.

Step 1 —Implement Driver Callbacks

```
#include <unistd.h>
#include <cstring>
#include <sys/socket.h>
#include <linux/can.h>
#include <linux/can/raw.h>
#include <net/if.h>
#include <sys/ioctl.h>

#include "xynova/c/can_api.h"

struct SocketCanHandle
{
    int fd{-1};
};

static void* sc_create(const XynovaCommunicationCanConfig* cfg)
{
    auto* h = new SocketCanHandle{};
    return h;
}

static void sc_destroy(void* handle)
{
    delete static_cast<SocketCanHandle*>(handle);
}

static XynovaErrorCode sc_open(void* handle)
{
    auto* h = static_cast<SocketCanHandle*>(handle);

    h->fd = socket(PF_CAN, SOCK_RAW, CAN_RAW);
    if (h->fd < 0)
        return XYNova_ERROR_CODE_CAN_OPEN_FAILED;
}
```

(continues on next page)

(continued from previous page)

```

    return XYNova_ERROR_CODE_OK;
}

static XynovaErrorCode sc_close(void* handle)
{
    auto* h = static_cast<SocketCanHandle*>(handle);

    if (h->fd >= 0)
        close(h->fd);

    return XYNova_ERROR_CODE_OK;
}

static bool sc_isOpen(void* handle)
{
    auto* h = static_cast<SocketCanHandle*>(handle);
    return h->fd >= 0;
}

static XynovaErrorCode sc_send(
    void* handle,
    XynovaCommunicationCanFrameType,
    uint32_t id,
    const uint8_t* data,
    size_t len,
    uint32_t)
{
    auto* h = static_cast<SocketCanHandle*>(handle);

    struct can_frame frame{};
    frame.can_id = id;
    frame.can_dlc = len;
    std::memcpy(frame.data, data, len);

    write(h->fd, &frame, sizeof(frame));

    return XYNova_ERROR_CODE_OK;
}

static XynovaErrorCode sc_recv(
    void* handle,
    XynovaCommunicationCanFrameType*,
    uint32_t* id,
    uint8_t* data,
    size_t* len,
    uint32_t)
{

```

(continues on next page)

(continued from previous page)

```

    auto* h = static_cast<SocketCanHandle*>(handle);

    struct can_frame frame{};
    read(h->fd, &frame, sizeof(frame));

    *id = frame.can_id;
    *len = frame.can_dlc;
    std::memcpy(data, frame.data, *len);

    return XYNova_ERROR_CODE_OK;
}

static XynovaErrorCode sc_setFilters(
    void*, uint32_t, const XynovaCommunicationCanRecvFilter*)
{
    return XYNova_ERROR_CODE_OK;
}

```

Step 2 —Create VTable

```

static XynovaCommunicationCanVTable g_socketcan_vtable = {
    .create = sc_create,
    .destroy = sc_destroy,
    .open = sc_open,
    .close = sc_close,
    .isOpen = sc_isOpen,
    .setRecvFilters = sc_setFilters,
    .send = sc_send,
    .recv = sc_recv
};

```

Step 3 —Register Driver

```

XynovaCommunicationCan_registerDriver(
    "socketcan",
    &g_socketcan_vtable);

```

Step 4 —Use Driver in Manager

```

master_cfg.comm.type = CommType::Can;
master_cfg.comm.sub_type = "socketcan";    // match registered name

```

Best Practices

- Use **one-shot mode** for realtime systems
- Avoid blocking in send/recv
- Keep callbacks lock-free if possible
- Perform timeouts at driver layer

- Always validate buffer sizes

Summary

To integrate a new CAN backend:

1. Implement callbacks
2. Fill XynovaCommunicationCanVTable
3. Register using `registerDriver()`
4. Reference driver by name in config

The SDK will automatically route all CAN operations through your driver.

4.1.3.1.3 Upgrade Firmware

This example demonstrates how to upgrade the firmware of a slave device through the Communication Manager, using:

- `Manager::upgradeFirmware(master_name, slave_id, file)`

Prerequisites

- The manager must be initialized and the master must be running before the upgrade is started (see [Initialize Communication Manager](#)).

Example Code

```
using namespace xynova;
using namespace xynova::communication;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
            << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Manager must be initialized and running first
// (see the "Initialize Communication Manager" example)
// =====

auto& mgr = Manager::instance();

// =====
// Upgrade the firmware of the target slave
// =====

const uint32_t slave_id = static_cast<uint32_t>(-1); // ID = 0xFFFFFFFF
```

(continues on next page)

(continued from previous page)

```
const std::string firmware_file = "/path/to/firmware.bin";

CHECK_EC(mgr.upgradeFirmware(
    master.name,          // master name
    slave_id,             // target slave identifier
    firmware_file));     // firmware file path
```

Notes

- `master_name` must match the name of a configured and running master.
- `slave_id` identifies the target slave; adjust it to the actual device configuration.
- The firmware file format and the behavior of the device during the upgrade follow the device-side definition.
- See *Initialize Communication Manager* for the complete manager initialization and configuration flow.

4.1.3.2 Hand Device

4.1.3.2.1 Run Dexterous Hand Device

This guide explains how to start and stop a **Dexterous Hand device** through the Communication Manager.

It focuses on the interfaces currently available for Dexterous Hand users:

- `run()` / `stop()`
- `valid()` for handle validation

Workflow

1. Configure the CAN master and slave timing parameters
2. Register the device
3. Initialize and run the manager
4. Acquire the device handle
5. Start the hand device communication with `run()`
6. Stop it with `stop()`

Configuration Notes

The full manager / master / slave configuration (including the general and fast timing profiles) is described in *Initialize Communication Manager*. This example reuses that setup and only highlights the device startup flow.

Example Code

```
using namespace xynova;
using namespace xynova::communication;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
```

(continues on next page)

(continued from previous page)

```

        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

// =====
// Configure master / slave / device / manager
// (see the "Initialize Communication Manager" example)
// =====

MasterConfig master{};
master.name = "test_master";
master.cpu_index = -1;
master.period_us = 1000;

master.comm.type = CommType::Can;
master.comm.sub_type = "ChuangXinCAN";

master.comm.can.iface = "0.0";
master.comm.can.bitrate = 1'000'000;
master.comm.can.dbitrate = 5'000'000;
master.comm.can.sample_point = 0.75;
master.comm.can.dsampling_point = 0.75;

SlaveConfig slave{};
slave.type = "DexterousHandSlave";
slave.id = static_cast<uint32_t>(-1); // ID = 0xFFFFFFFF (all Fs)

DeviceConfig device_cfg{};
device_cfg.type = "DexterousHandDevice";
device_cfg.name = "test_hand";
device_cfg.master_name = master.name;
device_cfg.slaves.push_back(slave);

ManagerConfig cfg;
cfg.masters.push_back(master);
cfg.devices.push_back(device_cfg);

// =====
// Initialize and run
// =====

XynovaErrorCode ec;

auto& mgr = Manager::instance();

CHECK_EC(mgr.init(cfg));
CHECK_EC(mgr.run(master.name));

// =====
// Acquire and start device

```

(continues on next page)

(continued from previous page)

```
// =====
auto hand = mgr.getDevice<xynova::hand::dexterous_hand::Device>(
    master.name,
    device_cfg.type,
    device_cfg.name);

if (!hand || !hand->valid())
{
    std::cerr << "[ERROR] device invalid" << std::endl;
    return -1;
}

CHECK_EC(hand->run());

// ... control the hand ...

CHECK_EC(hand->stop());
```

Notes

- The manager owns the configured masters and devices.
- Always check `device->valid()` before use.
- The current Dexterous Hand interface set is limited to the APIs listed above.
- See *Initialize Communication Manager* for the complete manager initialization and configuration flow.

4.1.3.2.2 Product Information

This example demonstrates how to read the **product-related information** from a Dexterous Hand device, covering:

- `getProductModel()` – product model string
- `getProductSN()` – product serial number
- `getHardwareVersion()` – hardware version string
- `getSoftwareVersion()` – software version string
- `getHandType()` – left / right hand type

These interfaces are useful for device identification, version tracking, and compatibility checks before starting the control workflow.

Workflow

1. Acquire the device handle through the Communication Manager
2. Start the device with `run()`
3. Read the product model and serial number
4. Read the hardware / software version strings
5. Read the hand type and map it to `RIGHT` / `LEFT`

Concept Notes

The following device methods are used:

- `getProductModel(std::string &model)`
- `getProductSN(std::string &sn)`
- `getHardwareVersion(std::string &version)`
- `getSoftwareVersion(std::string &version)`
- `getHandType(HandType &type)`

The hand type is reported as a `HandType` value:

- `XYNOVA_DEXTEROUS_HAND_TYPE_UNKNOWN` – unknown hand type
- `XYNOVA_DEXTEROUS_HAND_TYPE_RIGHT` – right hand
- `XYNOVA_DEXTEROUS_HAND_TYPE_LEFT` – left hand

All version and model strings are returned through caller-provided `std::string` buffers.

Example Code

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
            << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Read product model and serial number
// =====

std::string product_model;
std::string product_sn;

CHECK_EC(device->getProductModel(product_model));
CHECK_EC(device->getProductSN(product_sn));

std::cout << "Product model : " << product_model << std::endl;
std::cout << "Serial number : " << product_sn << std::endl;

// =====
// Read hardware / software versions
// =====

std::string hw_version;
```

(continues on next page)

(continued from previous page)

```

std::string sw_version;

CHECK_EC(device->getHardwareVersion(hw_version));
CHECK_EC(device->getSoftwareVersion(sw_version));

std::cout << "Hardware ver  : " << hw_version << std::endl;
std::cout << "Software ver  : " << sw_version << std::endl;

// =====
// Read hand type
// =====

HandType hand_type = XYNOVA_DEXTEROUS_HAND_TYPE_UNKNOWN;

CHECK_EC(device->getHandType(hand_type));

switch (hand_type)
{
case XYNOVA_DEXTEROUS_HAND_TYPE_RIGHT:
    std::cout << "Hand type      : RIGHT" << std::endl;
    break;
case XYNOVA_DEXTEROUS_HAND_TYPE_LEFT:
    std::cout << "Hand type      : LEFT" << std::endl;
    break;
default:
    std::cout << "Hand type      : UNKNOWN" << std::endl;
    break;
}

```

Notes

- The device handle must be acquired and started before calling the product information interfaces.
- Always check the returned `XynovaErrorCode` to detect communication failures or device not-ready conditions.
- See *Run Dexterous Hand Device* for the complete manager initialization and device startup flow.

4.1.3.2.3 Joint Control Example

This example demonstrates the currently supported **joint-level control** flow for the dexterous hand, including:

- setting the control mode
- reading current joint states
- constructing a joint angle command
- sending a synchronous angle command

Workflow

1. Set the joint control mode to `ANGLE`.
2. Read current joint angles.
3. Build a new `AngleControl` command.

4. Send the command with `setJointAngle()`.

Concept Notes

The current implementation exposes the following supported device methods:

- `run() / stop()`
- `setJointControlMode(mode)`
- `setJointAngle(angle)`
- `setJointAngleAsync(angle)`
- `getJointAngle(states, timestamp)`

Each joint value is stored in an `AngleControl` or `AngleStates` object. The test code uses a simple loop to fill the values for all joints.

Example Code

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
            << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

ControlMode mode = XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_ANGLE;

// setJointControlMode() applies to all joints
CHECK_EC(device->setJointControlMode(mode));

AngleStates states{};
uint64_t ts = 0;

CHECK_EC(device->getJointAngle(states, ts));

AngleControl pos{};
for (size_t i = 0; i < JOINT_NUM; ++i)
{
    pos[i].setId(static_cast<JointID>(i + 1));
    pos[i].setValue(0.5236f);
}

CHECK_EC(device->setJointAngle(pos));
```

Notes

- The device handle must be acquired and started before joint control.
- `setJointControlMode()` applies to all joints.
- See [Run Dexterous Hand Device](#) for the complete manager initialization and device startup flow.

4.1.3.2.4 Asynchronous Joint Control

This example demonstrates the currently supported asynchronous control flow for the dexterous hand.

The implementation currently provides the following relevant methods:

- `setJointControlMode(mode)`
- `setJointAngleAsync(angle)`
- `getJointAngle(states, timestamp)`

This example follows the same pattern as the test case in the repository.

Workflow

1. Set the joint control mode to `ANGLE`.
2. Read the current joint states.
3. Build a new `AngleControl` command.
4. Send it with `setJointAngleAsync()`.

Example Code

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
    do { \
        ec = (expr); \
        if (ec != XYNOVA_ERROR_CODE_OK) { \
            std::cerr << "[ERROR] " << #expr \
                << " failed, ec=" << ec << std::endl; \
            return -1; \
        } \
    } while (0)

XynovaErrorCode ec;

ControlMode mode = XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_ANGLE;

CHECK_EC(device->setJointControlMode(mode));

AngleControl angle;
for (size_t i = 0; i < JOINT_NUM; ++i)
{
    angle[i].setId(static_cast<JointID>(i + 1));
    angle[i].setValue(-0.3491f);
}
```

(continues on next page)

(continued from previous page)

```

CHECK_EC(device->setJointAngleAsync(angle));

std::this_thread::sleep_for(std::chrono::seconds(1));

AngleStates states{};
uint64_t ts = 0;

CHECK_EC(device->getJointAngle(states, ts));

```

Notes

- The device handle must be acquired and started before joint control.
- `setJointAngleAsync()` queues the command and returns immediately; execution is handled by the communication backend.
- See *Run Dexterous Hand Device* for the complete manager initialization and device startup flow.

4.1.3.2.5 Tactile Sensing

This example demonstrates how to read the **tactile force / torque feedback** from a Dexterous Hand device, covering:

- `enableTactileWrench(bool)` – enable or disable tactile wrench reporting
- `getTactileWrench(TactileWrenchStates &states, uint64_t ×tamp_us)` – read the latest *wrench* data of all tactile regions

These interfaces are useful for contact-force monitoring, haptic feedback, and force-controlled operations.

Workflow

1. Acquire the device handle through the Communication Manager
2. Start the device with `run()`
3. Enable tactile wrench reporting with `enableTactileWrench(true)`
4. Periodically read the wrench states with `getTactileWrench()`
5. Parse the force / torque components of each region

Concept Notes

The dexterous hand exposes `TACTILE_NUM` (15) tactile regions. Each region is identified by a `TactileID` value, e.g.:

- `XYNOVA_DEXTEROUS_HAND_TACTILE_ID_TT1` – thumb fingertip
- `XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT1` – index fingertip
- `XYNOVA_DEXTEROUS_HAND_TACTILE_ID_PT1` – palm

Wrench dimension by region:

- 6D regions (`XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_6D`, full force + moment):
 - TT1 (thumb fingertip), FT1 (index fingertip), MT1 (middle fingertip), RT1 (ring fingertip), LT1 (little fingertip), PT1 (palm)
- 1D regions (`XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_1D`, normal force F_z only):

- TT2 (thumb pad)
- FT2 (index middle phalanx), FT3 (index proximal phalanx)
- MT2 (middle middle phalanx), MT3 (middle proximal phalanx)
- RT2 (ring middle phalanx), RT3 (ring proximal phalanx)
- LT2 (little middle phalanx), LT3 (little proximal phalanx)

Each `TactileWrench` entry contains:

- `timestamp_us` – sampling timestamp in microseconds (system monotonic clock)
- `region` – tactile region identifier
- `dimension` – valid wrench dimension, one of: - `XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_1D` (1) - `XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_3D` (3) - `XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_6D` (6)
- `wrench` – force / torque values as a 6-component array

The wrench vector is defined as:

```
[Fx, Fy, Fz, Mx, My, Mz]
```

where F^* are force components in Newtons (N) and M^* are moment (torque) components in Newton-meters (Nm).

Note

Tactile wrench reporting must be enabled with `enableTactileWrench(true)` before `getTactileWrench()` can return data.

Example Code

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Enable tactile wrench reporting
// =====

CHECK_EC(device->enableTactileWrench(true));

// =====
// Periodically read wrench states
```

(continues on next page)

(continued from previous page)

```
// =====
TactileWrenchStates states{};
uint64_t ts = 0;

CHECK_EC(device->getTactileWrench(states, ts));

// =====
// Parse force / torque of each tactile region
// =====

for (size_t i = 0; i < TACTILE_NUM; ++i)
{
    const auto &w = states[i];

    const float fx = w.wrench.value()[0];
    const float fy = w.wrench.value()[1];
    const float fz = w.wrench.value()[2];
    const float mx = w.wrench.value()[3];
    const float my = w.wrench.value()[4];
    const float mz = w.wrench.value()[5];

    std::cout << "region=" << static_cast<uint32_t>(w.region)
               << " dim=" << static_cast<uint32_t>(w.dimension)
               << " F=(" << fx << ", " << fy << ", " << fz << ") N"
               << " M=(" << mx << ", " << my << ", " << mz << ") Nm"
               << std::endl;
}
```

Notes

- The device handle must be acquired and started before enabling tactile wrench reporting.
- Calling `enableTactileWrench(true)` is required before the wrench data becomes available.
- The returned `wrench` array always has 6 components; unused components (depending on dimension) should be ignored.
- For 1D regions, only `wrench[0]` (Fz, normal force) contains valid data; the other components should be ignored. For 6D regions, all 6 components are valid.
- See *Run Dexterous Hand Device* for the complete manager initialization and device startup flow.

4.1.3.2.6 Tactile Taxel Sensing

This example demonstrates how to read the **detailed tactile taxel data** from a Dexterous Hand device, covering:

- `enableTactileTaxel(bool)` – enable or disable tactile *taxel* reporting
- `getTactileTaxel(TactileTaxelStates &states, uint64_t ×tamp_us)` – read the taxel-level force data of all tactile regions

Unlike the region-level *Tactile Sensing* (*wrench*) feedback, taxel data provides the force measured at every individual sensing element of the tactile array.

Sensor Array Overview

The full-hand tactile array consists of **15 modules** covering five fingers and the palm, with **312 taxels** in total:

Region ID	Module	Position	Force type	Taxels
0x01	TT1	Thumb fingertip	3D	33
0x02	TT2	Thumb pad	1D	12
0x03	FT1	Index fingertip	3D	24
0x04	FT2	Index middle phalanx	1D	8
0x05	FT3	Index proximal phalanx	1D	8
0x06	MT1	Middle fingertip	3D	24
0x07	MT2	Middle middle phalanx	1D	8
0x08	MT3	Middle proximal phalanx	1D	8
0x09	RT1	Ring fingertip	3D	24
0x0A	RT2	Ring middle phalanx	1D	8
0x0B	RT3	Ring proximal phalanx	1D	8
0x0C	LT1	Little fingertip	3D	19
0x0D	LT2	Little middle phalanx	1D	8
0x0E	LT3	Little proximal phalanx	1D	8
0x0F	PT1	Palm	3D	112

Naming convention: T = thumb, F = index, M = middle, R = ring, L = little, P = palm; 01 = fingertip, 02 = phalanx, 03 = proximal phalanx.

Left / Right Hand Mirroring

The product array contains **16 modules / 424 taxels** in total: the palm sensing area is provided by two separate modules (left palm and right palm, 112 taxels each). The SDK exposes **15 regions / 312 taxels** for the device currently connected, where the palm is reported through a single PT1 region (0x0F); the right-palm module is not exposed separately.

Spatial Layout

The following diagrams show the physical arrangement of the taxels within each module. The numbering is the product-side 1-based numbering used to locate a taxel on the physical array; the correspondence with the `taxel_id` value returned by the device interface is reported by the device itself.

Numbering rules

Module type	First taxel	Numbering direction	Row-major order
Fingertip modules (3D force)	Bottom-right corner	Right to left, bottom to top	Start at the bottom-right, fill each row from right to left, then move to the row above
Pad / phalanx / root / palm modules (1D or 3D force)	Top-left corner	Left to right, top to bottom	Start at the top-left, fill each row from left to right, then move to the row below

Fingertip modules

TT1 thumb fingertip (0x01, 33 taxels, 7 rows x 9 columns grid):

```
Row 1:      33 32 31
Row 2:      30 29 28
Row 3:      27 26 25 24
Row 4:      23 22 21 20 19
Row 5:      18 17 16 15 14 13
Row 6:      12 11 10 9 8 7
Row 7:      6 5 4 3 2 1    <- #1 bottom-right
```

FT1 / MT1 / RT1 index / middle / ring fingertip (0x03 / 0x06 / 0x09, 24 taxels each, 7 rows x 6 columns grid):

```
Row 1:      24 23
Row 2:      22 21 20
Row 3:      19 18 17 16
Row 4:      15 14 13 12
Row 5:      11 10 9 8
Row 6:      7 6 5 4
Row 7:      3 2 1    <- #1 bottom-right
```

LT1 little fingertip (0x0C, 19 taxels, 6 rows x 6 columns grid):

```
Row 1:      19 18
Row 2:      17 16 15
Row 3:      14 13 12
Row 4:      11 10 9 8
Row 5:      7 6 5 4
Row 6:      3 2 1    <- #1 bottom-right
```

Pad / phalanx / root modules

TT2 thumb pad (0x02, 12 taxels, 2 rows x 6 columns grid):

```
Row 1:  1  2  3  4  5  6
Row 2:  7  8  9 10 11 12    <- #1 top-left
```

Phalanx / root modules (FT2 / FT3 / MT2 / MT3 / RT2 / RT3 / LT2 / LT3, 0x04 ~ 0x0E, 8 taxels each, 2 rows x 4 columns grid, identical layout for every module):

```
Row 1:  1  2  3  4
Row 2:  5  6  7  8    <- #1 top-left
```

Palm modules

PT1 left palm (0x0F, 112 taxels, 20 rows x 17 columns grid, . = empty grid position):

```
      1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17  <- column
r 1:  .  .  .  .  1  2  .  .  .  .  .  .  .  .  .  .  .
r 2:  3  4  5  6  7  8  9 10  .  .  .  .  .  .  .  .  .
r 3: 11 12 13 14 15 16  .  .  .  .  .  .  .  .  .  .  .
r 4: 17 18 19 20 21  .  .  .  .  .  .  .  .  .  .  .
r 5:  . 22 23 24 25  .  .  .  .  .  .  .  .  .  .  .
```

(continues on next page)

(continued from previous page)

r 6:	.	.	26	27	28	29	.	.	.	.	.	.	.	.	.	.	.
r 7:	.	.	.	.	30	31	.	.	.	.	.	.	.	.	.	.	.
r 8:	.	.	.	.	.	32	33	34	.	.	.	.	.	.	.	.	.
r 9:	.	.	.	.	.	.	35	36	.	.	.	.	.	.	.	.	.
r10:	.	.	.	.	.	.	.	37	.	.	.	.	.	.	.	.	.
r11:	.	.	.	.	.	.	.	38	39	.	.	.	.	.	.	.	.
r12:	.	.	.	.	.	.	.	.	40	41	42	43	44	.	.	.	.
r13:	.	.	.	.	.	.	.	.	45	46	47	48	49	.	.	.	.
r14:	.	.	.	.	.	.	.	.	50	51	52	53	54	55	.	.	.
r15:	.	.	.	.	.	.	.	.	56	57	58	59	60	61	62	.	.
r16:	.	.	.	.	.	.	.	.	63	64	65	66	67	68	69	70	.
r17:	.	.	.	.	.	.	.	.	71	72	73	74	75	76	77	78	79
r18:	.	.	.	.	.	.	80	81	82	83	84	85	86	87	88	89	90
r19:	.	.	.	.	.	.	91	92	93	94	95	96	97	98	99	100	101
r20:	.	.	.	.	.	.	102	103	104	105	106	107	108	109	110	111	112

Right palm (112 taxels, 20 rows x 17 columns grid). The right palm is the horizontal mirror of the left palm: each row contains the same number of taxels at mirrored column positions, numbered from left to right within each row (#1 starts at the top-right area):

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	<- column
r 1:	.	.	.	.	.	.	.	.	.	.	.	1	2	.	.	.	.	
r 2:	.	.	.	.	.	.	.	.	.	3	4	5	6	7	8	9	10	
r 3:	.	.	.	.	.	.	.	.	.	.	.	11	12	13	14	15	16	
r 4:	.	.	.	.	.	.	.	.	.	.	.	.	17	18	19	20	21	
r 5:	.	.	.	.	.	.	.	.	.	.	.	.	22	23	24	25	.	
r 6:	.	.	.	.	.	.	.	.	.	.	.	26	27	28	29	.	.	
r 7:	.	.	.	.	.	.	.	.	.	.	.	30	31	.	.	.	.	
r 8:	.	.	.	.	.	.	.	.	.	32	33	34	.	.	.	.	.	
r 9:	.	.	.	.	.	.	.	.	.	35	36	.	.	.	.	.	.	
r10:	.	.	.	.	.	.	.	.	.	37	.	.	.	.	.	.	.	
r11:	.	.	.	.	.	.	.	.	38	39	.	.	.	.	.	.	.	
r12:	.	.	.	.	40	41	42	43	44	.	.	.	.	.	.	.	.	
r13:	.	.	.	.	45	46	47	48	49	.	.	.	.	.	.	.	.	
r14:	.	.	.	50	51	52	53	54	55	.	.	.	.	.	.	.	.	
r15:	.	.	.	56	57	58	59	60	61	62	.	.	.	.	.	.	.	
r16:	.	.	63	64	65	66	67	68	69	70	.	.	.	.	.	.	.	
r17:	.	71	72	73	74	75	76	77	78	79	.	.	.	.	.	.	.	
r18:	80	81	82	83	84	85	86	87	88	89	90	.	.	.	.	.	.	
r19:	91	92	93	94	95	96	97	98	99	100	101	.	.	.	.	.	.	
r20:	102	103	104	105	106	107	108	109	110	111	112	.	.	.	.	.	.	

Force Data

Each taxel reports its force through the `wrench` field, a fixed 6-component array with unit **N (Newton)**:

- **1D taxels** (non-fingertip regions): single normal force component, stored at `wrench[0]`; `wrench[1..5]` are zero.
- **3D taxels** (fingertip and palm regions): three orthogonal components `wrench[0..2] = [Fx, Fy, Fz]`, where `Fz` is the normal force and `Fx/Fy` are the shear components; `wrench[3..5]` are zero.

Taxel feedback carries force data only —no moment (torque) components. The $M_x / M_y / M_z$ moments of the 6D wrench output (see *Tactile Sensing*) are region-level resultant values, not per-taxel outputs.

The force is reported as a raw magnitude converted to Newtons by the device; the exact scale and resolution follow the device-side definition.

Workflow

1. Acquire the device handle through the Communication Manager
2. Start the device with `run()`
3. Enable tactile taxel reporting with `enableTactileTaxel(true)`
4. Periodically read the taxel states with `getTactileTaxel()`
5. Parse the region, dimension, and force components of each taxel
6. Disable reporting with `enableTactileTaxel(false)` when done

Example Code

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Enable tactile taxel reporting
// =====

CHECK_EC(device->enableTactileTaxel(true));

// Give the device time to start publishing taxel data.
std::this_thread::sleep_for(std::chrono::milliseconds(100));

// =====
// Periodically read taxel states
//
// The C++ wrapper resizes `states` to the number of taxels
// reported by the device (up to TACTILE_TOTAL_TAXEL_NUM).
// =====

TactileTaxelStates states;
uint64_t ts = 0;

CHECK_EC(device->getTactileTaxel(states, ts));

// =====
```

(continues on next page)

(continued from previous page)

```

// Parse force of each taxel
// =====

for (const auto &t : states)
{
    std::cout << "region=" << static_cast<uint32_t>(t.region)
                << " taxel=" << t.taxel_id
                << " dim="    << static_cast<uint32_t>(t.dimension);

    if (t.dimension == XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_1D)
    {
        std::cout << " Fz=" << t.wrench.value()[0];
    }
    else
    {
        std::cout << " F=(" << t.wrench.value()[0] << ", "
                    << t.wrench.value()[1] << ", "
                    << t.wrench.value()[2] << ") ";
    }

    std::cout << std::endl;
}

// =====
// Disable tactile taxel reporting
// =====

CHECK_EC(device->enableTactileTaxel(false));

```

Notes

- The device handle must be acquired and started before enabling tactile taxel reporting.
- Calling `enableTactileTaxel(true)` is required before the taxel data becomes available.
- If no new data has arrived, `getTactileTaxel()` returns `XYNOVA_ERROR_CODE_DATA_OUT_OF_DATE`, but the cached data is still copied into the output buffer.
- Each taxel carries its own `timestamp_us` sampling timestamp in microseconds (system monotonic clock).
- The `taxel_id` is reported as provided by the device.
- The taxel data is returned in ascending region-ID order.
- See *Run Dexterous Hand Device* for the complete manager initialization and device startup flow, and *Tactile Sensing* for the region-level wrench feedback.

4.1.3.2.7 Range Sensing

This example demonstrates how to read the **proximity (DToF) distance measurements** from a Dexterous Hand device, covering:

- `enableRange(bool)` – enable or disable range reporting
- `getRange(RangeStates &states, uint64_t ×tamp_us)` – read the latest range map

These interfaces are useful for proximity detection, obstacle avoidance, and gesture-related sensing.

Range Map Layout

The range data is returned as a two-dimensional map of `RANGE_POINT_NUM` (16) points in a 4x4 grid, stored in row-major order:

- $\text{index} = y * \text{width} + x$
- `id` of each point is a 1-based sequence number following the transmission order ($\text{id} = \text{index} + 1$)

Transmission order (row-major, `id` shown):

	col 0	col 1	col 2	col 3	
row 0:	id 1	id 2	id 3	id 4	<- index 0~3
row 1:	id 5	id 6	id 7	id 8	<- index 4~7
row 2:	id 9	id 10	id 11	id 12	<- index 8~11
row 3:	id 13	id 14	id 15	id 16	<- index 12~15

Physical sensor layout (product specification, the first row faces the fingertip direction):

+---+---+---+---+	
0 4 8 12	<- row 0 (top, toward fingertips)
+---+---+---+---+	
1 5 9 13	<- row 1
+---+---+---+---+	
2 6 10 14	<- row 2
+---+---+---+---+	
3 7 11 15	<- row 3 (bottom, toward wrist)
+---+---+---+---+	

Mapping between the transmission order and the physical layout:

- `points[i]` is located at ($\text{row} = i / 4$, $\text{col} = i \% 4$)
- the physical pixel number of `points[i]` is $(i \% 4) * 4 + (i / 4)$ (physical number = $\text{col} * 4 + \text{row}$)
- the `id` reported by the API is the transmission sequence number, not the physical pixel number

The sensor mounting direction is identical on both left and right hand gloves (viewed from the palm side), so the layout above applies to both hands without mirroring.

Data Layout

- `RangeStates`: width / height / count / points; the SDK currently reports a 4x4 range map of 16 points.
- `RangePoint` fields: - `id` - 1-based point sequence number - `status` - measurement status (see below) - `distance` - distance in meters (m) - `timestamp_us` - timestamp in microseconds (system monotonic clock)

`RangePointStatus` values:

Status	Meaning
NO_TARGET	No target detected
WEAK_SIGNAL	Weak signal, measurement not reliable
VALID	Valid measurement
INVALID	Invalid measurement

Workflow

1. Acquire the device handle through the Communication Manager
2. Start the device with `run()`
3. Enable range reporting with `enableRange(true)`
4. Periodically read the range map with `getRange()`
5. Iterate the grid and evaluate each point by its `status` and `distance`
6. Disable range reporting with `enableRange(false)` when done

Example Code

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Enable range reporting
// =====

CHECK_EC(device->enableRange(true));

// Give the device time to start publishing range data.
std::this_thread::sleep_for(std::chrono::milliseconds(100));

// =====
// Periodically read the range map
// =====

RangeStates states(RANGE_POINT_NUM);
uint64_t ts = 0;

CHECK_EC(device->getRange(states, ts));

// =====
// Iterate the grid in row-major order
// =====

for (uint16_t y = 0; y < states.height; ++y)
{
    for (uint16_t x = 0; x < states.width; ++x)
    {
```

(continues on next page)

(continued from previous page)

```

const auto &p = states.points[y * states.width + x];

if (p.status == XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_VALID)
{
    std::cout << "(" << x << ", " << y << ") id=" << p.id
               << " distance=" << p.distance << " m"
               << std::endl;
}
else
{
    std::cout << "(" << x << ", " << y << ") id=" << p.id
               << " status="
               << static_cast<uint32_t>(p.status)
               << std::endl;
}
}
}

// =====
// Disable range reporting
// =====

CHECK_EC(device->enableRange(false));

```

Notes

- The device handle must be acquired and started before enabling range reporting.
- Calling `enableRange(true)` is required before the range data becomes available.
- If no new data has arrived, `getRange()` returns `XYNOVA_ERROR_CODE_DATA_OUT_OF_DATE`, but the cached data is still copied into the output buffer.
- The caller provides the point buffer; the C++ wrapper manages the buffer automatically.
- See *Run Dexterous Hand Device* for the complete manager initialization and device startup flow.

4.1.3.2.8 Error State

This example demonstrates how to read the **device error state** from a Dexterous Hand device, covering:

- `getErrorState()` – read the latest global warning, global fault, and joint error information

These interfaces are useful for runtime health monitoring, fault detection, and safe shutdown handling.

Error State Structure

The error state is reported through the `ErrorState` type:

- `timestamp_us` – error state timestamp in microseconds
- `warning_flags` – global warning flags
- `fault_flags` – global fault flags
- `joint_error_codes` – error code of each joint (`JOINT_NUM` entries)

Convenience helpers:

- `hasWarning()` – true when any global warning is reported
- `hasFault()` – true when any global fault is reported
- `joint(id)` – lookup the error state of a specific joint

Each joint error code is a set of bit flags defined by the `JointError` enum. Multiple errors can be present at the same time:

- `StallProtection` – motor stall protection
- `OverTemperature` – over temperature protection
- `OverCurrent` – over current protection
- `MotorAbnormal` – motor abnormal
- `Communication` – communication error
- `EncoderAbnormal` – encoder abnormal

Workflow

1. Acquire the device handle through the Communication Manager
2. Start the device with `run()`
3. Read the error state with `getErrorState()`
4. Check the global warning / fault flags
5. Inspect the joint error codes of the joints that report errors

Example Code

```
using namespace xynova::hand::dexterous_hand;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
            << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// Read the device error state
// =====

ErrorState state{};

CHECK_EC(device->getErrorState(state));

// =====
// Check global warning / fault flags
```

(continues on next page)

(continued from previous page)

```
// =====
if (state.hasFault())
{
    std::cout << "Global fault : 0x"
                << std::hex << state.fault_flags << std::dec
                << std::endl;
}

if (state.hasWarning())
{
    std::cout << "Global warn  : 0x"
                << std::hex << state.warning_flags << std::dec
                << std::endl;
}

// =====
// Inspect joint error codes (print only joints with errors)
// =====

for (size_t i = 0; i < JOINT_NUM; ++i)
{
    const auto &joint_error = state.joint_error_codes[i];

    if (joint_error.error_code.ok())
        continue;

    std::cout << "Joint "
                << static_cast<uint32_t>(joint_error.id)
                << " error:";

    if (joint_error.error_code.has(JointError::StallProtection))
        std::cout << " stall-protection";
    if (joint_error.error_code.has(JointError::OverTemperature))
        std::cout << " over-temperature";
    if (joint_error.error_code.has(JointError::OverCurrent))
        std::cout << " over-current";
    if (joint_error.error_code.has(JointError::MotorAbnormal))
        std::cout << " motor-abnormal";
    if (joint_error.error_code.has(JointError::Communication))
        std::cout << " communication";
    if (joint_error.error_code.has(JointError::EncoderAbnormal))
        std::cout << " encoder-abnormal";

    std::cout << std::endl;
}

if (!state.hasFault() && !state.hasWarning())
    std::cout << "No warning or fault reported." << std::endl;
```

Notes

- The device handle must be acquired and started before reading the error state.
- `warning_flags` and `fault_flags` are bit flags reported by the device; the meaning of the individual bits follows the device-side definition.
- Joint error codes are bit flags and may indicate multiple errors simultaneously.
- The device error state is distinct from the `XynovaErrorCode` returned by the API functions: the former describes the device health, the latter describes the result of the API call itself.
- See *Run Dexterous Hand Device* for the complete manager initialization and device startup flow.

4.1.3.3 Logging

4.1.3.3.1 Create Logger Instance

This example demonstrates how to configure the **Xynova logging system** and attach dedicated realtime loggers to different modules (master and device).

Logging Architecture

The logging system is composed of:

- Writer → output destination (file / console)
- Sink → groups multiple writers
- Logger → used by modules to emit log messages

When to Use

Use custom loggers when you need:

- separate log files per module
- log file rotation
- asynchronous logging
- high-frequency logging without blocking control threads

Workflow

1. Initialize logging runtime
2. Create file writers
3. Create logger instances
4. Bind logger names to master/slave
5. Initialize and run manager
6. Stop the master communication loop

Example Code

```

using namespace xynova::infra::log;
using namespace xynova::communication;

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        std::cerr << "[ERROR] " << #expr \
        << " failed, ec=" << ec << std::endl; \
        return -1; \
    } \
} while (0)

XynovaErrorCode ec;

// =====
// 1. Initialize logging runtime (REQUIRED)
// Must be called before creating any writers or loggers
// =====

XynovaLogGlobalLogger_init(XYNOVA_LOG_LEVEL_DEBUG, nullptr);
LogManager::instance().initDefaultObjects();

// =====
// 2. Create file writers
// Each writer corresponds to one output file
// =====

FileWriterConfiguration file_conf{};
file_conf.base_conf.formater = "default";
file_conf.base_conf.auto_flush = true;
file_conf.base_conf.flush_interval_ms = 500;
file_conf.max_file_size = 100 * 1024 * 1024;
file_conf.max_roll_files = 3;

// master log file
file_conf.file_path = "/var/xynova/hand/log/master.log";
CHECK_EC(LogManager::instance().createFileWriter("default", "master", file_conf));

// hand log file
file_conf.file_path = "/var/xynova/hand/log/hand.log";
CHECK_EC(LogManager::instance().createFileWriter("default", "hand", file_conf));

// =====
// 3. Create logger instances
// A logger may bind one or more writers
// =====

LoggerConfiguration logger_conf{};
logger_conf.sink_name = "default";
logger_conf.level = XYNOVA_LOG_LEVEL_TRACE;
logger_conf.lb_type = "default";

```

(continues on next page)

(continued from previous page)

```

logger_conf.lb_config.pool_size = 50;
logger_conf.lb_config.block_size = 2048;
logger_conf.lb_config.message_size = 1024;
logger_conf.lb_config.max_continuous_repeated_log = 10;

// logger for master
logger_conf.writer_name_list = {"master"};
CHECK_EC(LogManager::instance().createLogger("master", logger_conf));

// logger for device
logger_conf.writer_name_list = {"hand"};
CHECK_EC(LogManager::instance().createLogger("hand", logger_conf));

// =====
// 4. Configure manager and bind logger names
//     Modules will emit logs through the assigned logger
// =====

ManagerConfig manager_cfg{};

MasterConfig master_cfg{};
master_cfg.name = "test_master";
master_cfg.cpu_index = -1;
master_cfg.period_us = 1000;

master_cfg.comm.type = CommType::Can;
master_cfg.comm.sub_type = "ChuangXinCAN";
master_cfg.comm.can.iface = "0.0";
master_cfg.comm.can.bitrate = 1'000'000;
master_cfg.comm.can.dbitrate = 5'000'000;

master_cfg.comm.logger_name = "master";

SlaveConfig slave_cfg{};
slave_cfg.type = "DexterousHandSlave";
slave_cfg.id = static_cast<uint32_t>(-1); // ID = 0xFFFFFFFF (all Fs)
slave_cfg.logger_name = "hand";

DeviceConfig device_cfg{};
device_cfg.type = "DexterousHandDevice";
device_cfg.name = "test_hand";
device_cfg.master_name = master_cfg.name;
device_cfg.slaves.push_back(slave_cfg);

manager_cfg.masters.push_back(master_cfg);
manager_cfg.devices.push_back(device_cfg);

// =====
// 5. Initialize and run manager
// =====

auto& mgr = Manager::instance();

```

(continues on next page)

(continued from previous page)

```

CHECK_EC (mgr.init (manager_cfg));
CHECK_EC (mgr.run (master_cfg.name));

// =====
// 6. Stop the master communication loop on exit
//     (the logging subsystem flushes writers asynchronously
//     and is shut down with the process)
// =====

CHECK_EC (mgr.stop (master_cfg.name));

```

4.1.3.4 C API

4.1.3.4.1 Dexterous Hand Control (C API)

This example demonstrates a complete control flow using the **C API**: manager initialization, device acquisition, joint control, and resource release. It is the C counterpart of the C++ examples such as [Run Dexterous Hand Device](#).

The C API requires manual management of the device handle: a handle obtained from `XynovaCommunicationManager_getDevice()` must be released with `XynovaCommunicationManager_freeDevice()`.

Workflow

1. Configure the master, slave, and device
2. Initialize and run the manager
3. Acquire the device handle
4. Start the hand device and send a joint angle command
5. Stop the hand device
6. Release the device handle and stop the manager

Example Code

```

#include <stdint.h>
#include <stdio.h>

#include "xynova/c/comm_api.h"
#include "xynova/c/comm_types.h"
#include "xynova/c/dexterous_hand_api.h"
#include "xynova/c/dexterous_hand_types.h"

#define CHECK_EC(expr) \
do { \
    ec = (expr); \
    if (ec != XYNOVA_ERROR_CODE_OK) { \
        fprintf(stderr, "[ERROR] %s failed, ec=%d\n", \
                #expr, ec); \
        return -1; \
    } \
}

```

(continues on next page)

(continued from previous page)

```

    } while (0)

int main(void)
{
    XynovaErrorCode ec;

    // =====
    // Configure master (CAN)
    // =====

    XynovaCommunicationMasterConfig master = {0};
    master.name = "test_master";
    master.cpu_index = -1;
    master.period_us = 1000;

    master.comm.type = XYNova_COMMUNICATION_COMM_TYPE_CAN;
    master.comm.sub_type = "ChuangXinCAN";
    master.comm.config.can.iface = "0.0";
    master.comm.config.can.bitrate = 1000000;
    master.comm.config.can.dbitrate = 5000000;
    master.comm.config.can.sample_point = 0.75f;
    master.comm.config.can.dsampling_point = 0.75f;

    // =====
    // Configure slave and device
    // =====

    XynovaCommunicationSlaveConfig slave = {0};
    slave.type = "DexterousHandSlave";
    slave.id = 0xFFFFFFFF; // ID = 0xFFFFFFFF (all Fs)

    XynovaCommunicationDeviceConfig device_cfg = {0};
    device_cfg.type = "DexterousHandDevice";
    device_cfg.name = "test_hand";
    device_cfg.master_name = master.name;
    device_cfg.slaves = &slave;
    device_cfg.slave_count = 1;

    XynovaCommunicationManagerConfig manager_cfg = {0};
    manager_cfg.masters = &master;
    manager_cfg.master_count = 1;
    manager_cfg.devices = &device_cfg;
    manager_cfg.device_count = 1;

    // =====
    // Initialize and run the manager
    // =====

    CHECK_EC(XynovaCommunicationManager_init(&manager_cfg));
    CHECK_EC(XynovaCommunicationManager_run(master.name));

    // =====

```

(continues on next page)

(continued from previous page)

```

// Acquire and start the device
// =====

XynovaDeviceHandle *device =
    XynovaCommunicationManager_getDevice(
        master.name, device_cfg.type, device_cfg.name);

if (device == NULL)
{
    fprintf(stderr, "[ERROR] device not found\n");
    return -1;
}

CHECK_EC(XynovaDexterousHand_run(device));

// =====
// Set control mode and send a joint angle command
// =====

CHECK_EC(XynovaDexterousHand_setJointControlAllMode(
    device,
    XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_ANGLE));

XynovaDexterousHandAngleControl angle = {0};
for (size_t i = 0; i < XYNOVA_DEXTEROUS_HAND_JOINT_NUM; ++i)
{
    angle.values[i].id = (uint16_t)(i + 1);
    angle.values[i].data.type = XYNOVA_DATA_TYPE_FLOAT;
    angle.values[i].data.value.f32 = 0.5236f;
}

CHECK_EC(XynovaDexterousHand_setJointAngle(device, &angle));

// =====
// Stop the device and release resources
// =====

CHECK_EC(XynovaDexterousHand_stop(device));

XynovaCommunicationManager_freeDevice(device);
CHECK_EC(XynovaCommunicationManager_stop(master.name));

return 0;
}

```

Notes

- The device handle obtained from `XynovaCommunicationManager_getDevice()` must be released with `XynovaCommunicationManager_freeDevice()`.
- Always check the device handle for `NULL` before use.
- The C API structures are populated with the same semantics as their C++ counterparts; see [Initialize Communication](#)

Manager for the configuration field descriptions.

4.1.4 Full Examples

End-to-end examples that demonstrate a complete workflow including manager initialization, device startup, control, and logging.

4.1.4.1 Dexterous Hand —Servo Control

4.1.4.1.1 Overview

This guide presents a **minimal end-to-end control workflow** for the **Dexterous Hand** using the **Xynova C++ API**.

It follows the same control path used by the Dexterous Hand test case and shows only the interfaces that are currently available to users.

4.1.4.1.2 Features

This manual covers:

- Logging system initialization
- Manager / Master / Device configuration
- CAN communication setup
- Manager runtime startup and logger binding
- Control mode selection
- Synchronous angle control

4.1.4.1.3 Prerequisites

- Linux system
- CAN driver installed (SocketCAN or vendor driver)
- Root privileges recommended for realtime scheduling

Note

Realtime scheduling (`SCHED_FIFO`) requires root or `CAP_SYS_NICE`.

4.1.4.1.4 Execution Flow

The workflow consists of the following steps:

1. Initialize logging
2. Create the manager configuration
3. Start the master communication loop
4. Create and run the device
5. Set realtime scheduling on the control thread
6. Switch to angle control mode
7. Send an angle command

8. Stop the device and release resources

4.1.4.1.5 Source Code

```

1  #include <atomic>
2  #include <iostream>
3  #include <thread>
4
5  #include <errno.h>
6  #include <pthread.h>
7  #include <sched.h>
8  #include <stdio.h>
9  #include <string.h>
10 #include <time.h>
11 #include <stdint.h>
12
13 #include "xynova/c/log_api.h"
14 #include "xynova/c/version.h"
15
16 #include "xynova/cpp/communication/Manager.hpp"
17 #include "xynova/cpp/hand/dexterous_hand/Device.hpp"
18 #include "xynova/cpp/infra/log/LogManager.hpp"
19
20 using namespace xynova::hand::dexterous_hand;
21
22 // =====
23 // helpers
24 // =====
25
26 #define CHECK_EC(expr) \
27     do { \
28         ec = (expr); \
29         if (ec != XYNOVA_ERROR_CODE_OK) { \
30             std::cerr << "[ERROR] " << #expr \
31                 << " failed, ec=" << ec << std::endl; \
32             return -1; \
33         } \
34     } while (0)
35
36 bool set_current_thread_realtime(int priority)
37 {
38     if (priority < 1 || priority > 99) {
39         return false;
40     }
41
42     struct sched_param param{};
43     param.sched_priority = priority;
44
45     int ret = pthread_setschedparam(pthread_self(), SCHED_FIFO, &param);
46     if (ret != 0)
47     {
48         std::cerr << "[WARN] set realtime failed: " << strerror(ret) << std::endl;
49         return false;

```

(continues on next page)

(continued from previous page)

```

50     }
51
52     return true;
53 }
54
55
56 // =====
57 // main
58 // =====
59
60 int main()
61 {
62     XynovaErrorCode ec;
63
64     // -----
65     // Version info
66     // -----
67     std::cout << "SDK Version : " << xynovaGetSdkVersion() << "\n"
68               << "Git Branch  : " << xynovaGetSdkGitBranch() << "\n"
69               << "Git Commit  : " << xynovaGetSdkGitCommit() << "\n"
70               << "Build Time  : " << xynovaGetSdkBuildTime() << "\n\n";
71
72     // -----
73     // Logging
74     // -----
75     XynovaLogGlobalLogger_init(XYNOVA_LOG_LEVEL_DEBUG, nullptr);
76
77     auto &log_mgr = xynova::infra::log::LogManager::instance();
78     log_mgr.initDefaultObjects();
79
80     xynova::infra::log::FileWriterConfiguration file_conf;
81     file_conf.base_conf.formater = "default";
82     file_conf.base_conf.auto_flush = true;
83     file_conf.base_conf.flush_interval_ms = 500;
84     file_conf.file_path = "/var/xynova/hand/log/master.log";
85     file_conf.max_file_size = 100 * 1024 * 1024;
86     file_conf.max_roll_files = 3;
87
88     CHECK_EC(log_mgr.createFileWriter("default", "master", file_conf));
89
90     file_conf.file_path = "/var/xynova/hand/log/hand.log";
91     CHECK_EC(log_mgr.createFileWriter("default", "hand", file_conf));
92
93     xynova::infra::log::LoggerConfiguration logger_conf;
94     logger_conf.sink_name = "default";
95     logger_conf.writer_name_list = {"master"};
96     logger_conf.level = XYNOVA_LOG_LEVEL_TRACE;
97     logger_conf.lb_type = "default";
98     logger_conf.lb_config.pool_size = 50;
99     logger_conf.lb_config.block_size = 2048;
100    logger_conf.lb_config.message_size = 1024;
101    logger_conf.lb_config.max_continuous_repeated_log = 10;

```

(continues on next page)

(continued from previous page)

```

102 CHECK_EC(log_mgr.createLogger("master", logger_conf));
103
104 logger_conf.writer_name_list = {"hand"};
105 CHECK_EC(log_mgr.createLogger("hand", logger_conf));
106
107 // -----
108 // Manager config
109 // -----
110 xynova::communication::ManagerConfig manager_cfg;
111
112 xynova::communication::MasterConfig master_cfg;
113 master_cfg.name = "test_master";
114 master_cfg.cpu_index = -1;
115 master_cfg.period_us = 1000;
116 master_cfg.comm.type = xynova::communication::CommType::Can;
117 master_cfg.comm.sub_type = "ChuangXinCAN";
118 master_cfg.comm.can.iface = "0.0";
119 master_cfg.comm.can.bitrate = 1000000;
120 master_cfg.comm.can.dbitrate = 5000000;
121 master_cfg.comm.can.sample_point = 0.75;
122 master_cfg.comm.can.dsampling_point = 0.75;
123 // master_cfg.comm.can.manager_can = true;
124 master_cfg.comm.logger_name = "master";
125
126 xynova::communication::SlaveConfig slave_cfg;
127 slave_cfg.type = "DexterousHandSlave";
128 slave_cfg.id = static_cast<uint32_t>(-1); // ID = 0xFFFFFFFF (all Fs)
129 slave_cfg.logger_name = "hand";
130
131 slave_cfg.general_time_setting.user_max_timeout_us = 50 * 1000;
132 slave_cfg.general_time_setting.user_poll_interval_us = 1 * 1000;
133 slave_cfg.general_time_setting.execute_timeout_us = 5 * 1000;
134 slave_cfg.general_time_setting.execute_poll_interval_us = 10 * 1000;
135 slave_cfg.general_time_setting.execute_max_retry = 3;
136
137 slave_cfg.fast_time_setting.user_max_timeout_us = 5 * 1000;
138 slave_cfg.fast_time_setting.user_poll_interval_us = 100;
139 slave_cfg.fast_time_setting.execute_timeout_us = 3 * 1000;
140 slave_cfg.fast_time_setting.execute_poll_interval_us = 3 * 1000;
141 slave_cfg.fast_time_setting.execute_max_retry = 0;
142
143 xynova::communication::DeviceConfig device_cfg;
144 device_cfg.type = "DexterousHandDevice";
145 device_cfg.name = "test_hand";
146 device_cfg.master_name = master_cfg.name;
147 device_cfg.slaves.push_back(slave_cfg);
148
149 manager_cfg.masters.push_back(master_cfg);
150 manager_cfg.devices.push_back(device_cfg);
151
152 // -----
153

```

(continues on next page)

(continued from previous page)

```

154 // Manager start
155 // -----
156 auto &mgr = xynova::communication::Manager::instance();
157
158 CHECK_EC(mgr.init(manager_cfg));
159 CHECK_EC(mgr.run(master_cfg.name));
160
161 // -----
162 // Device
163 // -----
164 auto device = mgr.getDevice<Device>(
165     master_cfg.name,
166     device_cfg.type,
167     device_cfg.name);
168
169 if (!device || !device->valid())
170 {
171     std::cerr << "[ERROR] device invalid\n";
172     return -1;
173 }
174
175 // Switch the calling (control) thread to realtime scheduling.
176 // Note: the SDK's master communication threads are configured with
177 // realtime scheduling internally; this call only affects the current
178 // thread and requires root or CAP_SYS_NICE.
179 set_current_thread_realtime(80);
180
181 CHECK_EC(device->run());
182
183 // -----
184 // Control example
185 // -----
186 ControlMode mode = XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_ANGLE;
187 CHECK_EC(device->setJointControlMode(mode));
188
189 AngleControl angle;
190 for (size_t i = 0; i < JOINT_NUM; ++i)
191 {
192     angle[i].setId(static_cast<JointID>(i + 1));
193     angle[i].setValue(0.5236f);
194 }
195
196 CHECK_EC(device->setJointAngle(angle));
197
198 std::cout << "Control command sent successfully.\n";
199
200 // -----
201 // Stop the device and release resources
202 // -----
203 CHECK_EC(device->stop());
204 CHECK_EC(mgr.stop(master_cfg.name));
205

```

(continues on next page)

(continued from previous page)

```

206     return 0;
207 }

```

4.1.5 Usage Notes

- The examples are written around the C++ high-level API unless otherwise noted.
- The corresponding implementation lives in the communication, dexterous hand, and infrastructure source trees.
- They are intended to be minimal and easy to adapt to real hardware.

4.2 API Reference

This section provides the programming interfaces exposed by the **Xynova SDK** in this repository.

The SDK exposes two layers of APIs:

- **C API** —stable ABI, lightweight, and suitable for embedded systems or cross-language bindings.
- **C++ API** —object-oriented wrappers built on top of the C layer for safer and more convenient development.

The implementation currently spans the communication stack, dexterous hand control modules, logging utilities, and CAN driver integrations. You can use either layer depending on whether you need ABI stability or a more ergonomic C++ experience.

4.2.1 Overview

The Xynova SDK contains the following functional modules:

- **Communication modules** Manager initialization, master/device lifecycle, and CAN transport setup.
- **Hand control modules** Control and monitor dexterous hand devices, including joint actuation, state reporting, and runtime control.
- **Infrastructure utilities** Logging framework, error handling, and common data types for robust and maintainable code.

4.2.2 API Categories

4.2.2.1 C API

The C API modules are:

4.2.2.1.1 C API Reference

The **C API** provides a stable ABI for Xynova devices and infrastructure. It is lightweight and suitable for embedded systems, cross-language bindings, and performance-critical applications.

Overview

The C API includes the following modules:

- **Communication:** Manage masters, devices, and CAN transport layers.
- **Hand Control:** Dexterous hand device control and monitoring.
- **Infrastructure:** Logging, error codes, and common types.

Notes

- All C API functions are prefixed with *Xynova*.
- Manual memory management is required for some objects.
- For higher-level abstractions and RAII, see the C++ API.

4.2.2.1.2 Communication Module (C API)

Overview

The **Communication Module** provides the public C API for managing Xynova communication masters, devices, and CAN transport interfaces. It exposes the stable ABI entry points used by applications and bindings to initialize the communication manager, operate masters, access devices, and register CAN backends.

Public headers

The externally exposed C headers for this module are:

- `xynova/c/comm_api.h` —communication manager lifecycle and device access APIs
- `xynova/c/comm_types.h` —manager, master, device, and slave configuration types

The CAN driver registration entry points are documented on the [CAN transport module](#) page:

- `xynova/c/can_api.h` —CAN driver registration entry points
- `xynova/c/can_types.h` —CAN transport configuration and callback types

API Reference

Communication manager C API for the Xynova SDK.

Provides public APIs for initializing, running, stopping and managing communication managers and connected devices.

See also

XynovaCommunicationManagerConfig

See also

XynovaDeviceHandle

Version

1.0.0

Date

2025-10-31

Author

Xynova

Typedefs

typedef struct *XynovaDeviceHandle* **XynovaDeviceHandle**

Opaque handle representing a connected device.

The handle is obtained from *XynovaCommunicationManager_getDevice()* and released using *XynovaCommunicationManager_freeDevice()*.

Functions

XynovaErrorCode **XynovaCommunicationManager_init** (const *XynovaCommunicationManagerConfig* *config)

Initialize communication manager.

Creates communication masters and device instances according to the configuration.

Parameters

config – Communication manager configuration.

Returns

XynovaErrorCode.

XynovaErrorCode **XynovaCommunicationManager_run** (const char *master_name)

Start communication manager.

Starts communication on the specified master.

Parameters

master_name – Master name.

Returns

XynovaErrorCode.

XynovaErrorCode **XynovaCommunicationManager_stop** (const char *master_name)

Stop communication manager.

Stops communication on the specified master.

Parameters

master_name – Master name.

Returns

XynovaErrorCode.

XynovaDeviceHandle ***XynovaCommunicationManager_getDevice** (const char *master_name, const char *device_type, const char *device_name)

Get device handle.

Retrieves a device handle by master name, device type and device name.

Parameters

- **master_name** – Master name.
- **device_type** – Device type.
- **device_name** – Device name.

Returns

Device handle, or NULL if not found.

void **XynovaCommunicationManager_freeDevice** (*XynovaDeviceHandle* *device)

Free device handle.

Releases a device handle obtained from *XynovaCommunicationManager_getDevice()*.

Parameters

device – Device handle.

XynovaErrorCode **XynovaCommunicationManager_upgradeFirmware** (const char *master_name, uint32_t slave_id, const char *file)

Upgrade slave firmware.

Parameters

- **master_name** – Master name.
- **slave_id** – Slave identifier.
- **file** – Firmware file path.

Returns

XynovaErrorCode.

Core communication type definitions for the Xynova SDK.

Defines communication configuration types including communication bus, master, slave, device and manager topology.

➡ See also

XynovaCommunicationCanConfig

➡ See also

XynovaCommunicationManagerConfig

Version

1.0.0

Date

2025-10-29

Author

Xynova

Typedefs

typedef uint8_t **XynovaCommunicationCommType**

Communication type.

Stored as uint8_t to guarantee ABI compatibility.

typedef struct *XynovaCommunicationCommConfig* **XynovaCommunicationCommConfig**

Communication configuration.

```
typedef struct XynovaCommunicationMasterConfig XynovaCommunicationMasterConfig
    Master configuration.

typedef struct XynovaCommunicationResponseTimeSetting XynovaCommunicationResponseTimeSetting
    Response timing configuration.

typedef struct XynovaCommunicationSlaveConfig XynovaCommunicationSlaveConfig
    Slave configuration.

typedef struct XynovaCommunicationSchedulingCommand XynovaCommunicationSchedulingCommand
    Scheduling command configuration.
    Defines commands periodically sent to a device with response timing settings.

typedef struct XynovaCommunicationDeviceConfig XynovaCommunicationDeviceConfig
    Device configuration.

typedef struct XynovaCommunicationManagerConfig XynovaCommunicationManagerConfig
    Communication manager configuration.
```

Enums

```
enum [anonymous]
    Communication types.
    Values:

    enumerator XYNOVA_COMMUNICATION_COMM_TYPE_CAN
        CAN communication
```

```
struct XynovaCommunicationCommConfig
    #include <comm_types.h> Communication configuration.
```

Public Members

```
XynovaCommunicationCommType type
    Communication type

const char *sub_type
    Communication driver subtype

XynovaCommunicationCanConfig can
    CAN configuration

union XynovaCommunicationCommConfig::[anonymous] config
```

```
const char *logger_name
```

Logger name

```
struct XynovaCommunicationMasterConfig
```

#include <comm_types.h> Master configuration.

Public Members

```
const char *name
```

Master name

```
XynovaCommunicationCommConfig comm
```

Communication configuration

```
uint64_t period_us
```

Control loop period in microseconds

```
int32_t cpu_index
```

CPU affinity index, -1 disables binding

```
struct XynovaCommunicationResponseTimeSetting
```

#include <comm_types.h> Response timing configuration.

Public Members

```
uint32_t user_max_timeout_us
```

Maximum user timeout

```
uint32_t user_poll_interval_us
```

User polling interval

```
uint32_t execute_timeout_us
```

Backend request timeout

```
uint32_t execute_max_retry
```

Backend retry count

```
uint32_t execute_poll_interval_us
```

Backend polling interval

```
struct XynovaCommunicationSlaveConfig
```

#include <comm_types.h> Slave configuration.

Public Members

const char ***type**

Slave type

uint32_t **id**

Slave identifier

XynovaCommunicationResponseTimeSetting **general_time_setting**

General response timing

XynovaCommunicationResponseTimeSetting **fast_time_setting**

Fast response timing

const char ***logger_name**

Logger name

struct **XynovaCommunicationSchedulingCommand**

#include <comm_types.h> Scheduling command configuration.

Defines commands periodically sent to a device with response timing settings.

Public Members

const char ***command**

Command name

XynovaCommunicationResponseTimeSetting **response_time_setting**

Response timing

struct **XynovaCommunicationDeviceConfig**

#include <comm_types.h> Device configuration.

Public Members

const char ***type**

Device type

const char ***name**

Device name

const char ***master_name**

Associated master name

XynovaCommunicationSlaveConfig ***slaves**

Slave configuration array

size_t **slave_count**

Number of slaves

XynovaCommunicationSchedulingCommand ***commands**

Scheduling command array

size_t **command_count**

Number of commands

struct **XynovaCommunicationManagerConfig**

#include <comm_types.h> Communication manager configuration.

Public Members

XynovaCommunicationMasterConfig ***masters**

Master configuration array

size_t **master_count**

Number of masters

XynovaCommunicationDeviceConfig ***devices**

Device configuration array

size_t **device_count**

Number of devices

Notes

- All handles and configuration objects should be initialized before invoking master loops or accessing devices.
- For detailed function signatures and type layouts, refer to the generated Doxygen documentation.

4.2.2.1.3 CAN Transport Module (C API)

Overview

The **CAN Transport Module** provides low-level APIs for interfacing with CAN buses in Xynova devices. It allows sending, receiving, and configuring CAN messages in real-time.

This module is implemented in C and is part of the Communication API.

API Reference

CAN driver registration API for the Xynova SDK.

This header provides the interface to register custom CAN driver implementations with the Xynova SDK. Drivers must implement the *XynovaCommunicationCanVTable* structure and provide it to the SDK via this registration function.

Example:

```
XynovaCommunicationCanVTable my_driver = { ... };
XynovaCommunicationCan_registerDriver("socketcan", &my_driver);
```

➔ See also

XynovaCommunicationCanVTable

Version

1.0.0

Date

2025-10-31

Author

Xynova

Functions

XynovaErrorCode **XynovaCommunicationCan_registerDriver** (const char *type, const *XynovaCommunicationCanVTable* *vtable)

Register a CAN driver implementation.

The driver must provide a valid *XynovaCommunicationCanVTable*.

Parameters

- **type** – Driver type name, e.g. “socketcan” .
- **vtable** – Driver virtual function table.

Return values

XYNOVA_ERROR_CODE_OK – Driver registered successfully.

Returns

XynovaErrorCode.

Core CAN type definitions for the Xynova SDK communication subsystem.

Defines fundamental CAN communication types including frame types, identifier masks, interface configuration, receive filters, and driver callback interfaces.

 See also[*XynovaCommunicationCanConfig*](#) **See also**[*XynovaCommunicationCanVTable*](#)**Version**

1.0.0

Date

2025-10-31

Author

Xynova

Typedefs**typedef uint8_t XynovaCommunicationCanFrameType**

CAN frame identifier type.

Stored as uint8_t to guarantee ABI compatibility.

typedef uint32_t XynovaCommunicationCanMask

CAN identifier mask type.

Stored as uint32_t to guarantee ABI compatibility.

typedef struct [*XynovaCommunicationCanConfig*](#) XynovaCommunicationCanConfig

CAN interface configuration.

typedef struct [*XynovaCommunicationCanRecvFilter*](#) XynovaCommunicationCanRecvFilter

CAN receive filter.

A frame is accepted when: (received_id & mask) == (id & mask)

typedef struct [*XynovaCommunicationCanVTable*](#) XynovaCommunicationCanVTable

Virtual function table (VTable) for a CAN driver implementation.

Each driver must provide a filled instance of this structure, implementing the basic CAN interface operations such as `create`, `open`, `send`, and `recv`. This enables the SDK to dynamically load and manage different CAN backends.

Example registration:

```
XynovaCommunicationCanVTable my_driver = {
    .create = MyDriver_create,
    .destroy = MyDriver_destroy,
    .open = MyDriver_open,
    .close = MyDriver_close,
    .isOpen = MyDriver_isOpen,
    .setRecvFilters = MyDriver_setFilters,
    .send = MyDriver_send,
    .recv = MyDriver_recv
};
registerCanDriver("socketcan", my_driver);
```

Enums

enum **[anonymous]**

CAN frame identifier types.

Values:

enumerator **XYNOVA_COMMUNICATION_CAN_FRAME_TYPE_STANDARD**

Standard 11-bit CAN identifier

enumerator **XYNOVA_COMMUNICATION_CAN_FRAME_TYPE_EXTENDED**

Extended 29-bit CAN identifier

enum **[anonymous]**

CAN identifier masks.

Values:

enumerator **XYNOVA_COMMUNICATION_CAN_MASK_STANDARD**

Standard CAN identifier mask

enumerator **XYNOVA_COMMUNICATION_CAN_MASK_EXTENDED**

Extended CAN identifier mask

struct **XynovaCommunicationCanConfig**

#include <can_types.h> CAN interface configuration.

Public Members

const char ***iface**

CAN interface name, e.g. "can0"

bool **manager_can**

Whether SDK manages CAN interface

uint32_t **bitrate**

Arbitration bitrate

uint32_t **dbitrate**

CAN-FD data bitrate

float **sample_point**

Arbitration sample point, range 0.0 ~ 1.0

float **dsample_point**

CAN-FD data sample point, range 0.0 ~ 1.0

struct **XynovaCommunicationCanRecvFilter**

#include <can_types.h> CAN receive filter.

A frame is accepted when: (received_id & mask) == (id & mask)

Public Members

XynovaCommunicationCanFrameType **type**

CAN frame identifier type

uint32_t **id**

CAN identifier

XynovaCommunicationCanMask **mask**

CAN identifier mask

struct **XynovaCommunicationCanVTable**

#include <can_types.h> Virtual function table (VTable) for a CAN driver implementation.

Each driver must provide a filled instance of this structure, implementing the basic CAN interface operations such as create, open, send, and recv. This enables the SDK to dynamically load and manage different CAN backends.

Example registration:

```
XynovaCommunicationCanVTable my_driver = {
    .create = MyDriver_create,
    .destroy = MyDriver_destroy,
    .open = MyDriver_open,
    .close = MyDriver_close,
    .isOpen = MyDriver_isOpen,
    .setRecvFilters = MyDriver_setFilters,
    .send = MyDriver_send,
    .recv = MyDriver_recv
};
registerCanDriver("socketcan", my_driver);
```

Public Members

void **(*create)**(const *XynovaCommunicationCanConfig* *config)

Create driver instance.

Param config

CAN configuration.

Return

Driver handle, or NULL on failure.

void **(*destroy)**(void *handle)

Destroy driver instance.

Param handle

Driver handle.

XynovaErrorCode **(*open)**(void *handle)

Open CAN interface.

Param handle

Driver handle.

Return

XynovaErrorCode.

XynovaErrorCode **(*close)**(void *handle)

Close CAN interface.

Param handle

Driver handle.

Return

XynovaErrorCode.

bool **(*isOpen)**(void *handle)

Check CAN interface state.

Param handle

Driver handle.

Return

true if opened.

XynovaErrorCode **(*setRecvFilters)**(void *handle, uint32_t num, const *XynovaCommunicationCanRecvFilter* *filters)

Configure CAN receive filters.

Param handle

Driver handle.

Param num

Number of filters.

Param filters

Filter array.

Return

XynovaErrorCode.

XynovaErrorCode (***send**)(void *handle, *XynovaCommunicationCanFrameType* type, uint32_t id, const uint8_t *data, size_t len, uint32_t timeout_us)

Send CAN frame.

Param handle

Driver handle.

Param type

Frame identifier type.

Param id

CAN identifier.

Param data

Data buffer.

Param len

Data length.

Param timeout_us

Timeout in microseconds.

Return

XynovaErrorCode.

XynovaErrorCode (***recv**)(void *handle, *XynovaCommunicationCanFrameType* *type, uint32_t *id, uint8_t *data, size_t *len, uint32_t timeout_us)

Receive CAN frame.

Param handle

Driver handle.

Param type

Output frame type.

Param id

Output CAN identifier.

Param data

Receive buffer.

Param len

Buffer size and received length.

Param timeout_us

Timeout in microseconds.

Return

XynovaErrorCode.

4.2.2.1.4 Hand Control Module (C API)

This section documents the **hand control modules** in the Xynova C API.

The C API provides the public low-level entry points for controlling dexterous hand devices, including startup, shutdown, joint control, and joint state queries.

Public headers

The externally exposed headers for this module are:

- `xynova/c/dexterous_hand_api.h` — device lifecycle and joint-control APIs
- `xynova/c/dexterous_hand_types.h` — command and state type definitions

API Reference

See *Dexterous Hand (C API)* for the full API reference of this module.

4.2.2.1.5 Dexterous Hand (C API)

This section documents the **Dexterous Hand device APIs** in the Xynova C API.

It provides the low-level functions currently implemented for the hand, including:

- Device lifecycle management (`XynovaDexterousHand_run / stop`)
- Product information (model, serial number, hardware/software versions, hand type)
- Error state monitoring (`XynovaDexterousHand_getErrorState`)
- Joint control mode configuration (`XynovaDexterousHand_setJointControlAllMode`)
- Synchronous joint angle commands (`XynovaDexterousHand_setJointAngle`)
- Asynchronous joint angle commands (`XynovaDexterousHand_setJointAngle_async`)
- Joint angle state queries (`XynovaDexterousHand_getJointAngle`)
- Tactile feedback (wrench and taxel enable/query functions)
- Range feedback (`XynovaDexterousHand_enableRange / getRange`)

Overview

The **Dexterous Hand module** allows direct control over Xynova dexterous hand devices. All functions follow the C API conventions:

- Return *XynovaErrorCode* for error handling.
- Operate on *XynovaDeviceHandle* obtained from the Communication Manager.
- Require proper initialization before use.

API Functions

C API for Xynova Dexterous Hand control and monitoring.

This header provides the public C interface for controlling and monitoring Xynova Dexterous Hand devices, including device lifecycle management, joint control, and joint state acquisition.

Functions

XynovaErrorCode **XynovaDexterousHand_run** (*XynovaDeviceHandle* *device)

Start the dexterous hand device.

Initializes and starts device communication.

Parameters

device – Device handle obtained from *XynovaCommunicationManager_getDevice*.

Returns

XynovaErrorCode

- XYNOVA_ERROR_CODE_OK on success
- Other codes indicate startup failure.

XynovaErrorCode **XynovaDexterousHand_stop** (*XynovaDeviceHandle* *device)

Stop the dexterous hand device.

Parameters

device – Device handle.

Returns

XynovaErrorCode

XynovaErrorCode **XynovaDexterousHand_getProductModel** (*XynovaDeviceHandle* *device, char *product_model, uint32_t size)

Get product model.

Reads the product model string from the device.

Parameters

- **device** – Device handle.
- **product_model** – Output buffer for product model string.
- **size** – Size of product_model in bytes.

Returns

XynovaErrorCode

XynovaErrorCode **XynovaDexterousHand_getProductSN** (*XynovaDeviceHandle* *device, char *product_sn, uint32_t size)

Get product serial number.

Reads the product serial number from the device.

Parameters

- **device** – Device handle.
- **product_sn** – Output buffer for serial number string.
- **size** – Size of product_sn in bytes.

Returns

XynovaErrorCode

XynovaErrorCode **XynovaDexterousHand_getHardwareVersion** (*XynovaDeviceHandle* *device, char *version, uint32_t size)

Get hardware version.

Reads the hardware version string.

Parameters

- **device** – Device handle.
- **version** – Output buffer for hardware version.
- **size** – Size of version in bytes.

Returns*XynovaErrorCode*

XynovaErrorCode **XynovaDexterousHand_getSoftwareVersion** (*XynovaDeviceHandle* *device, char *version, uint32_t size)

Get software version.

Reads the software version string.

Parameters

- **device** – Device handle.
- **version** – Output buffer for software version.
- **size** – Size of version in bytes.

Returns*XynovaErrorCode*

XynovaErrorCode **XynovaDexterousHand_getHandType** (*XynovaDeviceHandle* *device, *XynovaDexterousHandType* *type)

Get hand type.

Reads the hand type from the device.

Parameters

- **device** – Device handle.
- **type** – Output hand type.

Returns*XynovaErrorCode*

XynovaErrorCode **XynovaDexterousHand_getErrorState** (*XynovaDeviceHandle* *device, *XynovaDexterousHandErrorState* *state)

Get current device error state.

Retrieves the latest global warning, global fault and joint error information from the dexterous hand.

The returned data contains:

- Global warning flags.
- Global fault flags.
- Joint error codes for all joints.

Parameters

- **device** – Device handle.
- **state** – Output error state.

Returns*XynovaErrorCode*

- XYNOVA_ERROR_CODE_OK on success.

XynovaErrorCode **XynovaDexterousHand_setJointControlAllMode** (*XynovaDeviceHandle* *device, *XynovaDexterousHandControlMode* mode)

Set control mode for all joints.

Parameters

- **device** – Device handle.
- **mode** – Joint control mode.

Returns

XynovaErrorCode

XynovaErrorCode **XynovaDexterousHand_enableTactileWrench** (*XynovaDeviceHandle* *device, bool enable)

Enable or disable tactile wrench reporting.

Controls whether the device publishes tactile wrench data. When disabled, tactile wrench feedback is not transmitted.

Parameters

- **device** – Device handle.
- **enable** –
 - true: Enable tactile wrench reporting.
 - false: Disable tactile wrench reporting.

Returns

XynovaErrorCode

XynovaErrorCode **XynovaDexterousHand_enableTactileTaxel** (*XynovaDeviceHandle* *device, bool enable)

Enable or disable tactile taxel reporting.

Controls whether the device publishes detailed tactile taxel data. When disabled, tactile taxel feedback is not transmitted.

Parameters

- **device** – Device handle.
- **enable** –
 - true: Enable tactile taxel reporting.
 - false: Disable tactile taxel reporting.

Returns

XynovaErrorCode

XynovaErrorCode **XynovaDexterousHand_enableRange** (*XynovaDeviceHandle* *device, bool enable)

Enable or disable range reporting.

Controls whether the device publishes range measurement data. When disabled, range feedback is not transmitted.

Parameters

- **device** – Device handle.
- **enable** –
 - true: Enable range reporting.

- false: Disable range reporting.

Returns*XynovaErrorCode*

XynovaErrorCode **XynovaDexterousHand_setJointAngle** (*XynovaDeviceHandle* *device, const *XynovaDexterousHandAngleControl* *angle)

Set joint angle control command.

Sends joint angle commands and waits until execution is completed.

Parameters

- **device** – Device handle.
- **angle** – Joint angle command.

Returns*XynovaErrorCode*

XynovaErrorCode **XynovaDexterousHand_setJointAngle_async** (*XynovaDeviceHandle* *device, const *XynovaDexterousHandAngleControl* *angle)

Asynchronously set joint angle control command.

Queues the joint command and returns immediately. The execution is handled internally by the communication backend.

Parameters

- **device** – Device handle.
- **angle** – Joint angle command.

Returns*XynovaErrorCode*

XynovaErrorCode **XynovaDexterousHand_getJointAngle** (*XynovaDeviceHandle* *device, *XynovaDexterousHandAngleStates* *states, uint64_t *timestamp_us)

Get current joint angle states.

Retrieves the latest joint state data from the device.

Parameters

- **device** – Device handle.
- **states** – Output joint angle states.
- **timestamp_us** – Output timestamp in microseconds. Can be NULL when timestamp is not required.

Returns*XynovaErrorCode*

XynovaErrorCode **XynovaDexterousHand_getTactileWrench** (*XynovaDeviceHandle* *device, *XynovaDexterousHandTactileWrenchStates* *states, uint64_t *timestamp_us)

Get tactile wrench states.

Retrieves the latest tactile wrench data from the dexterous hand.

Each tactile region contains:

- **timestamp_us**: Sampling timestamp in microseconds.
- **region**: Tactile region identifier.
- **dimension**: Valid wrench dimension.
- **wrench**: Force/torque values.

The wrench vector is defined as:

$[F_x, F_y, F_z, M_x, M_y, M_z]$

where:

- **F***: Force components. Unit: N (Newtons)
- **M***: Torque components. Unit: Nm (Newton-meters)

Parameters

- **device** – Device handle.
- **states** – Output tactile wrench states.
- **timestamp_us** – Output timestamp in microseconds. Can be NULL when timestamp is not required.

Returns

XynovaErrorCode

- XYNOVA_ERROR_CODE_OK on success.

XynovaErrorCode **XynovaDexterousHand_getTactileTaxel** (*XynovaDeviceHandle* *device, *XynovaDexterousHandTactileTaxelStates* *states, uint64_t *timestamp_us)

Get tactile taxel states.

Retrieves detailed tactile taxel data from the dexterous hand.

The caller provides the taxel buffer. The buffer size must be sufficient for the requested data.

Parameters

- **device** – Device handle.
- **states** – Output tactile taxel states.
- **timestamp_us** – Output timestamp in microseconds. Can be NULL when timestamp is not required.

Returns

XynovaErrorCode

XynovaErrorCode **XynovaDexterousHand_getRange** (*XynovaDeviceHandle* *device, *XynovaDexterousHandRangeStates* *states, uint64_t *timestamp_us)

Get range measurement states.

Retrieves the latest range measurement data from the dexterous hand.

The caller provides the range point buffer. The buffer size must be sufficient for the requested data.

Points are stored in row-major order:

```
index = y * width + x
```

Each point contains:

- **timestamp_us**: Sampling timestamp.
- **id**: Point identifier.
- **status**: Measurement status.
- **distance**: Measured distance in meters.

Parameters

- **device** – Device handle.
- **states** – Output range measurement states.
- **timestamp_us** – Output timestamp in microseconds. Represents the latest update time of the returned range states. Can be NULL when timestamp is not required.

Returns

XynovaErrorCode

Xynova Dexterous Hand type definitions.

This header defines fundamental data types used by the Xynova Dexterous Hand API, including joint identifiers, control modes, joint commands, and feedback state structures.

Defines

XYNOVA_DEXTEROUS_HAND_JOINT_NUM

Number of joints supported by Xynova Dexterous Hand.

XYNOVA_DEXTEROUS_HAND_TACTILE_NUM

Number of tactile regions supported by Xynova Dexterous Hand.

XYNOVA_DEXTEROUS_HAND_TACTILE_TOTAL_TAXEL_NUM

Total number of tactile taxels supported by Xynova Dexterous Hand.

XYNOVA_DEXTEROUS_HAND_RANGE_POINT_NUM

Number of range points supported by the default range sensor.

Typedefs

```
typedef uint16_t XynovaDexterousHandErrorCode
```

Dexterous hand error code type.

Stored as uint16_t to guarantee ABI compatibility.

Each bit represents a joint error condition.

typedef uint32_t XynovaDexterousHandWarningFlags

Dexterous hand global warning flags.

Stored as uint32_t to guarantee ABI compatibility.

Each bit represents a global warning condition.

typedef uint32_t XynovaDexterousHandFaultFlags

Dexterous hand global fault flags.

Stored as uint32_t to guarantee ABI compatibility.

Each bit represents a global fault condition.

typedef struct *XynovaDexterousHandErrorState* XynovaDexterousHandErrorState

Dexterous hand error state.

Provides global warning, global fault and joint error information.

typedef uint8_t XynovaDexterousHandType

Dexterous hand type.

Stored as uint8_t to guarantee ABI compatibility.

typedef uint8_t XynovaDexterousHandJointID

Dexterous hand joint identifier type.

Stored as uint8_t to guarantee ABI compatibility.

typedef uint8_t XynovaDexterousHandTactileID

Dexterous hand tactile region identifier type.

Stored as uint8_t to guarantee ABI compatibility.

typedef uint8_t XynovaDexterousHandWrenchDimension

Tactile force dimension type.

Stored as uint8_t to guarantee ABI compatibility.

typedef uint8_t XynovaDexterousHandControlMode

Dexterous hand control mode type.

Stored as uint8_t to guarantee ABI compatibility.

typedef uint8_t XynovaDexterousHandRangePointStatus

Range point status type.

Stored as uint8_t to guarantee ABI compatibility.

typedef struct *XynovaDexterousHandControlModeCommand* XynovaDexterousHandControlModeCommand

Joint control mode command.

Each element contains:

- id: Joint identifier
- type: *XYNOVA_DATA_TYPE_U8*
- value.u8: *XynovaDexterousHandControlMode*

typedef struct *XynovaDexterousHandAngleControl* **XynovaDexterousHandAngleControl**

Joint angle control command.

Each element contains:

- id: Joint identifier
- type: *XYNOVA_DATA_TYPE_FLOAT*
- value.f32: Joint angle in radians

typedef struct *XynovaDexterousHandAngleStates* **XynovaDexterousHandAngleStates**

Joint angle feedback states.

Each element contains:

- id: Joint identifier
- type: *XYNOVA_DATA_TYPE_FLOAT*
- value.f32: Joint angle in radians

typedef struct *XynovaDexterousHandTactileWrench* **XynovaDexterousHandTactileWrench**

Tactile resultant force.

typedef struct *XynovaDexterousHandTactileWrenchStates* **XynovaDexterousHandTactileWrenchStates**

Tactile resultant force states.

typedef struct *XynovaDexterousHandTactileTaxel* **XynovaDexterousHandTactileTaxel**

Tactile taxel force.

typedef struct *XynovaDexterousHandTactileTaxelStates* **XynovaDexterousHandTactileTaxelStates**

Tactile taxel states.

The taxels buffer is managed by the caller.

typedef struct *XynovaDexterousHandRangePoint* **XynovaDexterousHandRangePoint**

Range point measurement.

Distance unit: meter.

typedef struct *XynovaDexterousHandRangeStates* **XynovaDexterousHandRangeStates**

Range measurement states.

The points buffer is managed by the caller.

Points are stored in row-major order: $\text{index} = y * \text{width} + x$

Enums

enum [**anonymous**]

Dexterous hand joint error flags.

Values:

enumerator **XYNOVA_DEXTEROUS_HAND_ERROR_NONE**

No error

enumerator **XYNOVA_DEXTEROUS_HAND_ERROR_STALL_PROTECTION**

Motor stall protection

enumerator **XYNOVA_DEXTEROUS_HAND_ERROR_OVER_TEMPERATURE**

Over temperature protection

enumerator **XYNOVA_DEXTEROUS_HAND_ERROR_OVER_CURRENT**

Over current protection

enumerator **XYNOVA_DEXTEROUS_HAND_ERROR_MOTOR_ABNORMAL**

Motor abnormal

enumerator **XYNOVA_DEXTEROUS_HAND_ERROR_COMMUNICATION**

Communication error

enumerator **XYNOVA_DEXTEROUS_HAND_ERROR_ENCODER_ABNORMAL**

Encoder abnormal

enum [**anonymous**]

Dexterous hand global warning flags.

Values:

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_NONE**

No warning

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_ANGLE_SENSOR_COMMUNICATION**

Angle sensor communication abnormal

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_TACTILE_SENSOR_COMMUNICATION**

Tactile sensor communication abnormal

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_RANGE_SENSOR_COMMUNICATION**

Range (dToF) sensor communication abnormal

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_MAINBOARD_OVER_TEMPERATURE**

Main control board over temperature

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_MAINBOARD_UNDER_TEMPERATURE**

Main control board under temperature

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_SUBBOARD_OVER_TEMPERATURE**

Sub board over temperature

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_SUBBOARD_UNDER_TEMPERATURE**

Sub board under temperature

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_CANFD1_STATUS**

CAN FD1 status warning

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_CANFD2_STATUS**

CAN FD2 status warning

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_FIRMWARE_UPGRADE**

Firmware upgrade warning

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_PDM_FRAME_LOSS**

PDM frame loss

enumerator **XYNOVA_DEXTEROUS_HAND_WARNING_CONTROL_LOOP_TIMEOUT**

Control loop timeout

enum **[anonymous]**

Dexterous hand global fault flags.

Values:

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_NONE**

No fault

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_SUBBOARD_COMMUNICATION**

Sub board communication abnormal

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_ACTUATOR_COMMUNICATION**

Actuator communication abnormal

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_ANGLE_SENSOR_COMMUNICATION**

Angle sensor communication abnormal

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_TACTILE_SENSOR_COMMUNICATION**

Tactile sensor communication abnormal

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_RANGE_SENSOR_COMMUNICATION**

Range (dToF) sensor communication abnormal

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_OVER_TEMPERATURE**

Main control board over temperature

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_UNDER_TEMPERATURE**

Main control board under temperature

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_SUBBOARD_OVER_TEMPERATURE**

Sub board over temperature

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_SUBBOARD_UNDER_TEMPERATURE**

Sub board under temperature

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_48V_OVER_VOLTAGE**

Main board 48V over voltage

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_24V_OVER_VOLTAGE**

Main board 24V over voltage

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_48V_UNDER_VOLTAGE**

Main board 48V under voltage

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_24V_UNDER_VOLTAGE**

Main board 24V under voltage

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_OVER_CURRENT1**

Main board over current channel 1

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_OVER_CURRENT2**

Main board over current channel 2

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_OVER_CURRENT3**

Main board over current channel 3

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_CANFD1_STATUS**

CAN FD1 status fault

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_CANFD2_STATUS**

CAN FD2 status fault

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_PARAMETER_STORAGE**

Parameter storage fault

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_ACTUATOR_OVER_TEMPERATURE**

Actuator over temperature

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_ACTUATOR_OVER_CURRENT**

Actuator over current

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_ACTUATOR_UNDER_VOLTAGE**

Actuator under voltage

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_ACTUATOR_OVER_VOLTAGE**

Actuator over voltage

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_PDM_FRAME_LOSS**

PDM frame loss

enumerator **XYNOVA_DEXTEROUS_HAND_FAULT_CONTROL_LOOP_TIMEOUT**

Control loop timeout

enum **[anonymous]**

Dexterous hand types.

Values:

enumerator **XYNOVA_DEXTEROUS_HAND_TYPE_UNKNOWN**

Unknown hand type

enumerator **XYNOVA_DEXTEROUS_HAND_TYPE_LEFT**

Left hand

enumerator **XYNOVA_DEXTEROUS_HAND_TYPE_RIGHT**

Right hand

enum **[anonymous]**

Dexterous hand joint identifiers.

Values:

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF1**

Index fingertip flex (Passive)

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF2**

Index middle flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF3**
Index base flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF4**
Index abduction

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF1**
Middle fingertip flex (Passive)

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF2**
Middle middle flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF3**
Middle base flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF4**
Middle abduction

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF1**
Ring fingertip flex (Passive)

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF2**
Ring middle flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF3**
Ring base flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF4**
Ring abduction

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF1**
Little fingertip flex (Passive)

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF2**
Little middle flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF3**
Little base flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF4**
Little abduction

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF5**
Little rotation

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH1**
Thumb tip flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH2**
Thumb middle flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH3**
Thumb base flex

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH4**
Thumb abduction

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_WR1**
Wrist pitch

enumerator **XYNOVA_DEXTEROUS_HAND_JOINT_ID_WR2**
Wrist yaw

enum **[anonymous]**
Dexterous hand tactile region identifiers.

Values:

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_TT1**
Thumb fingertip

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_TT2**
Thumb pad

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT1**
Index fingertip

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT2**
Index middle phalanx

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT3**
Index proximal phalanx

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_MT1**
Middle fingertip

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_MT2**
Middle middle phalanx

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_MT3**

Middle proximal phalanx

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_RT1**

Ring fingertip

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_RT2**

Ring middle phalanx

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_RT3**

Ring proximal phalanx

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_LT1**

Little fingertip

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_LT2**

Little middle phalanx

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_LT3**

Little proximal phalanx

enumerator **XYNOVA_DEXTEROUS_HAND_TACTILE_ID_PT1**

Palm

enum **[anonymous]**

Tactile wrench dimensions.

Values:

enumerator **XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_1D**

One-dimensional wrench

enumerator **XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_3D**

Three-dimensional wrench

enumerator **XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_6D**

Six-dimensional wrench

enum **[anonymous]**

Dexterous hand control modes.

Values:

enumerator **XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_DISABLE**

Control disabled

enumerator **XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_ANGLE**

Angle control

enum [**anonymous**]

Range point measurement status.

Values:

enumerator **XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_NO_TARGET**

No target detected

enumerator **XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_WEAK_SIGNAL**

Weak signal

enumerator **XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_VALID**

Valid measurement

enumerator **XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_INVALID**

Invalid measurement

struct **XynovaDexterousHandErrorState**

#include <dexterous_hand_types.h> Dexterous hand error state.

Provides global warning, global fault and joint error information.

Public Members

uint64_t **timestamp_us**

Error timestamp (us): system monotonic clock

XynovaDexterousHandWarningFlags **warning_flags**

Global warning flags

XynovaDexterousHandFaultFlags **fault_flags**

Global fault flags

XynovaIdValue **joint_error_code**[XYNOVA_DEXTEROUS_HAND_JOINT_NUM]

Joint error codes

struct **XynovaDexterousHandControlModeCommand**

#include <dexterous_hand_types.h> Joint control mode command.

Each element contains:

- id: Joint identifier
- type: *XYNOVA_DATA_TYPE_U8*
- value.u8: *XynovaDexterousHandControlMode*

Public Members

XynovaIdValue **values**[XYNOVA_DEXTEROUS_HAND_JOINT_NUM]

Joint control mode values

struct **XynovaDexterousHandAngleControl**

#include <dexterous_hand_types.h> Joint angle control command.

Each element contains:

- id: Joint identifier
- type: *XYNOVA_DATA_TYPE_FLOAT*
- value.f32: Joint angle in radians

Public Members

XynovaIdValue **values**[XYNOVA_DEXTEROUS_HAND_JOINT_NUM]

Joint angle commands

struct **XynovaDexterousHandAngleStates**

#include <dexterous_hand_types.h> Joint angle feedback states.

Each element contains:

- id: Joint identifier
- type: *XYNOVA_DATA_TYPE_FLOAT*
- value.f32: Joint angle in radians

Public Members

XynovaIdValue **values**[XYNOVA_DEXTEROUS_HAND_JOINT_NUM]

Joint angle states

struct **XynovaDexterousHandTactileWrench**

#include <dexterous_hand_types.h> Tactile resultant force.

Public Members

uint64_t **timestamp_us**

Sampling timestamp (us): system monotonic clock

XynovaDexterousHandTactileID **region**

Tactile region

XynovaDexterousHandWrenchDimension **dimension**

Force dimension

XynovaTypedValueArray6 **wrench**

Force/torque values: force unit: N, torque unit: Nm

struct **XynovaDexterousHandTactileWrenchStates**

#include <dexterous_hand_types.h> Tactile resultant force states.

Public Members

XynovaDexterousHandTactileWrench **values**[XYNOVA_DEXTEROUS_HAND_TACTILE_NUM]

struct **XynovaDexterousHandTactileTaxel**

#include <dexterous_hand_types.h> Tactile taxel force.

Public Members

uint64_t **timestamp_us**

Sampling timestamp (us): system monotonic clock

XynovaDexterousHandTactileID **region**

Tactile region

uint16_t **taxel_id**

Taxel identifier

XynovaDexterousHandWrenchDimension **dimension**

Force dimension

uint8_t **reserved**

Reserved

XynovaTypedValueArray6 **wrench**

Force/torque values: force unit: N, torque unit: Nm

struct **XynovaDexterousHandTactileTaxelStates**

#include <dexterous_hand_types.h> Tactile taxel states.

The taxels buffer is managed by the caller.

Public Members

uint32_t **count**

Number of taxels

XynovaDexterousHandTactileTaxel ***taxels**

Taxel array

```
struct XynovaDexterousHandRangePoint
```

#include <dexterous_hand_types.h> Range point measurement.

Distance unit: meter.

Public Members

```
uint64_t timestamp_us
```

Measurement timestamp (us): system monotonic clock

```
uint16_t id
```

Point identifier

```
XynovaDexterousHandRangePointStatus status
```

Measurement status

```
uint8_t reserved
```

Reserved

```
float distance
```

Distance in meter

```
struct XynovaDexterousHandRangeStates
```

#include <dexterous_hand_types.h> Range measurement states.

The points buffer is managed by the caller.

Points are stored in row-major order: $\text{index} = y * \text{width} + x$

Public Members

```
uint16_t width
```

Range map width

```
uint16_t height
```

Range map height

```
uint32_t count
```

Number of range points

```
XynovaDexterousHandRangePoint *points
```

Range point buffer

4.2.2.1.6 Infrastructure (C API)

The **Infrastructure module** provides foundational utilities used across the Xynova SDK. This includes logging, error handling, and common value helpers that support consistent and reliable behavior for devices and communication layers.

Key features:

- Logging system (sinks, writers, formatters, and backends)
- Standardized error codes and result types
- Common value and result wrappers for cross-module data exchange
- ABI-safe base types and symbol visibility macros
- SDK version information

Public headers

The externally exposed C headers for this module are:

- `xynova/c/base_types.h` —ABI-safe base types (`XynovaBool`)
- `xynova/c/abi.h` —symbol visibility and ABI control macros
- `xynova/c/error_code.h` —standardized error code definitions (see also the [error codes](#) reference)
- `xynova/c/result.h` —result wrapper helpers
- `xynova/c/value_types.h` —typed value and identifier container types
- `xynova/c/version.h` —SDK version information

The logging entry points are documented on the [logging](#) page:

- `xynova/c/log_api.h` —global and object logger entry points
- `xynova/c/log_types.h` —log level, record, sink, writer, and logger configuration types

API Reference

Fundamental ABI-safe type definitions for the Xynova SDK.

Defines common scalar types shared by all public C APIs.

All public types use fixed-width integers to guarantee ABI compatibility.

Version

1.0.0

Date

2025-10-29

Author

Xynova

Typedefs

```
typedef uint8_t XynovaBool
```

ABI-safe boolean type.

Stored as `uint8_t` for cross-platform compatibility.

Valid values are `XYNOVA_FALSE` and `XYNOVA_TRUE`.

Enums

enum **[anonymous]**

Boolean constants.

Values:

enumerator **XYNOVA_FALSE**

Boolean false.

enumerator **XYNOVA_TRUE**

Boolean true.

Definition of symbol visibility and ABI control macros for the Xynova SDK.

This header defines platform-independent macros that control symbol visibility, ABI stability, and linkage behavior across different compilers and operating systems. It ensures that public SDK interfaces are exported properly, while internal symbols remain hidden to maintain binary compatibility.

The macros in this file provide consistent export/import rules for both Windows and POSIX-based systems (Linux/macOS), making shared library integration straightforward.

Defines

XYNOVA_API

When ABI stability control is disabled, all symbols are public.

XYNOVA_LOCAL

XYNOVA_IMPL_API

XYNOVA_INLINE_API

Defines inline functions with hidden visibility for internal usage.

Used to prevent unnecessary symbol export from inline helper functions. On GCC/Clang, adds `__attribute__((visibility("hidden")))`.

Note

C++17 `constexpr` functions are implicitly inline.

XYNOVA_NOEXCEPT

Provides a consistent `noexcept` macro for function declarations.

XYNOVA_CONSTEXPR

Provides a consistent `constexpr` macro for function or variable declarations.

XYNOVA_EXPORT_TEMPLATE (class_name, ...)

Explicitly exports template class instantiations for shared library usage.

```
template class XYNOVA_API MyTemplate<int>;
```

Parameters

- **class_name** – The name of the template class.
- ... – The template type arguments.

Definition of Xynova SDK error codes and conversion utilities.

Provides stable error code definitions and string conversion support.

Error codes are maintained through X-Macro for centralized management.

Version

1.0.0

Date

2025-10-29

Author

Xynova

Defines**XYNOVA_ERROR_CODE_LIST**

Central definition list of SDK error codes.

Format: X(NAME, VALUE, DESCRIPTION)

X (name, value, desc)

desc

X (name, value, desc)

desc

Typedefs

typedef int32_t **XynovaErrorCode**

Xynova SDK error code type.

Uses int32_t to guarantee ABI compatibility.

Enums

enum [**anonymous**]

Xynova SDK error code constants.

Generated from XYNOVA_ERROR_CODE_LIST.

Values:

enumerator **XYNOVA_ERROR_CODE_LIST**

Functions

static inline const char ***xynovaErrorCodeToString** (*XynovaErrorCode* code)

Convert error code to string.

Parameters

code – Error code.

Returns

Static string describing the error.

Definition of *XynovaResult* structure for representing operation outcomes.

Defines *XynovaResult* for returning operation status and optional message. Designed for stable C ABI usage across shared libraries.

Features:

- Standardized operation result representation.
- Combines *XynovaErrorCode* with an optional message.
- Compatible with C and C++ callers.

See also

xynova/c/error_code.h

Version

1.0.0

Date

2025-10-09

Author

Xynova

Typedefs

typedef struct *XynovaResult* **XynovaResult**

struct **XynovaResult**

#include <result.h> Represents the result of an SDK operation.

Contains an error code and optional descriptive message.

Note

The message pointer does not transfer ownership. The pointed string must remain valid while the result is used.

Public Members

XynovaErrorCode **code**

Operation result code.

const char ***message**

Optional descriptive message.

Generic value type definitions for the Xynova C API.

Defines generic scalar and fixed-size vector value containers used by the Xynova SDK. All public structures use fixed-width integer types to maintain ABI compatibility.

Version

1.0.0

Date

2026-07-17

Author

Xynova

Typedefs

typedef uint8_t **XynovaDataType**

Value type identifier.

Stored as uint8_t to guarantee ABI compatibility.

typedef union *XynovaValue* **XynovaValue**

Generic scalar value storage.

The active member is selected by *XynovaDataType*.

typedef struct *XynovaTypedValue* **XynovaTypedValue**

Typed scalar value.

typedef struct *XynovaTypedValueArray2* **XynovaTypedValueArray2**

Typed two-element value.

Commonly used for 2-D vector data.

```
typedef struct XynovaTypedValueArray3 XynovaTypedValueArray3
```

Typed three-element value.

Commonly used for 3-D vector data.

```
typedef struct XynovaTypedValueArray4 XynovaTypedValueArray4
```

Typed four-element value.

Commonly used for 4-D vector data.

```
typedef struct XynovaTypedValueArray5 XynovaTypedValueArray5
```

Typed five-element value.

Commonly used for 5-D vector data.

```
typedef struct XynovaTypedValueArray6 XynovaTypedValueArray6
```

Typed six-element value.

Commonly used for 6-DoF vector data.

```
typedef struct XynovaIdValue XynovaIdValue
```

Identifier with typed scalar value.

```
typedef struct XynovaIdValueArray2 XynovaIdValueArray2
```

Identifier with typed two-element value.

```
typedef struct XynovaIdValueArray3 XynovaIdValueArray3
```

Identifier with typed three-element value.

```
typedef struct XynovaIdValueArray4 XynovaIdValueArray4
```

Identifier with typed four-element value.

```
typedef struct XynovaIdValueArray5 XynovaIdValueArray5
```

Identifier with typed five-element value.

```
typedef struct XynovaIdValueArray6 XynovaIdValueArray6
```

Identifier with typed six-element value.

Enums

```
enum [anonymous]
```

Value type identifiers.

Values are explicitly assigned for protocol and ABI compatibility.

Values:

enumerator **XYNOVA_DATA_TYPE_NONE**

Undefined type

enumerator **XYNOVA_DATA_TYPE_BOOL**

Boolean value

enumerator **XYNOVA_DATA_TYPE_U8**

Unsigned 8-bit integer

enumerator **XYNOVA_DATA_TYPE_S8**

Signed 8-bit integer

enumerator **XYNOVA_DATA_TYPE_U16**

Unsigned 16-bit integer

enumerator **XYNOVA_DATA_TYPE_S16**

Signed 16-bit integer

enumerator **XYNOVA_DATA_TYPE_U32**

Unsigned 32-bit integer

enumerator **XYNOVA_DATA_TYPE_S32**

Signed 32-bit integer

enumerator **XYNOVA_DATA_TYPE_U64**

Unsigned 64-bit integer

enumerator **XYNOVA_DATA_TYPE_S64**

Signed 64-bit integer

enumerator **XYNOVA_DATA_TYPE_FLOAT**

32-bit floating point

enumerator **XYNOVA_DATA_TYPE_DOUBLE**

64-bit floating point

union **XynovaValue**

#include <value_types.h> Generic scalar value storage.

The active member is selected by *XynovaDataType*.

Public Members

uint8_t u8

Unsigned 8-bit value

`int8_t s8`

Signed 8-bit value

`uint16_t u16`

Unsigned 16-bit value

`int16_t s16`

Signed 16-bit value

`uint32_t u32`

Unsigned 32-bit value

`int32_t s32`

Signed 32-bit value

`uint64_t u64`

Unsigned 64-bit value

`int64_t s64`

Signed 64-bit value

`float f32`

32-bit floating point value

`double f64`

64-bit floating point value

struct **XynovaTypedValue**

#include <value_types.h> Typed scalar value.

Public Members

XynovaDataType **type**

Value type

XynovaValue **value**

Scalar value

struct **XynovaTypedValueArray2**

#include <value_types.h> Typed two-element value.

Commonly used for 2-D vector data.

Public Members*XynovaDataType* **type**

Value type

XynovaValue **values[2]**

Value elements

struct **XynovaTypedValueArray3***#include <value_types.h>* Typed three-element value.

Commonly used for 3-D vector data.

Public Members*XynovaDataType* **type**

Value type

XynovaValue **values[3]**

Value elements

struct **XynovaTypedValueArray4***#include <value_types.h>* Typed four-element value.

Commonly used for 4-D vector data.

Public Members*XynovaDataType* **type**

Value type

XynovaValue **values[4]**

Value elements

struct **XynovaTypedValueArray5***#include <value_types.h>* Typed five-element value.

Commonly used for 5-D vector data.

Public Members*XynovaDataType* **type**

Value type

XynovaValue **values[5]**

Value elements

struct **XynovaTypedValueArray6**

#include <value_types.h> Typed six-element value.

Commonly used for 6-DoF vector data.

Public Members

XynovaDataType **type**

Value type

XynovaValue **values**[6]

Value elements

struct **XynovaIdValue**

#include <value_types.h> Identifier with typed scalar value.

Public Members

uint16_t **id**

Value identifier

XynovaTypedValue **data**

Typed scalar value

struct **XynovaIdValueArray2**

#include <value_types.h> Identifier with typed two-element value.

Public Members

uint16_t **id**

Value identifier

XynovaTypedValueArray2 **data**

Typed two-element value

struct **XynovaIdValueArray3**

#include <value_types.h> Identifier with typed three-element value.

Public Members

uint16_t **id**

Value identifier

XynovaTypedValueArray3 **data**

Typed three-element value

```
struct XynovaIdValueArray4
```

#include <value_types.h> Identifier with typed four-element value.

Public Members

```
uint16_t id
```

Value identifier

```
XynovaTypedValueArray4 data
```

Typed four-element value

```
struct XynovaIdValueArray5
```

#include <value_types.h> Identifier with typed five-element value.

Public Members

```
uint16_t id
```

Value identifier

```
XynovaTypedValueArray5 data
```

Typed five-element value

```
struct XynovaIdValueArray6
```

#include <value_types.h> Identifier with typed six-element value.

Public Members

```
uint16_t id
```

Value identifier

```
XynovaTypedValueArray6 data
```

Typed six-element value

SDK version and build metadata interface for Xynova SDK.

This header provides functions to query the SDK version, Git branch, Git commit hash, and build timestamp. The version information is automatically injected at build time by CMake.

Defines

```
XYNOVA_SDK_VERSION
```

SDK semantic version string injected at build time.

```
XYNOVA_SDK_LOCAL_VERSION
```

SDK semantic local version string injected at build time.

XYNova_SDK_GIT_BRANCH

Git branch name of the SDK build.

XYNova_SDK_GIT_COMMIT

Git commit hash corresponding to this SDK build.

XYNova_SDK_BUILD_TIME

ISO 8601 timestamp of the SDK build.

Functions

const char ***xynovaGetSdkVersion**() noexcept

Get the SDK semantic version string.

Returns

Null-terminated string (e.g., "1.4.0").

const char ***xynovaGetSdkLocalVersion**() noexcept

Get the SDK semantic local version string.

Returns

Null-terminated string (e.g., "#1").

const char ***xynovaGetSdkGitBranch**() noexcept

Get the Git branch name used for this build.

Returns

Null-terminated string (e.g., "main" or "develop").

const char ***xynovaGetSdkGitCommit**() noexcept

Get the Git commit hash of the build.

Returns

Null-terminated string (e.g., "a1b2c3d4e5f6").

const char ***xynovaGetSdkBuildTime**() noexcept

Get the build timestamp in ISO 8601 format.

Returns

Null-terminated string (e.g., "2025-10-29T12:34:56Z").

4.2.2.1.7 Logging (C API)

The **Logging module** provides a flexible and high-performance logging infrastructure for Xynova devices and SDK components.

It supports:

- Multiple sinks (thread-pool, file, console)
- Custom formatters
- Logger backends with configurable buffer pools, flush intervals, and message filtering
- Level-based logging

The following header files define the C API:

Public C API for the Xynova SDK logging subsystem.

Provides global logging control and object-based logging management APIs. Supports configurable log levels, callbacks, sinks, writers, formatters and logger backends.

Version

1.0.0

Date

2025-10-29

Author

Xynova

Functions

void **XynovaLogGlobalLogger_init** (*XynovaLogLevel* log_level, *XynovaLogLogHandler* log_handler)

Initialize global logging subsystem.

Parameters

- **log_level** – Initial log level.
- **log_handler** – Log callback handler.

XynovaLogLevel **XynovaLogGlobalLogger_setLogLevel** (*XynovaLogLevel* level)

Set global logging level.

Parameters

level – New log level.

Returns

Previous log level.

XynovaLogLevel **XynovaLogGlobalLogger_getLogLevel** (void)

Get global logging level.

Returns

Current log level.

XynovaErrorCode **XynovaLogObjectLogger_initDefaultObjects** (void)

Initialize default logging objects.

Creates default sinks, writers, formatters and logger backend.

Returns

XynovaErrorCode.

XynovaBool **XynovaLogObjectLogger_isInitialized** (void)

Check whether object logger subsystem is initialized.

Returns

XYNOVA_TRUE if initialized, otherwise XYNOVA_FALSE.

XynovaErrorCode **XynovaLogObjectLogger_createSinkTP** (const char *type, const char *name, const *XynovaLogSinkTPConfiguration* *conf)

Create thread-pool based sink.

Parameters

- **type** – Sink type.
- **name** – Sink instance name.
- **conf** – Sink configuration.

Returns

XynovaErrorCode.

XynovaErrorCode **XynovaLogObjectLogger_createFileWriter** (const char *type, const char *name, const *XynovaLogFileWriterConfiguration* *conf)

Create file writer.

Parameters

- **type** – Writer type.
- **name** – Writer instance name.
- **conf** – Writer configuration.

Returns

XynovaErrorCode.

XynovaErrorCode **XynovaLogObjectLogger_createConsoleWriter** (const char *type, const char *name, const *XynovaLogConsoleWriterConfiguration* *conf)

Create console writer.

Parameters

- **type** – Writer type.
- **name** – Writer instance name.
- **conf** – Writer configuration.

Returns

XynovaErrorCode.

XynovaErrorCode **XynovaLogObjectLogger_createFormatter** (const char *type, const char *name, const *XynovaLogFormatterConfiguration* *conf)

Create formatter.

Parameters

- **type** – Formatter type.
- **name** – Formatter instance name.
- **conf** – Formatter configuration.

Returns

XynovaErrorCode.

XynovaErrorCode **XynovaLogObjectLogger_createLogger** (const char *name, const *XynovaLogLoggerConfiguration* *conf)

Create logger backend.

Parameters

- **name** – Logger instance name.

- **conf** – Logger configuration.

Returns

XynovaErrorCode.

XynovaBool **XynovaLogObjectLogger_hasSink** (const char *name)

Check whether sink exists.

Parameters**name** – Sink instance name.**Returns**

XYNOVA_TRUE if exists, otherwise XYNOVA_FALSE.

XynovaBool **XynovaLogObjectLogger_hasWriter** (const char *name)

Check whether writer exists.

Parameters**name** – Writer instance name.**Returns**

XYNOVA_TRUE if exists, otherwise XYNOVA_FALSE.

XynovaBool **XynovaLogObjectLogger_hasFormatter** (const char *name)

Check whether formatter exists.

Parameters**name** – Formatter instance name.**Returns**

XYNOVA_TRUE if exists, otherwise XYNOVA_FALSE.

XynovaBool **XynovaLogObjectLogger_hasLogger** (const char *name)

Check whether logger backend exists.

Parameters**name** – Logger backend instance name.**Returns**

XYNOVA_TRUE if exists, otherwise XYNOVA_FALSE.

XynovaErrorCode **XynovaLogObjectLogger_setLoggerLevel** (const char *name, *XynovaLogLevel* level)

Set logger backend log level.

Parameters

- **name** – Logger backend instance name.
- **level** – New logger level.

Returns

XynovaErrorCode.

Core data types for the Xynova SDK logging subsystem.

Defines logging structures, severity levels, callbacks and configuration types used by the Xynova SDK logging module.

Version

1.0.0

Date

2025-10-29

Author

Xynova

Typedefstypedef uint8_t **XynovaLogLevel**

Logging severity level type.

Stored as uint8_t to guarantee ABI compatibility.

typedef struct *XynovaLogDateTimeRaw* **XynovaLogDateTimeRaw**

Raw timestamp representation.

Uses UNIX epoch time with microsecond precision.

typedef struct *XynovaLogLogRecordMeta* **XynovaLogLogRecordMeta**

Metadata of a log record.

typedef struct *XynovaLogLogRecord* **XynovaLogLogRecord**

Complete log record.

Delivered to user callback handlers.

typedef void (***XynovaLogLogHandler**)(const *XynovaLogLogRecord* *record)

Log record callback type.

typedef struct *XynovaLogSinkConfiguration* **XynovaLogSinkConfiguration**

Common sink configuration.

Defines queue and worker loop timing parameters.

typedef struct *XynovaLogSinkTPConfiguration* **XynovaLogSinkTPConfiguration**

Thread-pool sink configuration.

typedef struct *XynovaLogWriterConfiguration* **XynovaLogWriterConfiguration**

Base writer configuration.

typedef struct *XynovaLogFileWriterConfiguration* **XynovaLogFileWriterConfiguration**

File writer configuration.

typedef struct *XynovaLogConsoleWriterConfiguration* **XynovaLogConsoleWriterConfiguration**

Console writer configuration.

typedef struct *XynovaLogFormatterConfiguration* **XynovaLogFormatterConfiguration**

Formatter configuration.

Reserved for future extension.

typedef struct *XynovaLogLoggerBackendConfiguration* **XynovaLogLoggerBackendConfiguration**
 Logger backend configuration.

typedef struct *XynovaLogLoggerConfiguration* **XynovaLogLoggerConfiguration**
 Logger creation configuration.
 Defines the relationship between sink, writers and backend logger.

Enums

enum **[anonymous]**
 Logging severity levels.
Values:

enumerator **XYNOVA_LOG_LEVEL_TRACE**
 Fine-grained diagnostic tracing.

enumerator **XYNOVA_LOG_LEVEL_DEBUG**
 Debug information.

enumerator **XYNOVA_LOG_LEVEL_INFO**
 Normal information messages.

enumerator **XYNOVA_LOG_LEVEL_WARN**
 Warning messages.

enumerator **XYNOVA_LOG_LEVEL_ERROR**
 Error messages.

enumerator **XYNOVA_LOG_LEVEL_FATAL**
 Fatal failures.

struct **XynovaLogDateTimeRaw**
#include <log_types.h> Raw timestamp representation.
 Uses UNIX epoch time with microsecond precision.

Public Members

uint64_t **seconds**
 Seconds since UNIX epoch UTC.

uint32_t **microseconds**
 Microsecond offset.

struct **XynovaLogLogRecordMeta**

#include <log_types.h> Metadata of a log record.

Public Members

XynovaLogDateTimeRaw **timestamp**

Log timestamp.

XynovaLogLevel **level**

Log severity level.

uint64_t thread_id

Thread identifier.

const char *file

Source file name.

uint32_t line

Source line number.

const char *func

Function name.

struct **XynovaLogLogRecord**

#include <log_types.h> Complete log record.

Delivered to user callback handlers.

Public Members

XynovaLogLogRecordMeta **meta**

Log metadata.

const char *message

UTF-8 message string.

struct **XynovaLogSinkConfiguration**

#include <log_types.h> Common sink configuration.

Defines queue and worker loop timing parameters.

Public Members

uint64_t block_obtain_timeout_us

Block obtain timeout.

`uint64_t writing_task_push_timeout_us`

Write task push timeout.

`uint64_t writing_task_wait_timeout_us`

Write task wait timeout.

`uint64_t min_looping_task_idle_interval_us`

Minimum idle interval.

`uint64_t max_looping_task_idle_interval_us`

Maximum idle interval.

`uint32_t max_continuous_push_retry_time_ms`

Maximum retry duration.

struct **XynovaLogSinkTPConfiguration**

#include <log_types.h> Thread-pool sink configuration.

Public Members

XynovaLogSinkConfiguration **base_conf**

Common sink configuration.

`uint32_t num_threads`

Worker thread count.

`uint32_t max_queue_size`

Maximum queue size.

struct **XynovaLogWriterConfiguration**

#include <log_types.h> Base writer configuration.

Public Members

`const char *formater`

Formatter instance name.

XynovaBool **auto_flush**

Enable automatic flush.

`uint32_t flush_interval_ms`

Flush interval in milliseconds.

struct **XynovaLogFileWriterConfiguration**
#include <log_types.h> File writer configuration.

Public Members

XynovaLogWriterConfiguration **base_conf**
 Base writer configuration.

const char ***file_path**
 Output file path.

uint64_t **max_file_size**
 Maximum file size before rotation.

uint32_t **max_roll_files**
 Number of retained files.

struct **XynovaLogConsoleWriterConfiguration**
#include <log_types.h> Console writer configuration.

Public Members

XynovaLogWriterConfiguration **base_conf**
 Base writer configuration.

XynovaBool **use_stderr**
 Output to stderr when true.

struct **XynovaLogFormatterConfiguration**
#include <log_types.h> Formatter configuration.
 Reserved for future extension.

Public Members

uint32_t **reserved**
 Reserved field.

struct **XynovaLogLoggerBackendConfiguration**
#include <log_types.h> Logger backend configuration.

Public Members

uint32_t **pool_size**
 Log memory pool size.

`uint32_t block_size`

Records per block.

`uint32_t message_size`

Maximum message size.

`uint32_t max_continuous_repeated_log`

Maximum repeated logs.

`uint32_t flush_interval_us`

Periodic flush interval.

`float queue_size_multiple`

Queue size multiplier.

struct **XynovaLogLoggerConfiguration**

#include <log_types.h> Logger creation configuration.

Defines the relationship between sink, writers and backend logger.

Public Members

`const char *sink_name`

Sink instance name.

`const char **writer_name_list`

NULL-terminated writer names.

`const char *lb_type`

Backend logger type.

XynovaLogLevel `level`

Minimum output level.

XynovaLogLoggerBackendConfiguration `lb_config`

Backend configuration.

4.2.2.1.8 Error Codes

The **XynovaErrorCode** is the standardized error code type returned by the public C and C++ API functions. It is stored as `int32_t` to guarantee ABI compatibility.

Use `xynovaErrorCodeToString()` to convert an error code into a human-readable string.

General

Constant	Value	Description
XYNOVA_ERROR_CODE_OK	0	No error
XYNOVA_ERROR_CODE_UNKNOWN	-1	Unknown error
XYNOVA_ERROR_CODE_INVALID_ARG	-2	Invalid argument
XYNOVA_ERROR_CODE_NOT_IMPLEMENTED	-3	Functionality not implemented
XYNOVA_ERROR_CODE_TIMEOUT	-4	Operation timed out
XYNOVA_ERROR_CODE_NOT_FOUND	-5	Resource not found
XYNOVA_ERROR_CODE_ALREADY_EXISTS	-6	Resource already exists
XYNOVA_ERROR_CODE_NO_MEMORY	-7	Memory allocation failed
XYNOVA_ERROR_CODE_IO	-8	Input/output error
XYNOVA_ERROR_CODE_BUSY	-9	Resource is busy
XYNOVA_ERROR_CODE_INTERRUPTED	-10	Operation interrupted
XYNOVA_ERROR_CODE_OUT_OF_RANGE	-11	Value out of allowed range
XYNOVA_ERROR_CODE_PERMISSION_DENIED	-12	Permission denied
XYNOVA_ERROR_CODE_UNSUPPORTED	-13	Operation unsupported
XYNOVA_ERROR_CODE_ALREADY_INITIALIZED	-14	Already initialized
XYNOVA_ERROR_CODE_DATA_OUT_OF_DATE	-15	Data is out of date
XYNOVA_ERROR_CODE_NO_DATA_UPDATED	-16	No data updated
XYNOVA_ERROR_CODE_INTERNAL_ERROR	-17	Internal SDK configuration or logic error

Logging

Constant	Value	Description
XYNOVA_ERROR_CODE_LOG_SINK_ALREADY_EXISTS	-1000	Log sink already exists
XYNOVA_ERROR_CODE_LOG_WRITER_ALREADY_EXISTS	-1001	Log writer already exists
XYNOVA_ERROR_CODE_LOG_FORMATTER_ALREADY_EXISTS	-1002	Log formatter already exists
XYNOVA_ERROR_CODE_LOG_LOGGER_ALREADY_EXISTS	-1003	Log logger already exists
XYNOVA_ERROR_CODE_LOG_SINK_NOT_FOUND	-1010	Log sink not found
XYNOVA_ERROR_CODE_LOG_WRITER_NOT_FOUND	-1011	Log writer not found
XYNOVA_ERROR_CODE_LOG_FORMATTER_NOT_FOUND	-1012	Log formatter not found
XYNOVA_ERROR_CODE_LOG_LOGGER_NOT_FOUND	-1013	Log logger not found
XYNOVA_ERROR_CODE_LOG_LOGGER_CREATION_FAILED	-1014	Log logger creation failed

Communication

Constant	Value	Description
XYNOVA_ERROR_CODE_COMM_INIT_FAILED	-2000	Communication initialization failed
XYNOVA_ERROR_CODE_COMM_SHUTDOWN_FAILED	-2001	Communication shutdown failed
XYNOVA_ERROR_CODE_COMM_INVALID_CONFIG	-2002	Invalid communication configuration
XYNOVA_ERROR_CODE_COMM_ALREADY_INITIALIZED	-2003	Communication already initialized
XYNOVA_ERROR_CODE_COMM_NOT_INITIALIZED	-2004	Communication not initialized
XYNOVA_ERROR_CODE_COMM_NOT_LOOPING	-2005	Communication loop not running
XYNOVA_ERROR_CODE_COMM_NO_MAPPING	-2006	No mapping found for communication
XYNOVA_ERROR_CODE_COMM_RT_LOOP_TIMEOUT	-2007	Realtime loop stalled or not running

CAN

Constant	Value	Description
XYNOVA_ERROR_CODE_CAN_DRIVER_NOT_FOUND	-3000	CAN driver not found
XYNOVA_ERROR_CODE_CAN_DRIVER_ALREADY_REGISTERED	-3001	CAN driver already registered
XYNOVA_ERROR_CODE_CAN_INVALID_VTABLE	-3002	Invalid CAN driver vtable
XYNOVA_ERROR_CODE_CAN_OPEN_FAILED	-3003	Failed to open CAN interface
XYNOVA_ERROR_CODE_CAN_CLOSE_FAILED	-3004	Failed to close CAN interface
XYNOVA_ERROR_CODE_CAN_SEND_FAILED	-3005	Failed to send CAN message
XYNOVA_ERROR_CODE_CAN_RECV_FAILED	-3006	Failed to receive CAN message
XYNOVA_ERROR_CODE_CAN_SET_FILTERS_FAILED	-3007	Failed to set CAN filters
XYNOVA_ERROR_CODE_CAN_NOT_OPEN	-3008	CAN interface not open
XYNOVA_ERROR_CODE_CAN_ALREADY_OPEN	-3009	CAN interface already open
XYNOVA_ERROR_CODE_CAN_INVALID_CONFIG	-3010	Invalid CAN configuration
XYNOVA_ERROR_CODE_CAN_NO_DATA	-3011	No CAN data available
XYNOVA_ERROR_CODE_CAN_INVALID_VTABLE_NAME	-3012	Invalid CAN vtable name
XYNOVA_ERROR_CODE_CAN_SET_BUFFER_FAILED	-3013	Failed to set recv/send buffer
XYNOVA_ERROR_CODE_CAN_NO_TX_BUFFERS	-3014	No buffer available for sending messages
XYNOVA_ERROR_CODE_CAN_REQUEST_FAILED	-3015	CAN request failed
XYNOVA_ERROR_CODE_CAN_REQUEST_VERIFY_FAILED	-3016	CAN request verification failed
XYNOVA_ERROR_CODE_CAN_ERROR_FRAME	-3017	Received CAN error frame
XYNOVA_ERROR_CODE_CAN_BUS_OFF	-3018	CAN bus is off
XYNOVA_ERROR_CODE_CAN_BUS_ERROR	-3019	General CAN bus error

Slave

Constant	Value	Description
XYNOVA_ERROR_CODE_SLAVE_CREATE_FAILED	-4000	Failed to create slave
XYNOVA_ERROR_CODE_SLAVE_ADD_SCHEDULE_COMMAND_FAIL	-4001	Failed to add schedule command
XYNOVA_ERROR_CODE_SLAVE_INIT_FAILED	-4002	Failed to init slave

Device

Constant	Value	Description
XYNOVA_ERROR_CODE_DEVICE_CREATE_FAILED	-5000	Failed to create device
XYNOVA_ERROR_CODE_DEVICE_CREATE_SLAVE_FAILED	-5001	Failed to create slave for device
XYNOVA_ERROR_CODE_DEVICE_INIT_SCHEDULE_COMMAND_FAIL	-5002	Failed to initialize schedule command

Dexterous Hand

Constant	Value	Description
XYNOVA_ERROR_CODE_HAND_SET_MODE_FAILED	-6000	Failed to set hand mode
XYNOVA_ERROR_CODE_HAND_SET_SYSTEM_ENABLEMENT_FA	-6001	Failed to set system enablement
XYNOVA_ERROR_CODE_HAND_SET_BACK_TO_ZERO_FAILED	-6002	Failed to reset hand to zero position
XYNOVA_ERROR_CODE_HAND_CLEAR_ERRORS_FAILED	-6003	Failed to clear hand errors
XYNOVA_ERROR_CODE_HAND_NO_DATA_UPDATED	-6004	Hand data not updated
XYNOVA_ERROR_CODE_HAND_DATA_OUT_OF_DATE	-6005	Hand data out of date
XYNOVA_ERROR_CODE_HAND_CREATE_SLAVE_FAILED	-6006	Failed to create hand slave device

Protocol Stack

Constant	Value	Description
XYNOVA_ERROR_CODE_PROTOCOL_STACK_INIT_FAILED	-7000	Failed to initialize protocol stack

See Also

- *Infrastructure (C API)*
- *Logging (C API)*
- *Glossary*
- *Examples*
- *Dexterous Hand (C API)*

4.2.2.2 C++ API

The C++ API modules are:

4.2.2.2.1 C++ API Reference

The **C++ API** provides modern object-oriented wrappers around the C API. It adds:

- RAII-based resource management
- High-level device and logger classes
- Type safety and convenience functions

Overview

The C++ API modules include:

- **Communication:** *Manager* singleton and *DeviceHandle* RAII wrapper.
- **Hand Control:** *dexterous_hand::Device* for joint-level control and state reporting.
- **Infrastructure / Logging:** *LogManager* and related configuration types.

Notes

- C++ API classes are mostly thin wrappers over corresponding C structures and functions.
- Memory and device handle management are automatic (RAII).
- C++ API retains consistent naming and module hierarchy with the C API for easier cross-reference.

4.2.2.2.2 Communication (C++ API)

The **Communication module** provides high-level C++ wrappers for managing Xynova communication interfaces, masters, and devices.

It is built on top of the C API and offers:

- RAII management of device handles
- `Singleton Manager` for master/device initialization and runtime control
- Type-safe interfaces for synchronous and asynchronous device operations
- CAN and transport support through dedicated helper types

Public headers

The externally exposed C++ headers for this module are:

- `xynova/cpp/communication/Manager.hpp` — communication manager singleton and device-handle wrapper
- `xynova/cpp/communication/types.h` — C++ communication configuration types

The CAN transport helper types are documented on the [CAN communication](#) page:

- `xynova/cpp/communication/can/types.h` — CAN transport helper types

API Reference

C++ wrapper for Xynova Communication Manager.

Provides RAII resource management and type-safe C++ interfaces for the Xynova communication manager C API.

The module supports:

- Communication manager initialization
- Master runtime control
- Device handle management
- Firmware upgrade

See also

[XynovaCommunicationManagerConfig](#)

 See also

XynovaCommunicationManager_init

Version

1.0.0

Date

2025-10-29

Author

Xynova

namespace **xynova**

namespace **communication**

class **DeviceHandle**

#include <Manager.hpp> RAII wrapper for C device handle.

Automatically releases the underlying C API handle when the object is destroyed.

Public Functions

inline explicit **DeviceHandle** (*XynovaDeviceHandle* *handle = nullptr)

inline **~DeviceHandle** ()

DeviceHandle (const *DeviceHandle*&) = delete

DeviceHandle &**operator**= (const *DeviceHandle*&) = delete

inline **DeviceHandle** (*DeviceHandle* &&other) noexcept

inline *DeviceHandle* &**operator**= (*DeviceHandle* &&other) noexcept

inline *XynovaDeviceHandle* ***get** () const noexcept

inline explicit **operator bool** () const noexcept

Private Members

XynovaDeviceHandle ***i_handle** = {nullptr}

class **Manager**

#include <Manager.hpp> Communication manager singleton.

Provides high-level C++ interfaces for communication lifecycle, device discovery and firmware operations.

Public Functions

Manager (const *Manager*&) = delete

Manager &**operator=** (const *Manager*&) = delete

inline *XynovaErrorCode* **init** (const *ManagerConfig* &config)

Initialize communication manager.

Converts C++ configuration objects into C-compatible structures and initializes communication masters and devices.

Note

Temporary storage is maintained during initialization to guarantee C API pointer validity.

Parameters

config – C++ manager configuration.

Returns

XynovaErrorCode.

inline *XynovaErrorCode* **run** (const std::string &master_name)

Start communication loop.

Parameters

master_name – Master name.

Returns

XynovaErrorCode.

inline *XynovaErrorCode* **stop** (const std::string &master_name)

Stop communication loop.

Parameters

master_name – Master name.

Returns

XynovaErrorCode.

template<typename **T**>

inline std::shared_ptr<**T**> **getDevice** (const std::string &master_name, const std::string &type, const std::string &device_name)

Get device instance.

Wraps the C API device handle into a C++ RAII object.

Template Parameters

T – Device wrapper type.

Parameters

- **master_name** – Master name.
- **type** – Device type.
- **device_name** – Device name.

Returns

Shared pointer to device instance. Returns nullptr when device is unavailable.

inline *XynovaErrorCode* **upgradeFirmware** (const std::string &master_name, uint32_t slave_id, const std::string &file)

Upgrade slave firmware.

Parameters

- **master_name** – Master name.

- **slave_id** – Slave identifier.
- **file** – Firmware file path.

Returns*XynovaErrorCode.***Public Static Functions**static inline *Manager* &**instance** ()

Get singleton instance.

Returns*Manager* instance.**Private Functions****Manager** () = default**~Manager** () = default

C++ wrapper definitions for Xynova communication system configuration.

This header provides modern C++ representations of Xynova communication configuration structures, bridging the low-level C API (*xynova/c/comm_types.h*) with STL-based and type-safe C++ structures.

```
namespace xynova
```

```
    namespace communication
```

Enumsenum class **CommType** : uint32_t

Supported communication transport types.

*Values:*enumerator **Can**

Controller Area Network

struct **CommConfig***#include <types.h>* Communication interface configuration.

Provides a common configuration layer for different communication backends.

Public Members*CommType* **type**

Communication type

std::string **sub_type**

Communication driver subtype, e.g. “SockCAN”

can::CanConfig **can**
CAN specific configuration

std::string **logger_name**
Logger name

struct **MasterConfig**

#include <types.h> Configuration of a communication master.

A master represents a communication controller managing one or more devices.

Public Members

std::string **name**
Master name

CommConfig **comm**
Communication configuration

uint64_t **period_us**
Control loop period in microseconds

int32_t **cpu_index**
CPU affinity index, -1 disables binding

struct **ResponseTimeSetting**

#include <types.h> Communication response timeout and retry configuration.

This structure corresponds to *XynovaCommunicationResponseTimeSetting*.

Public Members

uint32_t **user_max_timeout_us**
Maximum timeout for user API calls

uint32_t **user_poll_interval_us**
User polling interval

uint32_t **execute_timeout_us**
Backend request timeout

uint32_t **execute_max_retry**
Backend retry count

uint32_t **execute_poll_interval_us**
Backend polling interval

struct **SlaveConfig**

#include <types.h> Configuration of a communication slave device.

Public Members

std::string **type**

Slave type or model name

uint32_t **id**

Slave identifier

ResponseTimeSetting **general_time_setting**

General response timing

ResponseTimeSetting **fast_time_setting**

Fast response timing

std::string **logger_name**

Logger name

struct **SchedulingCommand**

#include <types.h> Periodic command scheduling configuration.

Defines commands that are periodically sent to or reported from devices.

Public Members

std::string **command**

Command name

ResponseTimeSetting **response_time_setting**

Response timing configuration

struct **DeviceConfig**

#include <types.h> Logical device configuration.

A device may contain multiple communication slaves and scheduled commands.

Public Members

std::string **type**

Device type

std::string **name**

Device name

std::string **master_name**
Associated master name

std::vector<*SlaveConfig*> **slaves**
Slave configuration list

std::vector<*SchedulingCommand*> **commands**
Scheduled command list

struct **ManagerConfig**

#include <types.h> Complete communication manager configuration.

Contains all communication masters and logical devices required to initialize the communication subsystem.

 **See also**

xynova::communication::Manager

Public Members

std::vector<*MasterConfig*> **masters**
Master configuration list

std::vector<*DeviceConfig*> **devices**
Device configuration list

4.2.2.2.3 CAN Communication (C++ API)

The **CAN module** provides types and utilities for CAN-based communication within the Xynova SDK. It is mainly used by the *Manager* and device interfaces for sending and receiving CAN frames.

This module defines:

- CAN message structures
- CAN error codes
- CAN device helpers

Header files

```
namespace xynova
```

```
    namespace communication
```

```
        namespace can
```

Enums

enum class **FrameType** : uint32_t

CAN frame identifier type.

Values:

enumerator **Standard**

Standard 11-bit CAN identifier

enumerator **Extended**

Extended 29-bit CAN identifier

enum class **CanMask** : uint32_t

CAN identifier mask.

Values:

enumerator **Standard**

Standard 11-bit identifier mask

enumerator **Extended**

Extended 29-bit identifier mask

struct **CanConfig**

#include <types.h> CAN interface configuration.

Supports both classical CAN and CAN-FD configuration.

Public Members

std::string **iface**

CAN interface name, e.g. "can0"

bool **manager_can** = false

Whether SDK manages CAN interface setup

uint32_t **bitrate** = 0

Arbitration bitrate in bits per second

uint32_t **dbitrate** = 0

CAN-FD data bitrate in bits per second

float **sample_point** = 0.75f

Arbitration sample point ratio

```
float dsample_point = 0.75f
    CAN-FD data sample point ratio

struct RecvFilter
    #include <types.h> CAN receive filter.
    Filters CAN frames using identifier and mask:
    (received_id & mask) == (id & mask)
```

Public Members

FrameType **type**

CAN frame type

uint32_t **id**

CAN identifier

uint32_t **mask**

CAN identifier mask

4.2.2.2.4 Hand Control (C++ API)

Overview

The **Hand Control module (C++ API)** provides modern, object-oriented wrappers for controlling Xynova dexterous hand devices.

This layer is built on top of the stable **C ABI**, and offers:

- RAII device lifetime management
- Strongly typed state structures
- Safer interfaces (references instead of raw pointers)
- Cleaner synchronous / asynchronous APIs
- Better integration with modern C++ projects

Compared with the C API, this layer:

- reduces boilerplate code
- improves readability
- prevents common misuse
- keeps zero runtime overhead

Use this module when developing C++ robotics applications.

Relationship to C API

Internally, all C++ interfaces delegate to the corresponding C functions:

Example:

```
XynovaErrorCode Device::getJointAngle(
    AngleStates& joints,
    uint64_t& ts)
{
    return XynovaDexterousHand_getJointAngle(...);
}
```

Therefore:

- C API → ABI stable / cross-language
- C++ API → ergonomic wrapper

Choose according to your project needs.

4.2.2.2.5 Dexterous Hand (C++ API)

Overview

The **Dexterous Hand C++ module** provides the currently available C++ wrappers for controlling Xynova dexterous hand devices.

The implementation currently exposes the following device-level operations:

- device lifecycle: `run()` / `stop()` / `valid()`
- product information: `getProductModel()` / `getProductSN()` / `getHardwareVersion()` / `getSoftwareVersion()` / `getHandType()`
- error state: `getErrorState()`
- control mode setup (all joints): `setJointControlMode()`
- synchronous angle command: `setJointAngle()`
- asynchronous angle command: `setJointAngleAsync()`
- joint feedback: `getJointAngle()`
- tactile feedback: `enableTactileWrench()` / `getTactileWrench()` / `enableTactileTaxel()` / `getTactileTaxel()`
- range feedback: `enableRange()` / `getRange()`

Device

High-level device wrapper class.

Provides:

- async joint control
- feedback queries
- product information
- error state monitoring

- tactile feedback
- range feedback
- device lifecycle management

C++ wrapper for Xynova Dexterous Hand device.

This header provides a high-level C++ interface for controlling and monitoring Xynova Dexterous Hand devices. It wraps the C API and provides RAII-based resource management together with strongly typed interfaces.

```
namespace xynova
```

```
    namespace hand
```

```
        namespace dexterous_hand
```

```
            class Device
```

```
                #include <Device.hpp> RAII wrapper for Xynova Dexterous Hand device.
```

This class provides a type-safe C++ interface over the C API. The underlying device handle is automatically released when the object is destroyed.

Public Types

```
using DeviceHandle = communication::DeviceHandle
```

Public Functions

```
inline explicit Device (DeviceHandle &&handle) noexcept
```

Construct a device wrapper.

Parameters

handle – *Device* handle.

```
Device (const Device&) = delete
```

```
Device &operator= (const Device&) = delete
```

```
Device (Device&&) noexcept = delete
```

```
Device &operator= (Device&&) noexcept = delete
```

```
inline bool valid () const noexcept
```

Check whether the device handle is valid.

Returns

true if valid, otherwise false.

```
inline XynovaErrorCode run ()
```

Start dexterous hand device.

Returns

XynovaErrorCode

```

inline XynovaErrorCode stop ()
    Stop dexterous hand device.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getProductModel (std::string &product_model)
    Get product model.
    Parameters
        product_model – Output product model string.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getProductSN (std::string &product_sn)
    Get product serial number.
    Parameters
        product_sn – Output serial number string.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getHardwareVersion (std::string &version)
    Get hardware version.
    Parameters
        version – Output hardware version.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getSoftwareVersion (std::string &version)
    Get software version.
    Parameters
        version – Output software version.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getHandType (HandType &type)
    Get hand type.
    Parameters
        type – Output hand type.
    Returns
        XynovaErrorCode

inline XynovaErrorCode getErrorState (ErrorState &state)
    Get current device error state.

    Retrieves the latest global warning, global fault and joint error information from the dexterous hand.
    Parameters
        state – Output error state.
    Returns
        XynovaErrorCode

inline XynovaErrorCode setJointControlMode (ControlMode mode)
    Set control mode for all joints.
    Parameters
        mode – Joint control mode.

```

Returns*XynovaErrorCode*inline *XynovaErrorCode* **enableTactileWrench** (bool enable)

Enable or disable tactile wrench reporting.

Controls whether tactile wrench feedback is reported by the device.

Parameters**enable** –

- true: Enable tactile wrench reporting.
- false: Disable tactile wrench reporting.

Returns*XynovaErrorCode*inline *XynovaErrorCode* **enableTactileTaxel** (bool enable)

Enable or disable tactile taxel reporting.

Controls whether tactile taxel feedback is reported by the device.

Parameters**enable** –

- true: Enable tactile taxel reporting.
- false: Disable tactile taxel reporting.

Returns*XynovaErrorCode*inline *XynovaErrorCode* **enableRange** (bool enable)

Enable or disable range reporting.

Controls whether range feedback is reported by the device.

Parameters**enable** –

- true: Enable range reporting.
- false: Disable range reporting.

Returns*XynovaErrorCode*inline *XynovaErrorCode* **setJointAngle** (const *AngleControl* &angle)

Set joint angle command.

Parameters**angle** – Joint angle command.**Returns***XynovaErrorCode*inline *XynovaErrorCode* **setJointAngleAsync** (const *AngleControl* &angle)

Asynchronously set joint angle command.

The command is queued and returns immediately.

Parameters**angle** – Joint angle command.**Returns***XynovaErrorCode*inline *XynovaErrorCode* **getJointAngle** (*AngleStates* &states, uint64_t ×tamp_us)

Get current joint angle states.

Parameters

- **states** – Output joint angle states.
- **timestamp_us** – Output timestamp in microseconds.

Returns*XynovaErrorCode*

```
inline XynovaErrorCode getTactileWrench (TactileWrenchStates &states, uint64_t
                                     &timestamp_us)
```

Get tactile wrench states.

Parameters

- **states** – Output tactile wrench states.
- **timestamp_us** – Output timestamp in microseconds.

Returns*XynovaErrorCode*

```
inline XynovaErrorCode getTactileTaxel (TactileTaxelStates &states, uint64_t
                                       &timestamp_us)
```

Get tactile taxel states.

Parameters

- **states** – Output tactile taxel states.
- **timestamp_us** – Output timestamp in microseconds.

Returns*XynovaErrorCode*

```
inline XynovaErrorCode getRange (RangeStates &states, uint64_t &timestamp_us)
```

Get range measurement states.

Retrieves two-dimensional range measurement data from the dexterous hand.

The caller provides the range point buffer. The device fills:

- width
- height
- range point data

Points are stored in row-major order:

```
index = y * width + x
```

Parameters

- **states** – Output range measurement states.
- **timestamp_us** – Output timestamp in microseconds.

Returns*XynovaErrorCode***Private Functions**

```
inline XynovaDeviceHandle *c_handle () const noexcept
```

Get underlying C API handle.

Private Members

```
DeviceHandle i_dev_handle
```

Managed device handle

C++ type definitions for the Xynova Dexterous Hand.

This file provides strongly typed C++ data types corresponding to Xynova Dexterous Hand C API structures, including joint identifiers, control modes, joint commands, joint states, and tactile sensing data.

 See also

xynova/c/dexterous_hand_types.h

Version

1.0.0

Date

2026-07-17

Author

Xynova

namespace **xynova**

namespace **hand**

namespace **dexterous_hand**

Typedefs

using **JointID** = *XynovaDexterousHandJointID*

Joint identifier type.

using **TactileID** = *XynovaDexterousHandTactileID*

Tactile region identifier type.

using **ErrorCode** = *XynovaDexterousHandErrorCode*

Joint error code type.

using **WarningFlags** = *XynovaDexterousHandWarningFlags*

Global warning flags type.

using **FaultFlags** = *XynovaDexterousHandFaultFlags*

Global fault flags type.

template<typename **T**>

using **JointValueArray** = std::array<*infra::tools::IdValue<T>*, *JOINT_NUM*>

Fixed-size joint value array.

Each element contains:

- Joint identifier.
- Strongly typed joint value.

Template Parameters

T – Joint value type.

```
using HandType = XynovaDexterousHandType
    Hand type.

using ControlMode = XynovaDexterousHandControlMode
    Joint control mode type.

using RangePointStatus = XynovaDexterousHandRangePointStatus
    Range point measurement status type.

using WrenchDimension = XynovaDexterousHandWrenchDimension
    Tactile wrench dimension type.

using ControlModeCommand = JointValueArray<ControlMode>
    Joint control mode command.

    Each element stores:
    • Joint identifier.
    • Joint control mode.

using AngleControl = JointValueArray<float>
    Joint angle command.

    Each element stores:
    • Joint identifier.
    • Target joint angle.
    Unit: rad.

using AngleStates = JointValueArray<float>
    Joint angle feedback state.

    Each element stores:
    • Joint identifier.
    • Measured joint angle.
    Unit: rad.

using TactileWrenchStates = std::array<TactileWrench, TACTILE_NUM>
    Tactile resultant wrench states.

    Each element represents one tactile region.

using TactileTaxelStates = std::vector<TactileTaxel>
    Dynamic tactile taxel state array.

    The container owns tactile taxel data.

using RangePoint = XynovaDexterousHandRangePoint
    Range point measurement.

    Represents distance measurement from a single range sensing point.
```

Enums

enum class **JointError** : uint16_t

Joint error flags.

Values:

enumerator **None**

enumerator **StallProtection**

enumerator **OverTemperature**

enumerator **OverCurrent**

enumerator **MotorAbnormal**

enumerator **Communication**

enumerator **EncoderAbnormal**

Variables

static constexpr size_t **JOINT_NUM** = XYNOVA_DEXTEROUS_HAND_JOINT_NUM

Number of joints supported by the dexterous hand.

static constexpr size_t **TACTILE_NUM** = XYNOVA_DEXTEROUS_HAND_TACTILE_NUM

Number of tactile regions supported by the dexterous hand.

static constexpr size_t **TACTILE_TOTAL_TAXEL_NUM** =
XYNOVA_DEXTEROUS_HAND_TACTILE_TOTAL_TAXEL_NUM

Number of tactile taxels supported by the dexterous hand.

static constexpr size_t **RANGE_POINT_NUM** =
XYNOVA_DEXTEROUS_HAND_RANGE_POINT_NUM

Number of range points supported by the default range sensor.

class **JointErrorCode**

#include <types.h> Joint error code wrapper.

Provides type-safe access to joint error flags.

Public Functions

inline constexpr **JointErrorCode** (*ErrorCode* code = 0) noexcept

inline void **fromC** (const *XynovaTypedValue* &value)

```

inline XynovaTypedValue toC () const

inline constexpr bool has (JointError error) const noexcept

inline constexpr bool ok () const noexcept

inline constexpr ErrorCode value () const noexcept

```

Private Members

```
ErrorCode i_code = {0}
```

struct **JointErrorState**

#include <types.h> Joint error state.

Contains joint identifier and corresponding error code.

Public Members

```
JointID id = {0}
```

Joint identifier

```
JointErrorCode error_code = { }
```

Joint error code

struct **ErrorState**

#include <types.h> Dexterous hand error state.

Contains global warning, global fault and joint error information.

Public Functions

```
inline bool hasFault () const noexcept
```

Check whether any global fault exists.

```
inline bool hasWarning () const noexcept
```

Check whether any global warning exists.

```
inline JointErrorState &joint (JointID id) noexcept
```

Get joint error by joint identifier.

```
inline const JointErrorState &joint (JointID id) const noexcept
```

Get joint error by joint identifier.

Public Members

```
uint64_t timestamp_us = {0}
```

Error timestamp (us): system monotonic clock

WarningFlags **warning_flags** = {0}

Global warning flags

FaultFlags **fault_flags** = {0}

Global fault flags

`std::array<JointErrorState, JOINT_NUM>` **joint_error_codes** = {}

Joint error states

struct **TactileWrench**

#include <types.h> Tactile resultant wrench data.

Represents force/torque measurement of a tactile region.

Public Members

`uint64_t` **timestamp_us** = {0}

Sampling timestamp (us): system monotonic clock

TactileID **region** = {0}

Tactile region identifier

WrenchDimension **dimension** = {0}

Wrench dimension

infra::tools::Value<float, 6> **wrench**

Force/torque values: force unit: N, torque unit: Nm

struct **TactileTaxel**

#include <types.h> Tactile taxel measurement.

Represents force/torque data from a single tactile sensing element.

Public Members

`uint64_t` **timestamp_us** = {0}

Sampling timestamp (us): system monotonic clock

TactileID **region** = {0}

Tactile region identifier

`uint16_t` **taxel_id** = {0}

Taxel identifier

WrenchDimension **dimension** = {0}

Wrench dimension

```
uint8_t reserved = {0}
```

Reserved

```
infra::tools::Value<float, 6> wrench
```

Force/torque values: force unit: N, torque unit: Nm

```
struct RangeStates
```

```
#include <types.h> Dynamic range point states.
```

Represents a two-dimensional range map stored in row-major order.

The point corresponding to coordinate (x, y) is stored at: index = y * width + x

Public Functions

```
inline explicit RangeStates (size_t capacity = 0)
```

Public Members

```
uint16_t width = {0}
```

Range map width

```
uint16_t height = {0}
```

Range map height

```
std::vector<RangePoint> points
```

Range points

See Also

- [Communication \(C++ API\)](#)
- [Infrastructure \(C++ API\)](#)
- [Examples](#)

4.2.2.2.6 Infrastructure (C++ API)

Overview

The **Infrastructure module** provides common runtime utilities used by all Xynova SDK components.

It contains reusable, non-device-specific facilities that improve:

- reliability
- observability
- performance
- development convenience

This layer is independent of communication or hand devices and can be used standalone in general C++ projects.

Public headers

The externally exposed C++ headers for this module are:

- `xynova/cpp/infra/tools/value_types.hpp` —strongly typed value wrappers, documented on the [value types](#) page

The logging entry points are documented on the [logging](#) page:

- `xynova/cpp/infra/log/LogManager.hpp` —singleton logging manager
- `xynova/cpp/infra/log/types.h` —logging configuration types

See Also

- [Communication \(C++ API\)](#)
- [Hand Control \(C++ API\)](#)
- [Examples](#)

4.2.2.2.7 Logging (C++ API)

Overview

The **Logging module** provides a modern, high-performance, asynchronous logging framework for the Xynova SDK.

It is designed for:

- real-time robotics systems
- multi-threaded environments
- high-throughput message logging
- low latency and minimal blocking

This module wraps the low-level C logging backend and offers a clean, RAII-friendly, object-oriented C++ interface.

Key Features

- Thread-pool based sinks
- File and console writers
- Automatic log rotation
- Configurable formatting
- Runtime level control
- Lock-free backend queue
- Zero-copy memory pool design

Public headers

- `xynova/cpp/infra/log/LogManager.hpp`
- `xynova/cpp/infra/log/types.h`

API Reference

```
namespace xynova
```

```
    namespace infra
```

```
        namespace log
```

```
            class LogManager
```

#include <LogManager.hpp> Singleton manager for Xynova logging subsystem.

Provides C++ wrappers around the C logging API while keeping the C ABI boundary stable.

Supported operations:

- Initialize logging objects.
- Create sinks.
- Create writers.
- Create formatters.
- Create logger backends.

Public Functions

```
inline XynovaErrorCode initDefaultObjects ()
```

Initialize default logging objects.

Returns

Operation result code.

```
inline bool isInitialized () const
```

Check whether object logger is initialized.

Returns

true if initialized.

```
inline XynovaErrorCode createSinkTP (const std::string &type, const std::string &name,  
                                     const SinkTPConfiguration &config)
```

Create thread-pool sink.

Parameters

- **type** – Registered sink type.
- **name** – Sink instance name.
- **config** – Sink configuration.

Returns

Operation result code.

```
inline XynovaErrorCode createFileWriter (const std::string &type, const std::string  
                                         &name, const FileWriterConfiguration  
                                         &config)
```

Create file writer.

Parameters

- **type** – Registered writer type.
- **name** – Writer instance name.
- **config** – Writer configuration.

Returns

Operation result code.

```
inline XynovaErrorCode createConsoleWriter (const std::string &type, const
                                             std::string &name, const
                                             ConsoleWriterConfiguration &config)
```

Create console writer.

Parameters

- **type** – Registered writer type.
- **name** – Writer instance name.
- **config** – Writer configuration.

Returns

Operation result code.

```
inline XynovaErrorCode createFormatter (const std::string &type, const std::string
                                             &name, const FormatterConfiguration
                                             &config)
```

Create formatter.

Parameters

- **type** – Registered formatter type.
- **name** – Formatter instance name.
- **config** – Formatter configuration.

Returns

Operation result code.

```
inline XynovaErrorCode createLogger (const std::string &name, const
                                         LoggerConfiguration &conf)
```

Create logger backend.

Parameters

- **name** – Logger instance name.
- **conf** – Logger configuration.

Returns

Operation result code.

```
inline bool hasSink (const std::string &name) const
```

Check sink existence.

Parameters

name – Sink instance name.

Returns

true if exists.

```
inline bool hasWriter (const std::string &name) const
```

Check writer existence.

Parameters

name – Writer instance name.

Returns

true if exists.

```
inline bool hasFormatter (const std::string &name) const
```

Check formatter existence.

Parameters

name – Formatter instance name.

Returns

true if exists.

```
inline bool hasLogger (const std::string &name) const
```

Check logger existence.

Parameters

name – Logger instance name.

Returns

true if exists.

```
inline XynovaErrorCode setLoggerLevel (const std::string &name, XynovaLogLevel
                                     level)
```

Set logger level.

Parameters

- **name** – Logger instance name.
- **level** – New log level.

Returns

Operation result code.

Public Static Functions

```
static inline LogManager &instance ()
```

Get singleton instance.

Returns

Singleton *LogManager* reference.

Private Functions

```
LogManager () = default
```

```
~LogManager () = default
```

```
LogManager (const LogManager&) = delete
```

```
LogManager (LogManager&&) = delete
```

```
LogManager &operator= (const LogManager&) = delete
```

```
LogManager &operator= (LogManager&&) = delete
```

```
namespace xynova
```

```
    namespace infra
```

```
        namespace log
```

```
            struct SinkConfiguration
```

```
                #include <types.h> Logging sink base configuration.
```

Defines timeout parameters and worker loop intervals used by sink implementations.

Public Members

```
            uint64_t block_obtain_timeout_us = 0
```

Timeout for obtaining log blocks.

```
uint64_t writing_task_push_timeout_us = 500
```

Timeout for pushing writing tasks.

```
uint64_t writing_task_wait_timeout_us = 500
```

Timeout for waiting writing task completion.

```
uint64_t min_looping_task_idle_interval_us = 1 * 1000
```

Minimum worker loop idle interval in microseconds.

```
uint64_t max_looping_task_idle_interval_us = 5 * 1000
```

Maximum worker loop idle interval in microseconds.

```
uint32_t max_continuous_push_retry_time_ms = 16 * 1000
```

Maximum continuous retry time when pushing logs fails.

```
struct SinkTPConfiguration
```

#include <types.h> Thread-pool based sink configuration.

Extends *SinkConfiguration* with worker thread and queue parameters.

Public Members

```
SinkConfiguration base_conf
```

Common sink configuration.

```
uint32_t num_threads = std::thread::hardware_concurrency()
```

Number of worker threads.

```
uint32_t max_queue_size = 0
```

Maximum queue size.

A value of zero means unlimited.

```
struct WriterConfiguration
```

#include <types.h> Base writer configuration.

Shared configuration fields for all log writers.

Public Members

```
std::string formatter
```

Formatter instance name.

```
bool auto_flush = true
```

Enable automatic flushing.

```
uint32_t flush_interval_ms = 1000
```

Flush interval in milliseconds.

```
struct FileWriterConfiguration
```

#include <types.h> File writer configuration.

Configures output to a file with optional log rotation.

Public Members

WriterConfiguration **base_conf**

Base writer configuration.

std::string **file_path**

Output file path.

```
uint64_t max_file_size = 10 * 1024 * 1024
```

Maximum file size before rotation.

```
uint32_t max_roll_files = 5
```

Maximum number of rotated files.

```
struct ConsoleWriterConfiguration
```

#include <types.h> Console writer configuration.

Configures output to standard output or standard error.

Public Members

WriterConfiguration **base_conf**

Base writer configuration.

```
bool use_stderr = false
```

Output to stderr when true, otherwise stdout.

```
struct FormaterConfiguration
```

#include <types.h> Formatter configuration.

Reserved for future formatter extensions.

Public Members

```
uint32_t reserved = 0
```

Reserved field.

struct **LoggerBackendConfiguration**

#include <types.h> Logger backend configuration.

Defines memory pool parameters, buffering behavior and performance tuning options for logger backend instances.

Public Members

uint32_t **pool_size**

Number of blocks in the memory pool.

uint32_t **block_size**

Number of records per block.

uint32_t **message_size**

Maximum message size.

uint32_t **max_continuous_repeated_log** = std::numeric_limits<uint32_t>::max()

Maximum continuous repeated log count.

uint32_t **flush_interval_us** = 500 * 1000

Logger flush interval in microseconds.

float **queue_size_multiple** = 1.5

Queue size multiplier.

struct **LoggerConfiguration**

#include <types.h> Logger instance creation configuration.

Contains sink, writer and backend configuration information required to create a logger instance.

Public Members

std::string **sink_name**

Sink instance name.

std::vector<std::string> **writer_name_list**

Writer instance names.

std::string **lb_type**

Logger backend implementation type.

XynovaLogLevel **level**

Minimum log level.

LoggerBackendConfiguration **lb_config**

Logger backend configuration.

4.2.2.2.8 Value Types (C++ API)

Overview

The **Value Types module** provides strongly typed C++ wrappers for the Xynova SDK value containers. It is part of the infrastructure layer and is used by the dexterous hand and communication modules to exchange typed data (e.g. joint angles, force components) in a type-safe way.

Key features:

- `infra::tools::Value<T, N>` —fixed-size strongly typed value array
- `infra::tools::IdValue<T>` —typed value paired with an identifier
- `DataTypeTraits` —compile-time mapping between C++ types and `XynovaDataType`

These wrappers bridge the ABI-safe C value unions (`XynovaTypedValue`, `XynovaIdValue`) with modern C++ interfaces.

Header

- `xynova/cpp/infra/tools/value_types.hpp`

API Reference

C++ strongly typed value wrappers for Xynova SDK.

This file provides C++ wrappers for C API value types defined in `xynova/c/value_types.h`.

Features:

- Automatic C++ type to `XynovaDataType` mapping.
- Strongly typed scalar and array value containers.
- C API conversion support.
- Identifier value wrappers.

Example:

```
using namespace xynova::infra::tools;

Value<float> angle(30.0f);
Value<float, 3> position({1.0f, 2.0f, 3.0f});

auto c_value = angle.toC();
```

Version

1.0.0

Date

2026-07-17

Defines

XYNOVA_DEFINE_DATA_TYPE_TRAIT (cpp_type, c_type)

Define a C++ type to XynovaDataType mapping.

Parameters

- **cpp_type** – C++ type.
- **c_type** – Xynova data type identifier.

namespace **xynova**

namespace **infra**

namespace **tools**

Functions

template<typename **T**>

inline void **assignValue** (*XynovaValue* &dst, const *T* &src)

Store C++ value into C API union.

Template Parameters

T – *Value* type.

Parameters

- **dst** – Destination C union.
- **src** – Source C++ value.

template<typename **T**>

inline *T* **readValue** (const *XynovaValue* &src)

Read C API union into C++ value.

Template Parameters

T – *Value* type.

Parameters

src – Source C union.

Returns

Converted C++ value.

template<typename **T**>

struct **DataTypeTraits**

#include <value_types.hpp> Mapping between C++ type and Xynova data type.

Specializations are generated for supported SDK value types.

Template Parameters

T – C++ type.

Public Static Attributes

static constexpr uint8_t **value** = *XYNOVA_DATA_TYPE_NONE*

template<typename **T**, size_t **N** = 1>

class **Value**

#include <value_types.hpp> Strongly typed value wrapper.

This class provides a C++ abstraction over:

- *XynovaTypedValue*
- *XynovaTypedValueArray2*
- *XynovaTypedValueArray3*
- *XynovaTypedValueArray4*

Template Parameters

- **T** – Element type.
- **N** – Number of elements. Range: 1~4.

Public Types

using **Storage** = std::conditional_t<*N* == 1, *T*, std::array<*T*, *N*>>

Public Functions

Value () = default

Default constructor.

inline explicit **Value** (const *Storage* &value)

Construct from value.

Parameters

value – Initial value.

inline void **setValue** (const *Storage* &value)

Set value.

Parameters

value – New value.

inline const *Storage* &**value** () const noexcept

Get constant value.

Returns

Constant reference.

inline *Storage* &**value** () noexcept

Get mutable value.

Returns

Mutable reference.

template<typename **CValue**>

inline bool **fromC** (const *CValue* &value)

Convert from C API value.

The input type must match the template element count.

Template Parameters

CValue – C API value type.

Parameters

value – Source C API value.

Returns

true if conversion succeeds, false otherwise.

```
inline auto toC () const
    Convert to C API value.
Returns
    C API value object.
```

Private Members

```
Storage i_value = { }
```

```
template<typename T, size_t N = 1>
```

```
class IdValue
```

```
#include <value_types.hpp> Strongly typed identifier value wrapper.
```

Wraps an identifier together with a typed value.

Template Parameters

- **T** – Element type.
- **N** – Number of elements. Range: 1~6.

Public Types

```
using ValueType = Value<T, N>
```

```
using Storage = typename ValueType::Storage
```

Public Functions

```
IdValue () = default
```

Default constructor.

```
inline IdValue (uint16_t id, const Storage &value)
```

Construct identifier value.

Parameters

- **id** – Identifier.
- **value** – Initial value.

```
inline void setId (uint16_t id) noexcept
```

Set identifier.

Parameters

id – Identifier value.

```
inline void setValue (const Storage &value)
```

Set contained value.

Parameters

value – New value.

```
inline uint16_t id () const noexcept
```

Get identifier.

Returns

Identifier value.

```
inline const ValueType &value () const noexcept
```

Get constant value wrapper.

Returns

Constant reference.

```
inline ValueType &value () noexcept
```

Get mutable value wrapper.

Returns

Mutable reference.

```
template<typename CValue>
inline bool fromC (const CValue &value)
```

Convert from C API identifier value.

Template Parameters

CValue – C API identifier value type.

Parameters

value – Source C API value.

Returns

true if conversion succeeds, false otherwise.

```
inline auto toC () const
```

Convert to C API identifier value.

Returns

C API identifier value object.

Private Members

```
uint16_t i_id = {0}
```

```
ValueType i_value
```

See Also

- *Infrastructure (C++ API)*
- *Logging (C++ API)*
- *Infrastructure (C API)*
- *Examples*

4.2.3 Notes

- The C++ API classes are mostly thin wrappers over the corresponding C API structures and functions.
- Memory management in the C++ API is handled through RAII where applicable, reducing common misuse patterns.
- Both API layers share the same module naming and device conventions, which makes switching between them straightforward.

4.2.4 Usage

- Include the appropriate headers from the corresponding *xynova/c/...* or *xynova/cpp/...* tree depending on the layer you use.
- Initialize communication managers or logging backends before using device or logging interfaces.
- Consult the examples section for complete usage patterns and integration tips.

4.3 Glossary

taxel

A single sensitive element of the tactile sensor array. Each taxel measures a force component (1D taxels) or three orthogonal force components (3D taxels). See *Tactile Taxel Sensing*.

tactile region

A group of taxels covering one module of the sensor array, such as a fingertip, a phalanx, or the palm. Each region has a fixed taxel count and is identified by a region ID. See *Tactile Sensing*.

wrench

A six-component force/torque vector $[F_x, F_y, F_z, M_x, M_y, M_z]$. The wrench of a tactile region reports the resultant force and moment of that region, with force in Newtons (N) and moment in Newton-meters (Nm).

joint

A degree of freedom of the dexterous hand. The hand exposes a fixed set of joints (JOINT_NUM), each identified by a joint ID covering the fingers, thumb, and wrist.

control mode

The operating mode of the joints, configured through `setJointControlMode()` and applied to all joints. The SDK currently implements `DISABLE` (control disabled) and `ANGLE` (angle control); other modes such as position, velocity, or force control are reserved for future releases.

master

A communication node that drives the control loop and exchanges data with the slave devices connected to the bus. A master is configured with a communication type and a CAN interface.

slave

A physical device on the bus that responds to the master, such as a dexterous hand. A slave is identified by its slave ID and exposes the device functionality to the master.

device

A logical high-level wrapper that binds one master and one or more slaves into a single controllable unit, e.g. a dexterous hand device.

CAN / CAN-FD

Controller Area Network. CAN-FD extends classical CAN with higher data bitrates and larger payloads. The SDK supports CAN and CAN-FD through the `CommType::Can` transport with configurable bitrates and sample points.

sink

A logging component that groups one or more writers and manages the delivery of log records to them.

writer

A logging output destination, such as a file or the console, with its own formatting, flushing, and rotation settings.

logger

A named logging instance used by application or SDK modules to emit log records. A logger is bound to a sink and one or more writers.

SDK version

The version information of the SDK build, including version number, git branch, git commit, and build time, queryable through the functions in `xynova/c/version.h`.

range point

A single distance measurement of the proximity (DToF) sensor. Each range point reports a measurement status (valid, no target, weak signal, or invalid) and a distance in meters. See *Range Sensing*.

range map

A two-dimensional set of range points returned by `getRange()`, stored in row-major order (`index = y * width + x`). The SDK currently reports a 4×4 map of 16 points.

error code

The standardized `XynovaErrorCode` value returned by the public API functions, e.g. `XYNOVA_ERROR_CODE_OK` or `XYNOVA_ERROR_CODE_DATA_OUT_OF_DATE`. See [Error Codes](#) for the full reference.

ADDITIONAL RESOURCES

- **Generated reference:** the Doxygen XML and Sphinx HTML outputs are written under the build tree for local reference.
- **Best practices:** verify device validity before use, prefer asynchronous control for high-rate loops, and initialize logging before runtime setup.

Symbols

[anonymous] (*C++ enum*), 49, 55, 68, 69, 71, 73–75, 80, 82, 84, 95

[anonymous]::XYNOVA_COMMUNICATION_CAN_FRAME_TYPE_EXTENDED (*C++ enumerator*), 55

[anonymous]::XYNOVA_COMMUNICATION_CAN_FRAME_TYPE_STANDARD (*C++ enumerator*), 55

[anonymous]::XYNOVA_COMMUNICATION_CAN_MASK_EXTENDED (*C++ enumerator*), 55

[anonymous]::XYNOVA_COMMUNICATION_CAN_MASK_STANDARD (*C++ enumerator*), 55

[anonymous]::XYNOVA_COMMUNICATION_COMM_TYPE_CAN (*C++ enumerator*), 49

[anonymous]::XYNOVA_DATA_TYPE_BOOL (*C++ enumerator*), 85

[anonymous]::XYNOVA_DATA_TYPE_DOUBLE (*C++ enumerator*), 85

[anonymous]::XYNOVA_DATA_TYPE_FLOAT (*C++ enumerator*), 85

[anonymous]::XYNOVA_DATA_TYPE_NONE (*C++ enumerator*), 84

[anonymous]::XYNOVA_DATA_TYPE_S16 (*C++ enumerator*), 85

[anonymous]::XYNOVA_DATA_TYPE_S32 (*C++ enumerator*), 85

[anonymous]::XYNOVA_DATA_TYPE_S64 (*C++ enumerator*), 85

[anonymous]::XYNOVA_DATA_TYPE_S8 (*C++ enumerator*), 85

[anonymous]::XYNOVA_DATA_TYPE_U16 (*C++ enumerator*), 85

[anonymous]::XYNOVA_DATA_TYPE_U32 (*C++ enumerator*), 85

[anonymous]::XYNOVA_DATA_TYPE_U64 (*C++ enumerator*), 85

[anonymous]::XYNOVA_DATA_TYPE_U8 (*C++ enumerator*), 85

[anonymous]::XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_ANGLE (*C++ enumerator*), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_CONTROL_MODE_DISABLE (*C++ enumerator*), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_ERROR_COMMUNICATION (*C++ enumerator*), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_ERROR_ENCODER_ABNORMAL (*C++ enumerator*), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_ERROR_MOTOR_ABNORMAL (*C++ enumerator*), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_ERROR_NONE (*C++ enumerator*), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_ERROR_OVER_CURRENT (*C++ enumerator*), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_ERROR_OVER_TEMPERATURE (*C++ enumerator*), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_ERROR_STALL_PROTECTION (*C++ enumerator*), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_ACTUATOR_COMMUNICATION (*C++ enumerator*), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_ACTUATOR_OVER_CURRENT (*C++ enumerator*), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_ACTUATOR_OVER_TEMPERATURE (*C++ enumerator*), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_ACTUATOR_OVER_VOLTAGE (*C++ enumerator*), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_ACTUATOR_UNDER_VOLTAGE (*C++ enumerator*), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_ANGLE_SENSOR_COMMUNICATION (*C++ enumerator*), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_CANFD1_STATUS (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_CANFD2_STATUS (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_CONTROL_LOOP_TIMEOUT (*C++ enumerator*), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_24V_OVER_VOLTAGE (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_24V_UNDER_VOLTAGE (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_48V_OVER_VOLTAGE (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_48V_UNDER_VOLTAGE (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_OVER_CURRENT1 (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_OVER_CURRENT2 (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_OVER_CURRENT3 (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_OVER_TEMPERATURE (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_MAINBOARD_UNDER_TEMPERATURE (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_NONE (*C++ enumerator*), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_PARAMETER_STORAGE (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_PDM_FRAME_LOSS (*C++ enumerator*), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_RANGE_SENSOR_COMMUNICATION (*C++ enumerator*), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_SUBBOARD_COMMUNICATION (C++ enumerator), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_SUBBOARD_OVER_TEMPERATURE (C++ enumerator), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_SUBBOARD_UNDER_TEMPERATURE (C++ enumerator), 70

[anonymous]::XYNOVA_DEXTEROUS_HAND_FAULT_TACTILE_SENSOR_COMMUNICATION (C++ enumerator), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF1 (C++ enumerator), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF2 (C++ enumerator), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF3 (C++ enumerator), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_FF4 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF1 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF2 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF3 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF4 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_LF5 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF1 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF2 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF3 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_MF4 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF1 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF2 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF3 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_RF4 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH1 (C++ enumerator), 72

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH2 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH3 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_TH4 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_WR1 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_JOINT_ID_WR2 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_INVALID (C++ enumerator), 75

[anonymous]::XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_NO_TARGET (C++ enumerator), 75

[anonymous]::XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_VALID (C++ enumerator), 75

[anonymous]::XYNOVA_DEXTEROUS_HAND_RANGE_POINT_STATUS_WEAK_SIGNAL (C++ enumerator), 75

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT1 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT2 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_FT3 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_LT1 (C++ enumerator), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_LT2 (C++ enumerator), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_LT3 (C++ enumerator), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_MT1 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_MT2 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_MT3 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_PT1 (C++ enumerator), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_RT1 (C++ enumerator), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_RT2 (C++ enumerator), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_RT3 (C++ enumerator), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_TT1 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_TACTILE_ID_TT2 (C++ enumerator), 73

[anonymous]::XYNOVA_DEXTEROUS_HAND_TYPE_LEFT (C++ enumerator), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_TYPE_RIGHT (C++ enumerator), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_TYPE_UNKNOWN (C++ enumerator), 71

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_ANGLE_SENSOR_COMMUNICATION (C++ enumerator), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_CANFD1_STATUS (C++ enumerator), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_CANFD2_STATUS (C++ enumerator), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_CONTROL_LOOP_TIMEOUT (C++ enumerator), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_FIRMWARE_UPGRADE (C++ enumerator), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_MAINBOARD_OVER_TEMPERATURE (C++ enumerator), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_MAINBOARD_UNDER_TEMPERATURE (C++ enumerator), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_NONE (C++ enumerator), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_PDM_FRAME_LOSS (C++ enumerator), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_RANGE_SENSOR_COMMUNICATION (C++ enumerator), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_SUBBOARD_OVER_TEMPERATURE (C++ enumerator), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_SUBBOARD_UNDER_TEMPERATURE (C++ enumerator), 69

[anonymous]::XYNOVA_DEXTEROUS_HAND_WARNING_TACTILE_SENSOR_COMMUNICATION (C++ enumerator), 68

[anonymous]::XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_1D (C++ enumerator), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_3D (C++ enumerator), 74

[anonymous]::XYNOVA_DEXTEROUS_HAND_WRENCH_DIMENSION_6D (C++ enumerator), 74

[anonymous]::XYNOVA_ERROR_CODE_LIST (C++ enumerator), 82

[anonymous]::XYNOVA_FALSE (C++ enumerator), 80

[anonymous]::XYNOVA_LOG_LEVEL_DEBUG (C++ enumerator), 95

[anonymous]::XYNOVA_LOG_LEVEL_ERROR (C++ *enumerator*), 95
 [anonymous]::XYNOVA_LOG_LEVEL_FATAL (C++ *enumerator*), 95
 [anonymous]::XYNOVA_LOG_LEVEL_INFO (C++ *enumerator*), 95
 [anonymous]::XYNOVA_LOG_LEVEL_TRACE (C++ *enumerator*), 95
 [anonymous]::XYNOVA_LOG_LEVEL_WARN (C++ *enumerator*), 95
 [anonymous]::XYNOVA_TRUE (C++ *enumerator*), 80

C

CAN / CAN-FD, 135
 control mode, 135

D

device, 135

E

error code, 136

J

joint, 135

L

logger, 135

M

master, 135

R

range map, 135
 range point, 135

S

SDK version, 135
 sink, 135
 slave, 135

T

tactile region, 135
 taxel, 135

W

wrench, 135
 writer, 135

X

x (C *macro*), 81
 xynova (C++ *type*), 104, 106, 109, 113, 117, 124, 126, 131
 xynova::communication (C++ *type*), 104, 106, 109
 xynova::communication::can (C++ *type*), 109
 xynova::communication::can::CanConfig (C++ *struct*), 110
 xynova::communication::can::CanConfig::bitrate (C++ *member*), 110
 xynova::communication::can::CanConfig::dbitrate (C++ *member*), 110
 xynova::communication::can::CanConfig::dsample_point (C++ *member*), 110
 xynova::communication::can::CanConfig::iface (C++ *member*), 110
 xynova::communication::can::CanConfig::manager_can (C++ *member*), 110
 xynova::communication::can::CanConfig::sample_point (C++ *member*), 110
 xynova::communication::can::CanMask (C++ *enum*), 110
 xynova::communication::can::CanMask::Extended (C++ *enumerator*), 110
 xynova::communication::can::CanMask::Standard (C++ *enumerator*), 110
 xynova::communication::can::FrameType (C++ *enum*), 110
 xynova::communication::can::FrameType::Extended (C++ *enumerator*), 110
 xynova::communication::can::FrameType::Standard (C++ *enumerator*), 110

xynova::communication::can::RecvFilter (C++ struct), 111
 xynova::communication::can::RecvFilter::id (C++ member), 111
 xynova::communication::can::RecvFilter::mask (C++ member), 111
 xynova::communication::can::RecvFilter::type (C++ member), 111
 xynova::communication::CommConfig (C++ struct), 106
 xynova::communication::CommConfig::can (C++ member), 106
 xynova::communication::CommConfig::logger_name (C++ member), 107
 xynova::communication::CommConfig::sub_type (C++ member), 106
 xynova::communication::CommConfig::type (C++ member), 106
 xynova::communication::CommType (C++ enum), 106
 xynova::communication::CommType::Can (C++ enumerator), 106
 xynova::communication::DeviceConfig (C++ struct), 108
 xynova::communication::DeviceConfig::commands (C++ member), 109
 xynova::communication::DeviceConfig::master_name (C++ member), 108
 xynova::communication::DeviceConfig::name (C++ member), 108
 xynova::communication::DeviceConfig::slaves (C++ member), 109
 xynova::communication::DeviceConfig::type (C++ member), 108
 xynova::communication::DeviceHandle (C++ class), 104
 xynova::communication::DeviceHandle::~DeviceHandle (C++ function), 104
 xynova::communication::DeviceHandle::DeviceHandle (C++ function), 104
 xynova::communication::DeviceHandle::get (C++ function), 104
 xynova::communication::DeviceHandle::i_handle (C++ member), 104
 xynova::communication::DeviceHandle::operator bool (C++ function), 104
 xynova::communication::DeviceHandle::operator= (C++ function), 104
 xynova::communication::Manager (C++ class), 104
 xynova::communication::Manager::~Manager (C++ function), 106
 xynova::communication::Manager::getDevice (C++ function), 105
 xynova::communication::Manager::init (C++ function), 105
 xynova::communication::Manager::instance (C++ function), 106
 xynova::communication::Manager::Manager (C++ function), 105, 106
 xynova::communication::Manager::operator= (C++ function), 105
 xynova::communication::Manager::run (C++ function), 105
 xynova::communication::Manager::stop (C++ function), 105
 xynova::communication::Manager::upgradeFirmware (C++ function), 105
 xynova::communication::ManagerConfig (C++ struct), 109
 xynova::communication::ManagerConfig::devices (C++ member), 109
 xynova::communication::ManagerConfig::masters (C++ member), 109
 xynova::communication::MasterConfig (C++ struct), 107
 xynova::communication::MasterConfig::comm (C++ member), 107
 xynova::communication::MasterConfig::cpu_index (C++ member), 107
 xynova::communication::MasterConfig::name (C++ member), 107
 xynova::communication::MasterConfig::period_us (C++ member), 107
 xynova::communication::ResponseTimeSetting (C++ struct), 107
 xynova::communication::ResponseTimeSetting::execute_max_retry (C++ member), 107
 xynova::communication::ResponseTimeSetting::execute_poll_interval_us (C++ member), 107
 xynova::communication::ResponseTimeSetting::execute_timeout_us (C++ member), 107
 xynova::communication::ResponseTimeSetting::user_max_timeout_us (C++ member), 107
 xynova::communication::ResponseTimeSetting::user_poll_interval_us (C++ member), 107
 xynova::communication::SchedulingCommand (C++ struct), 108
 xynova::communication::SchedulingCommand::command (C++ member), 108
 xynova::communication::SchedulingCommand::response_time_setting (C++ member), 108
 xynova::communication::SlaveConfig (C++ struct), 107
 xynova::communication::SlaveConfig::fast_time_setting (C++ member), 108
 xynova::communication::SlaveConfig::general_time_setting (C++ member), 108
 xynova::communication::SlaveConfig::id (C++ member), 108
 xynova::communication::SlaveConfig::logger_name (C++ member), 108
 xynova::communication::SlaveConfig::type (C++ member), 108
 xynova::hand (C++ type), 113, 117
 xynova::hand::dexterous_hand (C++ type), 113, 117
 xynova::hand::dexterous_hand::AngleControl (C++ type), 118
 xynova::hand::dexterous_hand::AngleStates (C++ type), 118
 xynova::hand::dexterous_hand::ControlMode (C++ type), 118
 xynova::hand::dexterous_hand::ControlModeCommand (C++ type), 118
 xynova::hand::dexterous_hand::Device (C++ class), 113
 xynova::hand::dexterous_hand::Device::c_handle (C++ function), 116
 xynova::hand::dexterous_hand::Device::Device (C++ function), 113
 xynova::hand::dexterous_hand::Device::DeviceHandle (C++ type), 113
 xynova::hand::dexterous_hand::Device::enableRange (C++ function), 115

xynova::hand::dexterous_hand::Device::enableTactileTaxel (C++ function), 115
 xynova::hand::dexterous_hand::Device::enableTactileWrench (C++ function), 115
 xynova::hand::dexterous_hand::Device::getErrorState (C++ function), 114
 xynova::hand::dexterous_hand::Device::getHandType (C++ function), 114
 xynova::hand::dexterous_hand::Device::getHardwareVersion (C++ function), 114
 xynova::hand::dexterous_hand::Device::getJointAngle (C++ function), 115
 xynova::hand::dexterous_hand::Device::getProductModel (C++ function), 114
 xynova::hand::dexterous_hand::Device::getProductSN (C++ function), 114
 xynova::hand::dexterous_hand::Device::getRange (C++ function), 116
 xynova::hand::dexterous_hand::Device::getSoftwareVersion (C++ function), 114
 xynova::hand::dexterous_hand::Device::getTactileTaxel (C++ function), 116
 xynova::hand::dexterous_hand::Device::getTactileWrench (C++ function), 116
 xynova::hand::dexterous_hand::Device::i_dev_handle (C++ member), 116
 xynova::hand::dexterous_hand::Device::operator= (C++ function), 113
 xynova::hand::dexterous_hand::Device::run (C++ function), 113
 xynova::hand::dexterous_hand::Device::setJointAngle (C++ function), 115
 xynova::hand::dexterous_hand::Device::setJointAngleAsync (C++ function), 115
 xynova::hand::dexterous_hand::Device::setJointControlMode (C++ function), 114
 xynova::hand::dexterous_hand::Device::stop (C++ function), 113
 xynova::hand::dexterous_hand::Device::valid (C++ function), 113
 xynova::hand::dexterous_hand::ErrorCode (C++ type), 117
 xynova::hand::dexterous_hand::ErrorState (C++ struct), 120
 xynova::hand::dexterous_hand::ErrorState::fault_flags (C++ member), 121
 xynova::hand::dexterous_hand::ErrorState::hasFault (C++ function), 120
 xynova::hand::dexterous_hand::ErrorState::hasWarning (C++ function), 120
 xynova::hand::dexterous_hand::ErrorState::joint (C++ function), 120
 xynova::hand::dexterous_hand::ErrorState::joint_error_codes (C++ member), 121
 xynova::hand::dexterous_hand::ErrorState::timestamp_us (C++ member), 120
 xynova::hand::dexterous_hand::ErrorState::warning_flags (C++ member), 120
 xynova::hand::dexterous_hand::FaultFlags (C++ type), 117
 xynova::hand::dexterous_hand::HandType (C++ type), 117
 xynova::hand::dexterous_hand::JOINT_NUM (C++ member), 119
 xynova::hand::dexterous_hand::JointError (C++ enum), 119
 xynova::hand::dexterous_hand::JointError::Communication (C++ enumerator), 119
 xynova::hand::dexterous_hand::JointError::EncoderAbnormal (C++ enumerator), 119
 xynova::hand::dexterous_hand::JointError::MotorAbnormal (C++ enumerator), 119
 xynova::hand::dexterous_hand::JointError::None (C++ enumerator), 119
 xynova::hand::dexterous_hand::JointError::OverCurrent (C++ enumerator), 119
 xynova::hand::dexterous_hand::JointError::OverTemperature (C++ enumerator), 119
 xynova::hand::dexterous_hand::JointError::StallProtection (C++ enumerator), 119
 xynova::hand::dexterous_hand::JointErrorCode (C++ class), 119
 xynova::hand::dexterous_hand::JointErrorCode::fromC (C++ function), 119
 xynova::hand::dexterous_hand::JointErrorCode::has (C++ function), 120
 xynova::hand::dexterous_hand::JointErrorCode::i_code (C++ member), 120
 xynova::hand::dexterous_hand::JointErrorCode::JointErrorCode (C++ function), 119
 xynova::hand::dexterous_hand::JointErrorCode::ok (C++ function), 120
 xynova::hand::dexterous_hand::JointErrorCode::toC (C++ function), 119
 xynova::hand::dexterous_hand::JointErrorCode::value (C++ function), 120
 xynova::hand::dexterous_hand::JointErrorState (C++ struct), 120
 xynova::hand::dexterous_hand::JointErrorState::error_code (C++ member), 120
 xynova::hand::dexterous_hand::JointErrorState::id (C++ member), 120
 xynova::hand::dexterous_hand::JointID (C++ type), 117
 xynova::hand::dexterous_hand::JointValueArray (C++ type), 117
 xynova::hand::dexterous_hand::RANGE_POINT_NUM (C++ member), 119
 xynova::hand::dexterous_hand::RangePoint (C++ type), 118
 xynova::hand::dexterous_hand::RangePointStatus (C++ type), 118
 xynova::hand::dexterous_hand::RangeStates (C++ struct), 122
 xynova::hand::dexterous_hand::RangeStates::height (C++ member), 122
 xynova::hand::dexterous_hand::RangeStates::points (C++ member), 122
 xynova::hand::dexterous_hand::RangeStates::RangeStates (C++ function), 122
 xynova::hand::dexterous_hand::RangeStates::width (C++ member), 122
 xynova::hand::dexterous_hand::TACTILE_NUM (C++ member), 119
 xynova::hand::dexterous_hand::TACTILE_TOTAL_TAXEL_NUM (C++ member), 119
 xynova::hand::dexterous_hand::TactileID (C++ type), 117
 xynova::hand::dexterous_hand::TactileTaxel (C++ struct), 121
 xynova::hand::dexterous_hand::TactileTaxel::dimension (C++ member), 121
 xynova::hand::dexterous_hand::TactileTaxel::region (C++ member), 121
 xynova::hand::dexterous_hand::TactileTaxel::reserved (C++ member), 121

xynova::hand::dexterous_hand::TactileTaxel::taxel_id (C++ member), 121
 xynova::hand::dexterous_hand::TactileTaxel::timestamp_us (C++ member), 121
 xynova::hand::dexterous_hand::TactileTaxel::wrench (C++ member), 122
 xynova::hand::dexterous_hand::TactileTaxelStates (C++ type), 118
 xynova::hand::dexterous_hand::TactileWrench (C++ struct), 121
 xynova::hand::dexterous_hand::TactileWrench::dimension (C++ member), 121
 xynova::hand::dexterous_hand::TactileWrench::region (C++ member), 121
 xynova::hand::dexterous_hand::TactileWrench::timestamp_us (C++ member), 121
 xynova::hand::dexterous_hand::TactileWrench::wrench (C++ member), 121
 xynova::hand::dexterous_hand::TactileWrenchStates (C++ type), 118
 xynova::hand::dexterous_hand::WarningFlags (C++ type), 117
 xynova::hand::dexterous_hand::WrenchDimension (C++ type), 118
 xynova::infra (C++ type), 124, 126, 131
 xynova::infra::log (C++ type), 124, 126
 xynova::infra::log::ConsoleWriterConfiguration (C++ struct), 128
 xynova::infra::log::ConsoleWriterConfiguration::base_conf (C++ member), 128
 xynova::infra::log::ConsoleWriterConfiguration::use_stderr (C++ member), 128
 xynova::infra::log::FileWriterConfiguration (C++ struct), 128
 xynova::infra::log::FileWriterConfiguration::base_conf (C++ member), 128
 xynova::infra::log::FileWriterConfiguration::file_path (C++ member), 128
 xynova::infra::log::FileWriterConfiguration::max_file_size (C++ member), 128
 xynova::infra::log::FileWriterConfiguration::max_roll_files (C++ member), 128
 xynova::infra::log::FormaterConfiguration (C++ struct), 128
 xynova::infra::log::FormaterConfiguration::reserved (C++ member), 128
 xynova::infra::log::LoggerBackendConfiguration (C++ struct), 128
 xynova::infra::log::LoggerBackendConfiguration::block_size (C++ member), 129
 xynova::infra::log::LoggerBackendConfiguration::flush_interval_us (C++ member), 129
 xynova::infra::log::LoggerBackendConfiguration::max_continuous_repeated_log (C++ member), 129
 xynova::infra::log::LoggerBackendConfiguration::message_size (C++ member), 129
 xynova::infra::log::LoggerBackendConfiguration::pool_size (C++ member), 129
 xynova::infra::log::LoggerBackendConfiguration::queue_size_multiple (C++ member), 129
 xynova::infra::log::LoggerConfiguration (C++ struct), 129
 xynova::infra::log::LoggerConfiguration::lb_config (C++ member), 129
 xynova::infra::log::LoggerConfiguration::lb_type (C++ member), 129
 xynova::infra::log::LoggerConfiguration::level (C++ member), 129
 xynova::infra::log::LoggerConfiguration::sink_name (C++ member), 129
 xynova::infra::log::LoggerConfiguration::writer_name_list (C++ member), 129
 xynova::infra::log::LogManager (C++ class), 124
 xynova::infra::log::LogManager::~LogManager (C++ function), 126
 xynova::infra::log::LogManager::createConsoleWriter (C++ function), 124
 xynova::infra::log::LogManager::createFileWriter (C++ function), 124
 xynova::infra::log::LogManager::createFormater (C++ function), 125
 xynova::infra::log::LogManager::createLogger (C++ function), 125
 xynova::infra::log::LogManager::createSinkTP (C++ function), 124
 xynova::infra::log::LogManager::hasFormater (C++ function), 125
 xynova::infra::log::LogManager::hasLogger (C++ function), 125
 xynova::infra::log::LogManager::hasSink (C++ function), 125
 xynova::infra::log::LogManager::hasWriter (C++ function), 125
 xynova::infra::log::LogManager::initDefaultObjects (C++ function), 124
 xynova::infra::log::LogManager::instance (C++ function), 126
 xynova::infra::log::LogManager::isInitialized (C++ function), 124
 xynova::infra::log::LogManager::LogManager (C++ function), 126
 xynova::infra::log::LogManager::operator= (C++ function), 126
 xynova::infra::log::LogManager::setLoggerLevel (C++ function), 126
 xynova::infra::log::SinkConfiguration (C++ struct), 126
 xynova::infra::log::SinkConfiguration::block_obtain_timeout_us (C++ member), 126
 xynova::infra::log::SinkConfiguration::max_continuous_push_retry_time_ms (C++ member), 127
 xynova::infra::log::SinkConfiguration::max_looping_task_idle_interval_us (C++ member), 127
 xynova::infra::log::SinkConfiguration::min_looping_task_idle_interval_us (C++ member), 127
 xynova::infra::log::SinkConfiguration::writing_task_push_timeout_us (C++ member), 126
 xynova::infra::log::SinkConfiguration::writing_task_wait_timeout_us (C++ member), 127
 xynova::infra::log::SinkTPConfiguration (C++ struct), 127
 xynova::infra::log::SinkTPConfiguration::base_conf (C++ member), 127
 xynova::infra::log::SinkTPConfiguration::max_queue_size (C++ member), 127
 xynova::infra::log::SinkTPConfiguration::num_threads (C++ member), 127
 xynova::infra::log::WriterConfiguration (C++ struct), 127
 xynova::infra::log::WriterConfiguration::auto_flush (C++ member), 127
 xynova::infra::log::WriterConfiguration::flush_interval_ms (C++ member), 127

xynova::infra::log::WriterConfiguration::formatter (C++ member), 127
 xynova::infra::tools (C++ type), 131
 xynova::infra::tools::assignValue (C++ function), 131
 xynova::infra::tools::DataTypeTraits (C++ struct), 131
 xynova::infra::tools::DataTypeTraits::value (C++ member), 131
 xynova::infra::tools::IdValue (C++ class), 133
 xynova::infra::tools::IdValue::fromC (C++ function), 134
 xynova::infra::tools::IdValue::i_id (C++ member), 134
 xynova::infra::tools::IdValue::i_value (C++ member), 134
 xynova::infra::tools::IdValue::id (C++ function), 133
 xynova::infra::tools::IdValue::IdValue (C++ function), 133
 xynova::infra::tools::IdValue::setId (C++ function), 133
 xynova::infra::tools::IdValue::setValue (C++ function), 133
 xynova::infra::tools::IdValue::Storage (C++ type), 133
 xynova::infra::tools::IdValue::toC (C++ function), 134
 xynova::infra::tools::IdValue::value (C++ function), 133, 134
 xynova::infra::tools::IdValue::ValueType (C++ type), 133
 xynova::infra::tools::readValue (C++ function), 131
 xynova::infra::tools::Value (C++ class), 131
 xynova::infra::tools::Value::fromC (C++ function), 132
 xynova::infra::tools::Value::i_value (C++ member), 133
 xynova::infra::tools::Value::setValue (C++ function), 132
 xynova::infra::tools::Value::Storage (C++ type), 132
 xynova::infra::tools::Value::toC (C++ function), 132
 xynova::infra::tools::Value::Value (C++ function), 132
 xynova::infra::tools::Value::value (C++ function), 132
 XYNOVA_API (C macro), 80
 XYNOVA_CONSTEXPR (C macro), 80
 XYNOVA_DEFINE_DATA_TYPE_TRAIT (C macro), 131
 XYNOVA_DEXTEROUS_HAND_JOINT_NUM (C macro), 65
 XYNOVA_DEXTEROUS_HAND_RANGE_POINT_NUM (C macro), 65
 XYNOVA_DEXTEROUS_HAND_TACTILE_NUM (C macro), 65
 XYNOVA_DEXTEROUS_HAND_TACTILE_TOTAL_TAXEL_NUM (C macro), 65
 XYNOVA_ERROR_CODE_LIST (C macro), 81
 XYNOVA_EXPORT_TEMPLATE (C macro), 81
 XYNOVA_IMPL_API (C macro), 80
 XYNOVA_INLINE_API (C macro), 80
 XYNOVA_LOCAL (C macro), 80
 XYNOVA_NOEXCEPT (C macro), 80
 XYNOVA_SDK_BUILD_TIME (C macro), 90
 XYNOVA_SDK_GIT_BRANCH (C macro), 89
 XYNOVA_SDK_GIT_COMMIT (C macro), 90
 XYNOVA_SDK_LOCAL_VERSION (C macro), 89
 XYNOVA_SDK_VERSION (C macro), 89
 XynovaBool (C++ type), 79
 XynovaCommunicationCan_registerDriver (C++ function), 53
 XynovaCommunicationCanConfig (C++ struct), 55
 XynovaCommunicationCanConfig (C++ type), 54
 XynovaCommunicationCanConfig::bitrate (C++ member), 55
 XynovaCommunicationCanConfig::dbitrate (C++ member), 56
 XynovaCommunicationCanConfig::dsample_point (C++ member), 56
 XynovaCommunicationCanConfig::iface (C++ member), 55
 XynovaCommunicationCanConfig::manager_can (C++ member), 55
 XynovaCommunicationCanConfig::sample_point (C++ member), 56
 XynovaCommunicationCanFrameType (C++ type), 54
 XynovaCommunicationCanMask (C++ type), 54
 XynovaCommunicationCanRecvFilter (C++ struct), 56
 XynovaCommunicationCanRecvFilter (C++ type), 54
 XynovaCommunicationCanRecvFilter::id (C++ member), 56
 XynovaCommunicationCanRecvFilter::mask (C++ member), 56
 XynovaCommunicationCanRecvFilter::type (C++ member), 56
 XynovaCommunicationCanVTable (C++ struct), 56
 XynovaCommunicationCanVTable (C++ type), 54
 XynovaCommunicationCanVTable::close (C++ member), 57
 XynovaCommunicationCanVTable::create (C++ member), 57
 XynovaCommunicationCanVTable::destroy (C++ member), 57
 XynovaCommunicationCanVTable::isOpen (C++ member), 57
 XynovaCommunicationCanVTable::open (C++ member), 57

XynovaCommunicationCanVTable::recv (C++ member), 58
 XynovaCommunicationCanVTable::send (C++ member), 58
 XynovaCommunicationCanVTable::setRecvFilters (C++ member), 57
 XynovaCommunicationCommConfig (C++ struct), 49
 XynovaCommunicationCommConfig (C++ type), 48
 XynovaCommunicationCommConfig::can (C++ member), 49
 XynovaCommunicationCommConfig::config (C++ member), 49
 XynovaCommunicationCommConfig::logger_name (C++ member), 49
 XynovaCommunicationCommConfig::sub_type (C++ member), 49
 XynovaCommunicationCommConfig::type (C++ member), 49
 XynovaCommunicationCommType (C++ type), 48
 XynovaCommunicationDeviceConfig (C++ struct), 51
 XynovaCommunicationDeviceConfig (C++ type), 49
 XynovaCommunicationDeviceConfig::command_count (C++ member), 52
 XynovaCommunicationDeviceConfig::commands (C++ member), 52
 XynovaCommunicationDeviceConfig::master_name (C++ member), 51
 XynovaCommunicationDeviceConfig::name (C++ member), 51
 XynovaCommunicationDeviceConfig::slave_count (C++ member), 52
 XynovaCommunicationDeviceConfig::slaves (C++ member), 51
 XynovaCommunicationDeviceConfig::type (C++ member), 51
 XynovaCommunicationManager_freeDevice (C++ function), 47
 XynovaCommunicationManager_getDevice (C++ function), 47
 XynovaCommunicationManager_init (C++ function), 47
 XynovaCommunicationManager_run (C++ function), 47
 XynovaCommunicationManager_stop (C++ function), 47
 XynovaCommunicationManager_upgradeFirmware (C++ function), 48
 XynovaCommunicationManagerConfig (C++ struct), 52
 XynovaCommunicationManagerConfig (C++ type), 49
 XynovaCommunicationManagerConfig::device_count (C++ member), 52
 XynovaCommunicationManagerConfig::devices (C++ member), 52
 XynovaCommunicationManagerConfig::master_count (C++ member), 52
 XynovaCommunicationManagerConfig::masters (C++ member), 52
 XynovaCommunicationMasterConfig (C++ struct), 50
 XynovaCommunicationMasterConfig (C++ type), 48
 XynovaCommunicationMasterConfig::comm (C++ member), 50
 XynovaCommunicationMasterConfig::cpu_index (C++ member), 50
 XynovaCommunicationMasterConfig::name (C++ member), 50
 XynovaCommunicationMasterConfig::period_us (C++ member), 50
 XynovaCommunicationResponseTimeSetting (C++ struct), 50
 XynovaCommunicationResponseTimeSetting (C++ type), 49
 XynovaCommunicationResponseTimeSetting::execute_max_retry (C++ member), 50
 XynovaCommunicationResponseTimeSetting::execute_poll_interval_us (C++ member), 50
 XynovaCommunicationResponseTimeSetting::execute_timeout_us (C++ member), 50
 XynovaCommunicationResponseTimeSetting::user_max_timeout_us (C++ member), 50
 XynovaCommunicationResponseTimeSetting::user_poll_interval_us (C++ member), 50
 XynovaCommunicationSchedulingCommand (C++ struct), 51
 XynovaCommunicationSchedulingCommand (C++ type), 49
 XynovaCommunicationSchedulingCommand::command (C++ member), 51
 XynovaCommunicationSchedulingCommand::response_time_setting (C++ member), 51
 XynovaCommunicationSlaveConfig (C++ struct), 50
 XynovaCommunicationSlaveConfig (C++ type), 49
 XynovaCommunicationSlaveConfig::fast_time_setting (C++ member), 51
 XynovaCommunicationSlaveConfig::general_time_setting (C++ member), 51
 XynovaCommunicationSlaveConfig::id (C++ member), 51
 XynovaCommunicationSlaveConfig::logger_name (C++ member), 51
 XynovaCommunicationSlaveConfig::type (C++ member), 51
 XynovaDataType (C++ type), 83
 XynovaDeviceHandle (C++ type), 47
 XynovaDexterousHand_enableRange (C++ function), 62
 XynovaDexterousHand_enableTactileTaxel (C++ function), 62
 XynovaDexterousHand_enableTactileWrench (C++ function), 62
 XynovaDexterousHand_getErrorState (C++ function), 61
 XynovaDexterousHand_getHandType (C++ function), 61
 XynovaDexterousHand_getHardwareVersion (C++ function), 60
 XynovaDexterousHand_getJointAngle (C++ function), 63
 XynovaDexterousHand_getProductModel (C++ function), 60
 XynovaDexterousHand_getProductSN (C++ function), 60
 XynovaDexterousHand_getRange (C++ function), 64

XynovaDexterousHand_getSoftwareVersion (C++ function), 61
 XynovaDexterousHand_getTactileTaxel (C++ function), 64
 XynovaDexterousHand_getTactileWrench (C++ function), 63
 XynovaDexterousHand_run (C++ function), 59
 XynovaDexterousHand_setJointAngle (C++ function), 63
 XynovaDexterousHand_setJointAngle_async (C++ function), 63
 XynovaDexterousHand_setJointControlAllMode (C++ function), 61
 XynovaDexterousHand_stop (C++ function), 60
 XynovaDexterousHandAngleControl (C++ struct), 76
 XynovaDexterousHandAngleControl (C++ type), 67
 XynovaDexterousHandAngleControl::values (C++ member), 76
 XynovaDexterousHandAngleStates (C++ struct), 76
 XynovaDexterousHandAngleStates (C++ type), 67
 XynovaDexterousHandAngleStates::values (C++ member), 76
 XynovaDexterousHandControlMode (C++ type), 66
 XynovaDexterousHandControlModeCommand (C++ struct), 75
 XynovaDexterousHandControlModeCommand (C++ type), 66
 XynovaDexterousHandControlModeCommand::values (C++ member), 76
 XynovaDexterousHandErrorCode (C++ type), 65
 XynovaDexterousHandErrorState (C++ struct), 75
 XynovaDexterousHandErrorState (C++ type), 66
 XynovaDexterousHandErrorState::fault_flags (C++ member), 75
 XynovaDexterousHandErrorState::joint_error_code (C++ member), 75
 XynovaDexterousHandErrorState::timestamp_us (C++ member), 75
 XynovaDexterousHandErrorState::warning_flags (C++ member), 75
 XynovaDexterousHandFaultFlags (C++ type), 66
 XynovaDexterousHandJointID (C++ type), 66
 XynovaDexterousHandRangePoint (C++ struct), 77
 XynovaDexterousHandRangePoint (C++ type), 67
 XynovaDexterousHandRangePoint::distance (C++ member), 78
 XynovaDexterousHandRangePoint::id (C++ member), 78
 XynovaDexterousHandRangePoint::reserved (C++ member), 78
 XynovaDexterousHandRangePoint::status (C++ member), 78
 XynovaDexterousHandRangePoint::timestamp_us (C++ member), 78
 XynovaDexterousHandRangePointStatus (C++ type), 66
 XynovaDexterousHandRangeStates (C++ struct), 78
 XynovaDexterousHandRangeStates (C++ type), 67
 XynovaDexterousHandRangeStates::count (C++ member), 78
 XynovaDexterousHandRangeStates::height (C++ member), 78
 XynovaDexterousHandRangeStates::points (C++ member), 78
 XynovaDexterousHandRangeStates::width (C++ member), 78
 XynovaDexterousHandTactileID (C++ type), 66
 XynovaDexterousHandTactileTaxel (C++ struct), 77
 XynovaDexterousHandTactileTaxel (C++ type), 67
 XynovaDexterousHandTactileTaxel::dimension (C++ member), 77
 XynovaDexterousHandTactileTaxel::region (C++ member), 77
 XynovaDexterousHandTactileTaxel::reserved (C++ member), 77
 XynovaDexterousHandTactileTaxel::taxel_id (C++ member), 77
 XynovaDexterousHandTactileTaxel::timestamp_us (C++ member), 77
 XynovaDexterousHandTactileTaxel::wrench (C++ member), 77
 XynovaDexterousHandTactileTaxelStates (C++ struct), 77
 XynovaDexterousHandTactileTaxelStates (C++ type), 67
 XynovaDexterousHandTactileTaxelStates::count (C++ member), 77
 XynovaDexterousHandTactileTaxelStates::taxels (C++ member), 77
 XynovaDexterousHandTactileWrench (C++ struct), 76
 XynovaDexterousHandTactileWrench (C++ type), 67
 XynovaDexterousHandTactileWrench::dimension (C++ member), 76
 XynovaDexterousHandTactileWrench::region (C++ member), 76
 XynovaDexterousHandTactileWrench::timestamp_us (C++ member), 76
 XynovaDexterousHandTactileWrench::wrench (C++ member), 76
 XynovaDexterousHandTactileWrenchStates (C++ struct), 77
 XynovaDexterousHandTactileWrenchStates (C++ type), 67
 XynovaDexterousHandTactileWrenchStates::values (C++ member), 77
 XynovaDexterousHandType (C++ type), 66
 XynovaDexterousHandWarningFlags (C++ type), 65
 XynovaDexterousHandWrenchDimension (C++ type), 66
 XynovaErrorCode (C++ type), 81
 xynovaErrorCodeToString (C++ function), 82

xynovaGetSdkBuildTime (C++ function), 90
 xynovaGetSdkGitBranch (C++ function), 90
 xynovaGetSdkGitCommit (C++ function), 90
 xynovaGetSdkLocalVersion (C++ function), 90
 xynovaGetSdkVersion (C++ function), 90
 XynovaIdValue (C++ struct), 88
 XynovaIdValue (C++ type), 84
 XynovaIdValue::data (C++ member), 88
 XynovaIdValue::id (C++ member), 88
 XynovaIdValueArray2 (C++ struct), 88
 XynovaIdValueArray2 (C++ type), 84
 XynovaIdValueArray2::data (C++ member), 88
 XynovaIdValueArray2::id (C++ member), 88
 XynovaIdValueArray3 (C++ struct), 88
 XynovaIdValueArray3 (C++ type), 84
 XynovaIdValueArray3::data (C++ member), 88
 XynovaIdValueArray3::id (C++ member), 88
 XynovaIdValueArray4 (C++ struct), 88
 XynovaIdValueArray4 (C++ type), 84
 XynovaIdValueArray4::data (C++ member), 89
 XynovaIdValueArray4::id (C++ member), 89
 XynovaIdValueArray5 (C++ struct), 89
 XynovaIdValueArray5 (C++ type), 84
 XynovaIdValueArray5::data (C++ member), 89
 XynovaIdValueArray5::id (C++ member), 89
 XynovaIdValueArray6 (C++ struct), 89
 XynovaIdValueArray6 (C++ type), 84
 XynovaIdValueArray6::data (C++ member), 89
 XynovaIdValueArray6::id (C++ member), 89
 XynovaLogConsoleWriterConfiguration (C++ struct), 98
 XynovaLogConsoleWriterConfiguration (C++ type), 94
 XynovaLogConsoleWriterConfiguration::base_conf (C++ member), 98
 XynovaLogConsoleWriterConfiguration::use_stderr (C++ member), 98
 XynovaLogDateTimeRaw (C++ struct), 95
 XynovaLogDateTimeRaw (C++ type), 94
 XynovaLogDateTimeRaw::microseconds (C++ member), 95
 XynovaLogDateTimeRaw::seconds (C++ member), 95
 XynovaLogFileWriterConfiguration (C++ struct), 97
 XynovaLogFileWriterConfiguration (C++ type), 94
 XynovaLogFileWriterConfiguration::base_conf (C++ member), 98
 XynovaLogFileWriterConfiguration::file_path (C++ member), 98
 XynovaLogFileWriterConfiguration::max_file_size (C++ member), 98
 XynovaLogFileWriterConfiguration::max_roll_files (C++ member), 98
 XynovaLogFormatterConfiguration (C++ struct), 98
 XynovaLogFormatterConfiguration (C++ type), 94
 XynovaLogFormatterConfiguration::reserved (C++ member), 98
 XynovaLogGlobalLogger_getLogLevel (C++ function), 91
 XynovaLogGlobalLogger_init (C++ function), 91
 XynovaLogGlobalLogger_setLogLevel (C++ function), 91
 XynovaLogLevel (C++ type), 94
 XynovaLogLoggerBackendConfiguration (C++ struct), 98
 XynovaLogLoggerBackendConfiguration (C++ type), 94
 XynovaLogLoggerBackendConfiguration::block_size (C++ member), 98
 XynovaLogLoggerBackendConfiguration::flush_interval_us (C++ member), 99
 XynovaLogLoggerBackendConfiguration::max_continuous_repeated_log (C++ member), 99
 XynovaLogLoggerBackendConfiguration::message_size (C++ member), 99
 XynovaLogLoggerBackendConfiguration::pool_size (C++ member), 98
 XynovaLogLoggerBackendConfiguration::queue_size_multiple (C++ member), 99
 XynovaLogLoggerConfiguration (C++ struct), 99
 XynovaLogLoggerConfiguration (C++ type), 95
 XynovaLogLoggerConfiguration::lb_config (C++ member), 99
 XynovaLogLoggerConfiguration::lb_type (C++ member), 99
 XynovaLogLoggerConfiguration::level (C++ member), 99
 XynovaLogLoggerConfiguration::sink_name (C++ member), 99
 XynovaLogLoggerConfiguration::writer_name_list (C++ member), 99
 XynovaLogLogHandler (C++ type), 94
 XynovaLogLogRecord (C++ struct), 96
 XynovaLogLogRecord (C++ type), 94

XynovaLogLogRecord::message (C++ member), 96
 XynovaLogLogRecord::meta (C++ member), 96
 XynovaLogLogRecordMeta (C++ struct), 95
 XynovaLogLogRecordMeta (C++ type), 94
 XynovaLogLogRecordMeta::file (C++ member), 96
 XynovaLogLogRecordMeta::func (C++ member), 96
 XynovaLogLogRecordMeta::level (C++ member), 96
 XynovaLogLogRecordMeta::line (C++ member), 96
 XynovaLogLogRecordMeta::thread_id (C++ member), 96
 XynovaLogLogRecordMeta::timestamp (C++ member), 96
 XynovaLogObjectLogger_createConsoleWriter (C++ function), 92
 XynovaLogObjectLogger_createFileWriter (C++ function), 92
 XynovaLogObjectLogger_createFormatter (C++ function), 92
 XynovaLogObjectLogger_createLogger (C++ function), 92
 XynovaLogObjectLogger_createSinkTP (C++ function), 91
 XynovaLogObjectLogger_hasFormatter (C++ function), 93
 XynovaLogObjectLogger_hasLogger (C++ function), 93
 XynovaLogObjectLogger_hasSink (C++ function), 93
 XynovaLogObjectLogger_hasWriter (C++ function), 93
 XynovaLogObjectLogger_initDefaultObjects (C++ function), 91
 XynovaLogObjectLogger_isInitialized (C++ function), 91
 XynovaLogObjectLogger_setLogLevel (C++ function), 93
 XynovaLogSinkConfiguration (C++ struct), 96
 XynovaLogSinkConfiguration (C++ type), 94
 XynovaLogSinkConfiguration::block_obtain_timeout_us (C++ member), 96
 XynovaLogSinkConfiguration::max_continuous_push_retry_time_ms (C++ member), 97
 XynovaLogSinkConfiguration::max_looping_task_idle_interval_us (C++ member), 97
 XynovaLogSinkConfiguration::min_looping_task_idle_interval_us (C++ member), 97
 XynovaLogSinkConfiguration::writing_task_push_timeout_us (C++ member), 96
 XynovaLogSinkConfiguration::writing_task_wait_timeout_us (C++ member), 97
 XynovaLogSinkTPConfiguration (C++ struct), 97
 XynovaLogSinkTPConfiguration (C++ type), 94
 XynovaLogSinkTPConfiguration::base_conf (C++ member), 97
 XynovaLogSinkTPConfiguration::max_queue_size (C++ member), 97
 XynovaLogSinkTPConfiguration::num_threads (C++ member), 97
 XynovaLogWriterConfiguration (C++ struct), 97
 XynovaLogWriterConfiguration (C++ type), 94
 XynovaLogWriterConfiguration::auto_flush (C++ member), 97
 XynovaLogWriterConfiguration::flush_interval_ms (C++ member), 97
 XynovaLogWriterConfiguration::formatter (C++ member), 97
 XynovaResult (C++ struct), 82
 XynovaResult (C++ type), 82
 XynovaResult::code (C++ member), 83
 XynovaResult::message (C++ member), 83
 XynovaTypedValue (C++ struct), 86
 XynovaTypedValue (C++ type), 83
 XynovaTypedValue::type (C++ member), 86
 XynovaTypedValue::value (C++ member), 86
 XynovaTypedValueArray2 (C++ struct), 86
 XynovaTypedValueArray2 (C++ type), 83
 XynovaTypedValueArray2::type (C++ member), 87
 XynovaTypedValueArray2::values (C++ member), 87
 XynovaTypedValueArray3 (C++ struct), 87
 XynovaTypedValueArray3 (C++ type), 84
 XynovaTypedValueArray3::type (C++ member), 87
 XynovaTypedValueArray3::values (C++ member), 87
 XynovaTypedValueArray4 (C++ struct), 87
 XynovaTypedValueArray4 (C++ type), 84
 XynovaTypedValueArray4::type (C++ member), 87
 XynovaTypedValueArray4::values (C++ member), 87
 XynovaTypedValueArray5 (C++ struct), 87
 XynovaTypedValueArray5 (C++ type), 84
 XynovaTypedValueArray5::type (C++ member), 87
 XynovaTypedValueArray5::values (C++ member), 87
 XynovaTypedValueArray6 (C++ struct), 87
 XynovaTypedValueArray6 (C++ type), 84
 XynovaTypedValueArray6::type (C++ member), 88
 XynovaTypedValueArray6::values (C++ member), 88

XynovaValue (C++ *type*), [83](#)
XynovaValue (C++ *union*), [85](#)
XynovaValue::f32 (C++ *member*), [86](#)
XynovaValue::f64 (C++ *member*), [86](#)
XynovaValue::s16 (C++ *member*), [86](#)
XynovaValue::s32 (C++ *member*), [86](#)
XynovaValue::s64 (C++ *member*), [86](#)
XynovaValue::s8 (C++ *member*), [85](#)
XynovaValue::u16 (C++ *member*), [86](#)
XynovaValue::u32 (C++ *member*), [86](#)
XynovaValue::u64 (C++ *member*), [86](#)
XynovaValue::u8 (C++ *member*), [85](#)